



Cairo University

Faculty of Computers and Artificial Intelligence

Information Systems Department



SHATLA

Presented by

<i>Ahmed Nabil Abdel-Hady</i>	<i>20210043</i>
<i>Abdelrahman Hossam Eldin Ahmed</i>	<i>20210201</i>
<i>Mohamed Kamal Mohamed</i>	<i>20210350</i>
<i>Youssef Mahmoud Okasha</i>	<i>20210486</i>
<i>Youssef Ezzat Mohamed</i>	<i>20210542</i>

Under the Supervisors of:

Dr. Mohamed Noor El-Dien

TA. Doaa Ghaleb

Department of Information Systems, Faculty of Computers and Artificial Intelligence, Cairo University,
Egypt

July 2025

Acknowledgments

We extend our deepest gratitude to the individuals who have contributed to the success of this project. First, we thank **Dr. Mohammed Noor El-Dien** for his invaluable guidance, mentorship, and expertise. His insightful feedback and unwavering support have significantly shaped this project from inception to completion.

Our heartfelt appreciation also goes to **Doaa Ghaleb**, our Teaching Assistant, whose dedicated assistance and constructive feedback were essential to the development of this project. Her encouragement and expertise created a positive and enriching learning environment, helping us navigate challenges effectively.

We cannot forget about our colleagues who can be considered as mentors for our AI section of the project. Special Thanks to **Nabil Sherif**, **Shهاب Diab**, and **Omar Elgammal**. They spent their time guiding the team on the AI technologies available and how to solve the proposed problems.

We would like to extend our sincere thanks to the department's esteemed faculty members for their continuous support and academic insights. Their encouragement and belief in our abilities have been key motivators throughout the project's development.

Additionally, we are deeply grateful to our families for their unwavering support, understanding, and encouragement throughout this journey. Their patience, belief in us, and constant motivation have played a vital role in our success.

This project would not have been possible without these individuals' collective efforts, guidance, and support, and we appreciate their contributions to this journey.

Abstract

The **Shatla** project addresses the need for an innovative mobile application that assists home gardeners in Egypt with the efficient care and management of their plants. Existing gardening tools often lack the necessary features to address plant care challenges specific to climate, soil quality, and plant health, driving the development of **Shatla**. This app sets a new standard by prioritizing user experience, accuracy in plant care, and personalized solutions through advanced image detection and plant analysis technologies.

Shatla aims to streamline the process of plant management by offering comprehensive care schedules, disease detection, and solutions for common problems faced by gardeners. To enhance convenience and functionality, the app integrates external services, including marketplace for purchasing seeds and farming tools, image recognition for plant health monitoring, and notifications for timely plant care.

Key features of the app include user management, robust plant search and browsing, personalized plant care schedules, and a community-driven marketplace. **Shatla's** ability to identify plant diseases and suggest care solutions contributes to the overall user experience, ensuring that users can maintain healthy plants with ease. The platform's design also allows for scalability, making it adaptable for future expansions in features and services.

Shatla offers a tailored solution to individual gardeners and plant enthusiasts seeking a dependable, informative, and user-friendly platform for plant care. The project's outcomes are significant in promoting green spaces, improving plant health, and encouraging sustainable gardening practices. By introducing advanced care tools and fostering a community of gardeners, **Shatla** contributes to the evolution of plant care technologies and practices.

Table of Contents

Chapter 1 Introduction	10
1.1 Background and Motivation	10
1.2 Problem Statement	10
1.3 Objectives	11
1.4 Project Scope and Limitations.....	11
1.5 Project Methodology	13
1.5.1 Flexibility and Adaptability:	13
1.5.2 Continuous Improvement:	13
1.5.3 Enhanced Collaboration:	14
1.5.4 Faster Time-to-Market:	14
Agile-Scrum Methodology in Action	14
1.6 Project Report Outline	15
Chapter 2 presents the literature.	15
Chapter 3 presents the analysis.....	15
Chapter 4 presents System Design and Implementation.....	15
Chapter 5 presents System Testing and Evaluation.....	15
Chapter 6 Conclusions and Future Work	15
2.1 Overview of Current Plant Care Apps	16
2.2 Related Work: Key Applications in the Market.....	16
2.2.1 Picture This	16
2.2.2 Plantum	17
2.2.3 Garden Answers	17
2.2.4 Blossom.....	18
2.2.5 PlantSnap.....	19
2.2.6 Mashtal and BthRah (Local Competitors)	19
2.3 Comparative Analysis of Existing Solutions	20
2.4 Gaps in the Market.....	20
2.5 How Shatla Solves These Issues.....	21
2.6 Conclusion and Market Potential	21
Chapter 3 Shatla Analysis	22
3.1 Main Problem	22

3.2	Functional Requirements	22
3.3	Non-Functional Requirements	47
3.3.1	Usability	47
3.3.2	Security	48
3.3.3	Maintainability	49
3.3.4	Performance	50
3.4	Stakeholders	51
	User	51
	Admin	51
3.5	System Scenarios	52
	User	52
	Admin	53
3.7	Use Case Tables	55
3.7.1	User Account Management	55
3.7.2	Garden and Plant Management	60
3.7.3	Plant Diagnosis and Recommendations	70
3.7.4	Care Schedule and Notifications	76
3.7.5	User Profile Management	82
3.7.6	E-Commerce Features	85
3.7.7	AI	90
3.7.8	Climate Data	94
3.7.9	Additional Functionalities	96
3.8	Class Diagram	100
3.9	Sequence Diagrams	104
3.10	ERD	107
Chapter 4 Shatla Design and Implementation		108
4.1	System Architecture	108
4.2	Prototype Screens	111
4.2.1	Mobile Screen	111
4.2.2	Web Screen	159
4.2.3	Admin Screen	167
4.3	Tools and Programming Languages with Selection Reasons	169
Chapter 5 System Testing and Evaluation		177
5.1	Ecommerce Package	177

5.2	Plant Management Package	191
5.3	Care & Task Management Package	209
5.4	Notifications Package	215
Chapter 6 Conclusions and Future Work		216
6.1	Supporting More Plants for Detection and Care Tasks.	216
6.2	Implementing Complete Purchasing Procedure.	216
6.3	Crafting Complete Premium Experience.	216
6.4	Integration with LOT Devices for smart plant monitoring	217
6.5	Multilingual Support	217
6.6	Community Forum & Plant Swap	217
6.7	AI-Powered Shatla Rule-Based System.....	217
6.8	Pruning Care Task.....	218
6.8	Customer Support Tickets	218

List of Figures

Figure 1 Project Plan.....	12
Figure 2 Use Case	54
Figure 3 Package System.....	100
Figure 4 Authentication Package	100
Figure 5 Diagnosis Package	101
Figure 6 Climate Package.....	101
Figure 7 Plant Gardener Package.....	102
Figure 8 Care Scheduler Package	102
Figure 9 E-commerce Package	103
Figure 10 Notification package	103
Figure 11 User Package.....	103
Figure 12 Disease Diagnosis Sequence.....	104
Figure 13 Disease Diagnosis Sequence Analysis.....	105
Figure 14 Care Schedule Sequence	105
Figure 15 Ecommerce Shopping Sequence	106
Figure 16 Ecommerce Shopping Sequence Analysis	106
Figure 17 ERD.....	107
Figure 18 System Architecture	109
Figure 19 Diagnosis Workflow	110

List of Tables

Table 1 Related Work.....	20
Table 2 Usability Requirements for Shatla Application	47
Table 3 Security Requirements for Shatla Application	48
Table 4 Maintainability Requirements for the Shatla Application	49
Table 5 Performance Metrics Requirements for Shatla Application.....	50

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
DDD	Domain Driven Design
GPU	Graphics Processing Unit
IDE	Integrated Development Environment
JWT	JSON Web Token
NLP	Natural Language Processing
RAM	Random Access Memory
UML	Unified Modeling Language
UI	User interface
URL	Uniform Resource locator

Chapter 1 Introduction

1.1 Background and Motivation

The evolution of green life matters has transformed how people treat and care for plants. Planting trees and even flowers at home are recommended to absorb carbon dioxide and help save the environment. However, many houseplants will often die due to a lack of care and information about their needs. For that reason, “**Shatla**” aims to educate home gardeners by providing essential information on plant care, including watering, pruning, and sunlight requirements.

The motivation behind “**Shatla**” is to provide a more engaging and transparent environment for users to care for their plants. It allows them to access personalized care schedules, general instructions, and information for each plant.

1.2 Problem Statement

The current plant care mobile applications do not address the specific needs of home gardeners, with appropriate care schedules and the steps required to save the plants. Also, they lack region-specific advice for each plant, and some applications offer limited image detection features for identifying diseases.

“**Shatla**” seeks to overcome these challenges by providing a comprehensive plant care schedule that outlines the required steps for watering, fertilizing, and placing each plant in the sun. Also, it gives region-specific advice for each home plant. Additionally, it offers a free image detection feature for all cared-for plants. Also, it has a reminder section for all tasks required for each plant in each garden. Finally, an e-commerce

section within the application to buy plants and tools for each home gardener, along with some recommendations based on their regional data and recorded cared-for plants.

1.3 Objectives

The objectives of the project encompass both tangible and intangible goals. Tangible objectives include providing a user-friendly application that users can rely on to care for their plants. Intangible objectives include enhancing user trust through a reliable mobile platform that satisfies all his needs. The project aims to provide a complete platform for all users to save their plants.

1.4 Project Scope and Limitations

The "**Shatla**" app is designed to assist home gardeners by identifying plant species and detecting plant diseases based on photos submitted by users. Currently, detection and diagnosis capabilities are limited to six plant species due to data availability constraints in the existing dataset, which restricts the range of supported diagnoses. Despite this limitation, the app will provide a personalized care schedule and general plant care guidance based on the analysis results of the detected plant species.

In addition to disease detection, the **Shatla** app will feature an e-commerce section where users can access plants, tools, and other gardening essentials. However, while basic payment procedures have been implemented, integration with external banking systems is excluded from the current scope due to operational constraints and is reserved for future development.

Finally, **Shatla** will include an AI section featuring a plant chatbot, camera-based plant detection, and a recommendation system for the plants available in the system. Currently, plant identification supports only six predefined plant types, with plans to expand this list in future updates.

The application will be developed using Nest.js as the backend framework and Flutter for the mobile front end, chosen to ensure stable, responsive, and scalable architecture. Data for plant disease detection will be sourced from publicly available datasets on Kaggle, while general plant care information will be retrieved from public APIs.

The design and functionality will undergo iterative user testing to gather feedback and refine the user experience. This approach aims to ensure that the app meets the needs of home gardeners, providing a platform that fosters plant health and offers practical tools for plant care. While the disease detection feature is initially limited to plant variety, the app offers a supportive form-based diagnostic tool, enabling users to effectively identify and care for supported plant species.

To view Clearer pictures, visit this link [Shatla Diagrams](#).

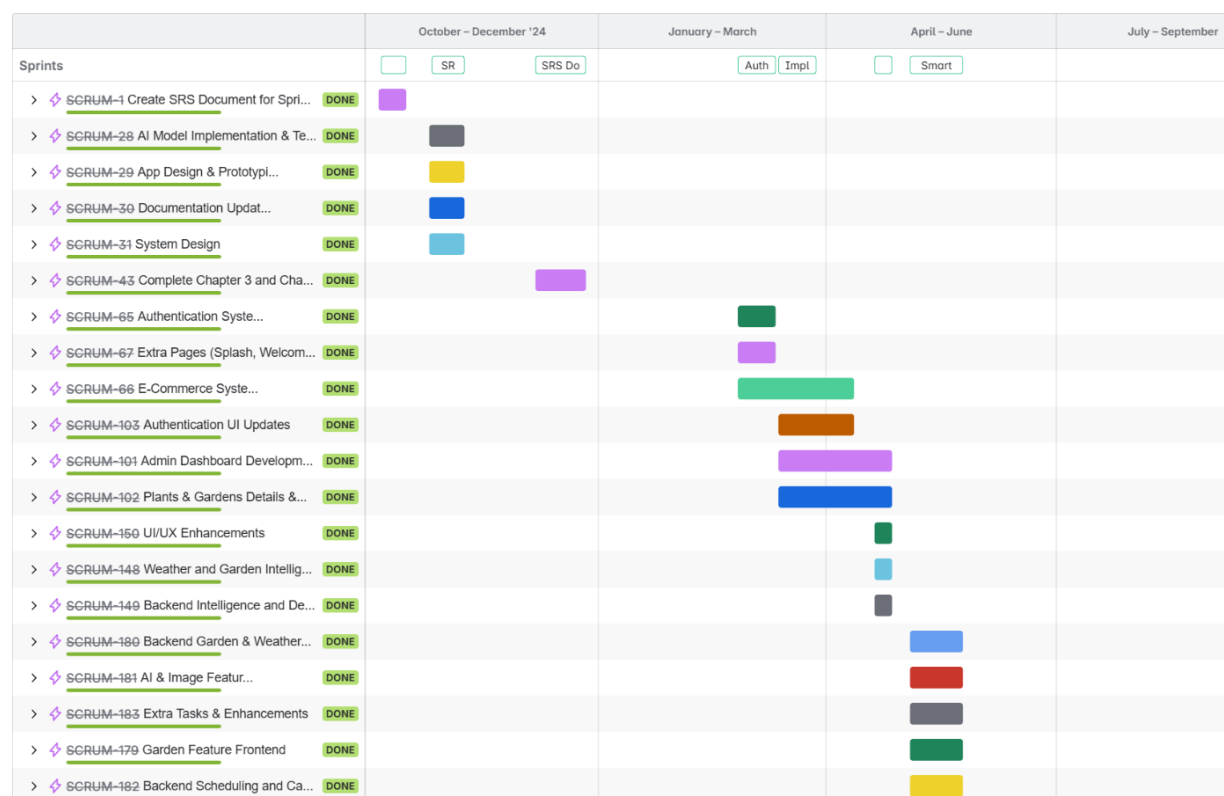


Figure 1 Project Plan

1.5 Project Methodology

The **Agile** methodology, specifically using the Scrum framework, has been adopted for this project. Agile is recognized for its structured yet flexible approach, promoting continuous improvement and collaboration through iterative cycles, known as sprints, to gradually refine and enhance the application's development.

The Scrum framework was chosen for the following reasons:

1.5.1 Flexibility and Adaptability:

Agile-Scrum methodology allows for a highly adaptive environment where project requirements can evolve based on user feedback, market changes, or new insights. Scrum's sprint-based structure ensures that the team can respond to adjustments quickly, making the final product relevant to both user needs and market conditions.

1.5.2 Continuous Improvement:

Scrum emphasizes iterative progress, with each sprint cycle (lasting two to four weeks) incorporating planning, development, and review. After each sprint, team members assess progress and identify improvement areas, making continuous adjustments to enhance the product's quality and alignment with user expectations.

1.5.3 Enhanced Collaboration:

Scrum promotes collaborative teamwork, where team members regularly communicate through daily stand-ups, sprint reviews, and retrospectives. This fosters shared ownership, knowledge sharing, and accountability, improving team efficiency and overall satisfaction.

1.5.4 Faster Time-to-Market:

Scrum's sprint-based delivery allows functional features to be released in shorter cycles, meaning core features of the Shatla app are made available to users earlier. This approach facilitates quicker time-to-market compared to traditional methodologies, allowing the team to receive and act on user feedback more rapidly.

Agile-Scrum Methodology in Action

For the Shatla project, work is divided into smaller, manageable sprints, each lasting two to four weeks. Each sprint cycle includes phases for planning, development, testing, and reviewing. Feedback from users or stakeholders is collected at the end of each sprint, guiding improvements and refinements in the following cycles. By implementing Agile Scrum, the Shatla project ensures a responsive and dynamic development process that aligns with evolving market needs and promotes continuous collaboration, adaptability, and customer satisfaction.

1.6 Project Report Outline

Chapter 2 presents the literature.

A comprehensive review of the literature and market analysis to identify project-relevant gaps and opportunities.

Chapter 3 presents the analysis.

A detailed examination of requirements, constraints, and specifications, setting the project's scope and objectives.

Chapter 4 presents System Design and Implementation

In-depth discussion on architectural design, technological choices, and the translation of requirements into a tangible solution.

Chapter 5 presents System Testing and Evaluation.

Dedicated to assessing system effectiveness and performance through outlined testing methodologies and criteria.

Chapter 6 Conclusions and Future Work

Concluding the work and lessons learned then shed light on future work.

Chapter 2 Market and Literature Survey

2.1 Overview of Current Plant Care Apps

Plant care apps have transformed the way home gardeners interact with their plants. By leveraging technology like AI and image recognition, these apps provide gardeners with tools for plant identification, care schedules, pest diagnosis, and even shopping for seeds and tools. However, there are gaps in functionality, particularly in region-specific care.

2.2 Related Work: Key Applications in the Market

2.2.1 Picture This

- **Overview:**

Launched in 2017, this app uses AI to recognize plants through images. It is widely regarded for its large plant database and high-speed identification.

- **Key Features:**

- **AI-Powered Plant Recognition:** Users can take a photo of a plant and receive instant identification.
- **Care Tips:** Provides basic plant care tips once a plant is identified.
- **Global User Base:** The app has millions of users worldwide, particularly in regions where general gardening advice is sufficient.

- **Limitations:**

- **No Region-Specific Advice:** Tips are often too generic and not suitable for unique climates like Egypt.
- **No Continuous Care:** The app does not offer long-term plant health tracking.

2.2.2 Plantum

- **Overview:**

Focused on pest identification alongside plant care tips, Plantum targets gardeners dealing with common plant pests.

- **Key Features:**

- **Pest Detection:** Users can upload photos of pest damage to receive diagnoses and treatment suggestions.
- **Custom Care Advice:** Offers general advice for common plants.

- **Limitations:**

- **Pest Database:** While useful, the pest database lacks depth for regions like Egypt where climate-specific pests are prevalent.
- **No Advanced Care Tracking:** The app does not provide continuous care updates based on plant growth stages.

2.2.3 Garden Answers

- **Overview:**

This app stands out because it allows users to interact with gardening experts by submitting photos for advice.

- **Key Features:**
 - **Expert Feedback:** Personalized care suggestions from gardening professionals.
 - **Plant Identification:** Instant plant recognition using photos.
- **Limitations:**
 - **Expert Response Time:** Delays in expert feedback can leave users waiting for crucial advice.
 - **No Automated Health Monitoring:** Unlike other apps, Garden Answers relies solely on human input rather than ongoing AI monitoring.

2.2.4 Blossom

- **Overview:**

Designed primarily to help users manage plant care schedules, Blossom is more of a plant tracker.
- **Key Features:**
 - **Custom Care Reminders:** Allows users to set watering, pruning, and fertilizing reminders for their plants.
 - **Basic Care Tips:** Offers care schedules for common plants.
- **Limitations:**
 - **Lacks Advanced Diagnostics:** No pest identification or detailed health monitoring.
 - **No Region-Specific Customization:** Care schedules are too generic for specialized climates.

2.2.5 PlantSnap

- **Overview:**

Known for its global plant database, PlantSnap focuses on rare plant identification and location-based advice.

- **Key Features:**

- **Large Database:** Suitable for rare plant species.
- **Location-Based Tips:** Provides basic care tips based on geographical location.

- **Limitations:**

- **Generalized Advice:** The advice is too broad and not tailored enough for regions like Egypt.

2.2.6 Mashtal and BthRah (Local Competitors)

- **Overview:**

These are websites offering local e-commerce services for grains and gardening tools.

- **Key Features:**

- **E-Commerce Focus:** They allow users to buy seeds and gardening tools but lack any form of plant care advice or diagnostics.

- **Limitations:**

- **No Plant Identification:** Unlike the other apps, these platforms do not provide plant identification or care schedules.

2.3 Comparative Analysis of Existing Solutions

Criteria	Shatla	Picture This	Plantum	PlantSnap	Blossom	Mashtal & BthRah
Plant Identification	Yes	Yes	Yes	Yes	No	No
Region-Specific Advice	Yes	No	No	Yes	No	No
Identification Diagnosis	Yes	No	Yes	No	No	No
Care Schedule	Yes	No	No	No	Yes	No
E-Commerce Integration	Yes	No	No	No	No	Yes

Table 1 Related Work

2.4 Gaps in the Market

1. Lack of Localized Advice: Despite the global reach of many plant care apps, most fail to consider regional needs. For example, none of the widely used apps like Picture This or PlantSnap offer localized care tips based on the unique environmental conditions in Egypt, where issues like soil salinity and extreme heat require specialized care.

2. No Continuous Monitoring: While some apps like Blossom offer plant care reminders, they do not track a plant's ongoing health or adjust schedules based on changes in its condition. Shatla's approach is to continuously monitor the plant's health and adjust care recommendations dynamically, making it far more responsive to the plant's needs over time.

3. Pest Solutions That Are Incomplete: Pest identification is a feature in apps like Plantum, but the solutions offered are generic and often require expensive pesticides. Shatla, in contrast, provides natural, home-based remedies that are tailored to local conditions in Egypt, where certain pests thrive due to the hot climate.

2.5 How Shatla Solves These Issues

Shatla is specifically designed to address the gaps in the existing plant care apps market. Below are the features that distinguish Shatla:

- **Region-Specific Care:** Every care schedule is tailored to the Egyptian climate, adjusting for factors like temperature and humidity.
- **Continuous Plant Health Monitoring:** Instead of providing a one-time care tip, Shatla monitors the plant's health over time and adapts its advice as conditions change.
- **Natural Pest Control Solutions:** Shatla offers solutions using ingredients commonly found in Egyptian homes, reducing the need for chemical pesticides.
- **Integrated Marketplace:** With Shatla, users can directly purchase seeds, tools, and even organic fertilizers suited to the Egyptian market.

2.6 Conclusion and Market Potential

As home gardening continues to grow in popularity, especially in regions like Egypt, the demand for region-specific plant care solutions will only increase. **Shatla's** unique approach, combining tailored care schedules, dynamic health monitoring, and an integrated marketplace, positions it as a leader in this niche market. By addressing the gaps left by global competitors, **Shatla** has the potential to significantly enhance the gardening experience for Egyptian users and beyond.

Chapter 3 Shatla Analysis

3.1 Main Problem

The main problem Shatla addresses is the lack of easily accessible and accurate information for users about the plants suitable for planting in different seasons. Users also face challenges related to plant care techniques and solutions for common issues such as soil quality, sun exposure, plant health, and disease management. Many home gardeners struggle to find the right guidance, which leads to reduced plant productivity or complete plant loss. **Shatla** seeks to resolve this by providing users with comprehensive information about plants, including their care schedules, growth requirements, and solutions for problems, allowing users to maintain healthier, more productive plants.

3.2 Functional Requirements

Functional requirements define the specific functions, features, capabilities, and behaviors a system, software, or product must have. They describe what the system is supposed to do and outline the interactions between the system and its users.

FR-01

Title: Image Capture for Diagnosis

Description:

- The app should allow users to upload images for diagnosis either by taking a new photo via the camera or selecting an existing photo from the gallery.

- Users should be able to preview the image before submitting it.

Priority: High

Dependencies: Users must grant access to the camera and gallery.

Testable Criteria: Users can successfully upload images from both the camera and gallery, and the uploaded image is displayed in the preview.

FR-02

Title: Submit Button for Image Upload

Description:

- The app should provide a clearly labeled "Submit" button to allow users to submit the selected image and begin the analysis process.
- The button should be easy to access and use.

Priority: High

Dependencies: Users must have taken or selected an image (FR-01) before submitting it.

Testable Criteria: The button is functional and visible. Clicking it initiates the image analysis process.

FR-03

Title: Image Analysis for Disease Detection

Description:

- The app should analyze the uploaded image to detect plant diseases, pests, or any related issues.
- Detected problem areas on the plant should be highlighted.

Priority: High

Dependencies: FR-01, FR-02, and an internet connection are required.

Testable Criteria: The app successfully detects issues in the image and highlights the affected areas.

FR-04

Title: Display Diagnosis Results

Description:

- The app should display the diagnosis results, including a summary of detected problems and the uploaded image with the problem areas highlighted (if detected).
- If needed, show the confidence level of the diagnosis.

Priority: High

Dependencies: FR-03 must be completed.

Testable Criteria: Users can view a clear summary of detected issues or receive a message indicating that the plant is healthy.

FR-05

Title: Save Diagnosis Results to Plant History

Description:

- The app should save the diagnosis results and any accompanying tips for future reference.
- Users should be able to view their past diagnoses and tips for each plant in their profile.

Priority: Medium

Dependencies: FR-04 must be completed.

Testable Criteria: The app saves the diagnosis and tips, allowing users to view this information later in their history.

FR-06

Title: Provide Actionable Tips

Description:

- The app should provide actionable tips if problems are detected, such as advice on pest control, treatments for diseases, or household remedies.
- Additionally, it should suggest related products like fertilizers.
- Tips must be region-specific to Egypt and relevant to the detected issues.

Priority: High

Dependencies: FR-04 must be completed.

Testable Criteria: The app gives relevant and useful tips after the diagnosis, which are specific to the region and issues.

FR-07

Title: Login with Email and Password

Description:

- The app should allow users to log in using their email address along with their password.
- Users should be directed to the home screen upon successful login.

Priority: High

Dependencies: The user must have an existing account registered in the system.

Testable Criteria: Users can successfully log in using email and password, and the app authenticates within a reasonable time.

FR-08

Title: Login Error Handling

Description:

- The system should show appropriate error messages if login credentials are incorrect.

Priority: High

Dependencies: The system must authenticate credentials to display error messages.

Testable Criteria: Users receive relevant error messages for incorrect credentials.

FR-09

Title: Generate and Send OTP

Description:

- The system should generate a unique OTP (One-Time Password).
- The OTP should be sent to the user's registered email, depending on the user's preference.

Priority: High

Dependencies: The user must input their registered email to receive the OTP.

Testable Criteria: Users successfully receive an OTP via email.

FR-10

Title: Validate OTP

Description:

- The system should allow users to input the OTP received via email.
- The OTP should be validated against the sent OTP and ensure it has not expired.
- If valid, users should be authenticated and directed at the next step.

Priority: High

Dependencies: Users must have received an OTP before validation.

Testable Criteria: The system successfully validates OTP, allowing access for valid entries and denying access for incorrect or expired ones.

FR-11

Title: Resend OTP

Description:

- The system should provide a "Resend OTP" option if users do not receive the OTP or if it expires.
- A new OTP should be generated, invalidating the previous one.

Priority: High

Dependencies: Users must request a resend after the previous OTP is generated.

Testable Criteria: Users successfully request and receive a new OTP, and the old OTP becomes invalid.

FR-12

Title: Handle OTP Expiration

Description:

- The system should notify users when their OTP has expired (e.g., after 3 minutes).

Priority: High

Dependencies: The system must track the time elapsed since the OTP was sent.

Testable Criteria: Users receive a clear message when their OTP expires and can request a new one.

FR-13

Title: Reset or Forgot Password with OTP

Description:

- After successful OTP verification, the system allows users to reset their password directly within the app.
- Users will be prompted to enter and confirm a new password within the app interface.

Priority: High

Dependencies: Users must successfully verify the OTP before resetting their password.

Testable Criteria: Users can reset their password after OTP verification.

FR-14

Title: Logout

Description:

- The system shall allow users to log out from their accounts at any time.
- After logging out, the session is terminated, and users must log in again to access their account.

Priority: High

Dependencies: The user must be logged in to log out.

Testable Criteria: Users can log out, and their session is properly terminated.

FR-15

Title: Secure Session Storage

Description:

- The system securely stores the session information on the user's device.
- Sensitive information (e.g., authentication tokens) shall be stored securely (e.g., using encryption) to prevent unauthorized access.

Priority: High

Dependencies: Users must log in.

Testable Criteria: The app safely saves session data without exposing private information, and the session stays active even after restarting the app.

FR-16

Title: Session Expiration

Description:

- The session shall have an expiration time (e.g., 7 days), after which users must log in again.
- If the user manually logs out, the session should be cleared immediately.

Priority: Medium

Dependencies: The session must be actively tracked for expiration.

Testable Criteria: Users remain logged in until the session expires or they manually log out, and expired sessions require re-login.

FR-17

Title: Sign-up with Email (Manual or via Google/Facebook sync)

Description:

- The system allows users to sign up by manually entering their email address or by using their Google or Facebook account to synchronize the email automatically.
- If users choose Google or Facebook, the app will automatically fill in their email.

Priority: High

Dependencies: Google and Facebook APIs must be integrated for syncing email, and email format validation is required for manual entry.

Testable Criteria: Users can either manually enter their email or sync it from Google/Facebook, and it shows up correctly on the form.

FR-18

Title: Email Verification

Description:

- When users manually enter their email address during sign-up, the system validates the email format to ensure it is correct (e.g., user@example.com).
- Users must verify their email by either clicking on the verification link (if the email is manually entered) or completing an OAuth flow (if synchronized with Google or Facebook).
- The system shall display an error message if the email format is invalid.

Priority: High

Dependencies: Users must input or synchronize a valid email address.

Testable Criteria: Users get a verification message, and their account is activated after successful verification.

FR-19

Title: Strong Password

Description:

- Users must create a strong password that meets the following rules:
 - At least eight characters.
 - Includes one uppercase letter, one lowercase letter, one number, and one distinctive character.
- The app should show feedback to help users know how to create a strong password.

Priority: High

Dependencies: Password validation logic must be in place to enforce strength criteria.

Testable Criteria: Users can only proceed after creating a strong password, and the app shows guidance on meeting the rules.

FR-20

Title: Context-Aware Plant Chatbot Responses

Description:

- The chatbot shall understand user queries related to plants, including general care, disease symptoms, watering schedules, and environmental needs.
- The chatbot shall respond contextually using previously provided plant data (e.g., plant type, previous problems, preferences)
- If the user has previously scanned or identified a plant, the chatbot should tailor its advice based on the identification history.

Priority: Medium

Dependencies: Integration with plant database and identification history and Natural language understanding (NLU) for plant-specific queries.

Testable Criteria: User asks, “Why are my aloe leaves turning brown?” → Chatbot gives aloe-specific causes and solutions.

FR-21

Title: Multi-Image Support in Plant Identification

Description:

- The identification feature shall allow users to upload or capture multiple images (e.g., leaf, stem, whole plant) in a single session.
- The system shall process all images together to increase identification accuracy and return the most probable plant species or disease.
- Results shall include plant name, health status, possible diseases, and confidence percentage.

Priority: High

Dependencies: Image processing model supporting multi-input classification, UI support for image preview and batch submission.

Testable Criteria: User selects or captures multiple images of a plant, System returns combined result with confidence and explanation, and Users can see a summary including name, health status, and treatment advice.

FR-22

Title: Personalized Plant Recommendation Engine

Description:

- The app shall recommend plants based on user preferences including:
 - Location (indoor/outdoor)
 - Watering habits
 - Lighting conditions
 - Difficulty level
 - Fertilizer availability
 - Seasonality
- Users can fill in a short preference form or let the app infer preferences based on behavior/history.

Priority: Medium

Dependencies: User profile system and preference collection logic, Recommendation algorithm based on plant dataset filters.

Testable Criteria: User selects preferences (e.g., low maintenance, indirect light), App suggests 3+ suitable plants with images, descriptions, and care tips, and Results update instantly if preferences are modified.

FR-23

Title: Fetch and Display Climate Data

Description:

- The system shall fetch real-time climate data (such as temperature, humidity, and weather conditions) based on the user's garden location.
- The system shall display this data to the user in a readable format within the app.

Priority: High

Dependencies: Requires integration with a reliable third-party weather API to retrieve climate data and location access permission from the user (FR-24).

Testable Criteria: The app fetches and displays climate data based on the user's location, and the data is accurate and up to date.

FR-24

Title: Location Data Access and Permissions

Description:

- The app shall ask users for permission to access their location clearly and simply, following privacy rules.
- The app shall only use the location data if the user gives permission and shall allow users to turn location services on or off at any time.
- Users will be informed about why their location is needed and how it helps (to provide accurate climate data).
- The app shall use encryption to protect location data when sending it to third-party services (like weather data providers).

Priority: High

Dependencies: Must follow privacy laws and require location services to be turned on.

Testable Criteria: The app asks users for permission, and climate data is only fetched when access is allowed. Users can easily turn location access on or off.

FR-25

Title: Create Garden

Description:

- Users shall be able to create a new garden by providing a name.
- Users can have multiple gardens, each with its list of plants.

Priority: High

Dependencies: Users must be logged in to create a garden.

Testable Criteria: Users can create a new garden, which appears in their list of gardens.

FR-26

Title: View Gardens List

Description:

- Users shall be able to view their gardens.

Priority: High

Dependencies: Users must be logged in to view gardens.

Testable Criteria: Users can view their gardens.

FR-27

Title: Delete Garden

Description:

- Users shall be able to delete a garden from their list if they no longer wish to manage it.

- The system should ask for confirmation before deletion to prevent accidental removals.

Priority: High

Dependencies: Requires confirmation dialog and validation.

Testable Criteria: Users can delete a garden, and it is removed from their list upon confirmation.

FR-28

Title: Access/Select Garden

Description:

- Users shall be able to access any of their gardens and view the list of plants contained within.

Priority: High

Dependencies: Users must be logged in to access their gardens.

Testable Criteria: Users can view their selected garden with all added plants listed.

FR-29

Title: Add New Plant

Description:

- Users shall be able to add a new plant to their selected garden by entering relevant details (e.g., plant name, type, description, health status, care needs, photo, etc.).

Priority: High

Dependencies: Requires input validation for plant details.

Testable Criteria: Users can successfully add a new plant to the selected garden, and it appears in the garden's plant list.

FR-30

Title: Delete Existing Plant

Description:

- Users shall be able to delete a plant from their garden list.
- The system should ask for confirmation before deleting it to prevent accidental removals.

Priority: High

Dependencies: Requires confirmation dialog and validation.

Testable Criteria: Users can delete a plant from the garden, and it is removed from the list upon confirmation.

FR-31

Title: View Plant Details

Description:

- Users shall be able to tap on a plant in their garden to view detailed information (e.g., plant name, type, description, health status, care needs, photo, etc.).

Priority: High

Dependencies: Users must be logged in to view plant details.

Testable Criteria: Users can view all relevant details for each plant when selected.

FR-32

Title: Search Plant

Description:

- Users shall be able to search for specific plants in their selected garden by entering keywords (e.g., plant name).
- The app shall display matching results based on the search criteria.

Priority: Medium

Dependencies: Requires optimized search techniques for fast retrieval.

Testable Criteria: Users can search for plants within their garden, and the app shows relevant results quickly.

FR-33

Title: Custom Reminders for Care Tasks

Description:

- Users shall be able to set custom reminders for care tasks (e.g., watering, fertilizing) based on the generated care schedule.
- Users shall receive notifications at the scheduled time for each task and can modify or dismiss reminders if needed.

Priority: High

Dependencies: A care schedule should be generated (FR-38) and integrated with a notification system.

Testable Criteria: Users receive timely reminders for each care task and can customize or dismiss them.

FR-34

Title: View Detailed Care Schedule

Description:

- Users shall be able to view detailed care schedules for each plant, showing upcoming tasks and past activities.
- The schedule will update automatically based on completed tasks or changes in the plant's condition.

Priority: High

Dependencies: Detailed view of plant schedule required.

Testable Criteria: Users can see a detailed view of the plant's care schedule.

FR-35

Title: Save Schedule Snapshot

Description:

- The system shall save each care schedule as a snapshot whenever a new schedule is created, or an existing one is updated.
- Previous schedules are stored and can be accessed at any time.

Priority: Medium

Dependencies: Requires storage for scheduled snapshots.

Testable Criteria: Previous schedules are saved and can be accessed later.

FR-36

Title: View Care Schedule History

Description:

- Users shall be able to view the history of all past care schedules for each plant.

- Each entry shall show the schedule's date, tasks, and any related notes or diagnosis results.

Priority: Medium

Dependencies: None.

Testable Criteria: Users can view a complete history of care schedules, showing changes over time.

FR-37

Title: Update Schedule Based on Diagnosis

Description:

- If no issues are found, the app shall keep the existing care schedule.
- If issues are diagnosed (e.g., pests or diseases), the app should update the care schedule.

Priority: High

Dependencies: Integration with diagnosis results.

Testable Criteria: Care schedules change according to diagnosis results.

FR-38

Title: Generate Care Schedule (Weekly)

Description:

- The app shall generate a care schedule for each plant, including tasks like watering, fertilizing, and pest control.
- The schedule will be based on the type of plant and its needs.

Priority: High

Dependencies: Requires plant-specific care information.

Testable Criteria: Users can view a generated care schedule specific to their plant's needs.

FR-39

Title: Monthly Care Overview

Description:

- The app shall create a long-term monthly care schedule, showing tasks and milestones like watering and fruit production.
- Users can view monthly summaries of care tasks.

Priority: Low

Dependencies: Plant growth cycles (FR-40).

Testable Criteria: Users can see a monthly summary of care tasks and milestones.

FR-40

Title: Plant Growth Tracking

Description:

- Users shall be able to track plant growth over time, adding details like height, leaf count, and changes in condition.
- The app shall provide insights based on growth data.

Priority: Low

Dependencies: Analysis of saved schedules and plant conditions over time.

Testable Criteria: Users can log plant growth and get insights.

FR-41

Title: Predictive Care Suggestions

Description:

- The app shall analyze past care and environmental data to suggest future tasks, like adjusting watering during hot months.

Priority: Low

Dependencies: Analysis of care and climate data.

Testable Criteria: Users get care suggestions based on past trends.

FR-42

Title: Browse Products

Description:

- The app shall allow users to browse a catalog of available seeds and farm tools.
- Products shall be displayed in categories for easy navigation.

Priority: Medium

Dependencies: Requires a product database with categories.

Testable Criteria: Users can browse dissimilar categories of products and view a variety of seeds and tools.

FR-43

Title: Search and Filter Products

Description:

- The app should provide a search function that allows users to quickly find specific products by entering keywords.

- Users shall also be able to filter products by categories (e.g., seed type, price range, ratings) to narrow down their options.

Priority: Medium

Dependencies: Requires optimized search and filter algorithms.

Testable Criteria: Users can search for specific products and apply filters to refine the product list.

FR-44

Title: View Product Details

Description:

- The app shall display detailed information for each product, including name, description, price, available stock, images, etc.

Priority: Medium

Dependencies: Requires a product database with detailed information.

Testable Criteria: Users can view full product details.

FR-45

Title: Shopping Cart

Description:

- Users shall be able to add products to a shopping cart for purchase.
- Users shall be able to view items in the cart, update quantities, or remove items before proceeding to check out.

Priority: Medium

Dependencies: Requires a shopping cart system.

Testable Criteria: Users can view items in their cart, update quantities, or remove items before checkout.

FR-46

Title: View Cart Summary

Description:

- The app shall display a summary of all items in the shopping cart, showing the total price, item quantity, and any applicable discounts or promotions.

Priority: Medium

Dependencies: Requires shopping cart (FR-45) to calculate totals and apply discounts.

Testable Criteria: Users can view a cart summary with an accurate total cost, including discounts.

FR-47

Title: View Profile Information

Description:

- Users shall be able to view their personal information, including name, phone number, email, and location in the profile section.
- The profile page shall display this information in a clean, organized layout.

Priority: High

Dependencies: Requires a user database to store personal details.

Testable Criteria: Users can view their personal information in the profile section.

FR-48

Title: Edit Profile Information

Description:

- Users shall be able to update their details, including name, phone number, email, and location.

Priority: High

Dependencies: Requires user database for storing and updating information.

Testable Criteria: Users can successfully update their profile information, which is saved and reflected immediately.

FR-49

Title: Logout

Description:

- Users shall be able to log out of the app, end their session, and return to the login screen.

Priority: High

Dependencies: Requires session management to manage user sessions and logout functionality.

Testable Criteria: Users can successfully log out and are redirected to the login screen.

FR-50

Title: Admin – Category Management

Description:

- The admin dashboard shall display a list of all existing product categories, each with its name, description, and the number of products within it.

- Admins shall be able to create, update, or delete categories.
- When a category is deleted, products associated with it shall either be reassigned to a default category or flagged for admin action.

Priority: High

Dependencies:

- Category CRUD APIs
- Product-category relationship model

Testable Criteria:

- Admin can view, add, edit, and delete categories.
 - Changes reflect immediately in the user-facing app.
 - Deleting a category triggers appropriate reassignment or warning
-

FR-51

Title: Admin – Product List Management

Description:

- The admin dashboard shall display a list of all products including name, image, category, price, quantity, and availability status.
- Admins can search, filter (by category, stock status), and sort products.
- Admins can add new products, edit product details, or remove products from the catalog.

Priority: High

Dependencies:

- Product CRUD APIs
- Product-category and inventory data

Testable Criteria:

- Admin can view and manage product listings.

- Filter/search functionality returns accurate results.
 - Product edits update in the front end instantly.
-

FR-52

Title: Admin – Customer List View

Description:

- The admin dashboard shall show a table of registered customers including full name, email, registration date, and number of orders.
- Admins can search and filter customers by name, activity, or registration date.
- Admins can deactivate or flag account for suspicious behavior.

Priority: Medium

Dependencies:

- User database access
- Customer order linkage

Testable Criteria:

- Admin views customer table with filters
 - Deactivating a customer disables their login/access.
 - The number of orders accurately reflects transaction history.
-

FR-53

Title: Admin – Sales Analytics Dashboard

Description:

- The admin dashboard shall include a visual analytics section showing:
 - Total sales overtime (daily/weekly/monthly)
 - Top-selling products.
 - Revenue per category

- Sales trends via charts and graphs
- Admins can filter analytics by date range and category.

Priority: High

Dependencies:

- Sales transaction data
- Charting libraries (e.g., Chart.js, Recharts)

Testable Criteria:

- Admin can view sales summaries and visual reports.
 - Filtering by date/category dynamically updates the dashboard.
 - Data matches backend transaction records
-

3.3 Non-Functional Requirements

known as quality attributes, are the characteristics that define how a system performs its functions rather than what functions it performs.

3.3.1 Usability

Table 2 Usability Requirements for Shatla Application

Requirement	Description
R01	The system should offer a straightforward flow in the Shatla application, enhanced by a tour window or video tutorial to facilitate learning.

Requirement	Description
R02	The system should use tooltips to explain functionalities and errors, helping to reduce the learning curve for users.
R03	The system should provide intuitive and logical navigation structures that make it easy for users to find what they need.
R04	The system should provide immediate feedback on user actions (e.g., success messages, error alerts) to help users learn what works and what does not.
R05	The system should provide a consistent layout and design to help users feel comfortable returning to the application.

3.3.2 Security

Table 3 Security Requirements for Shatla Application

Requirement	Description
R01	The system should implement secure authentication mechanisms, including JWT for user authentication.
R02	The system should implement role-based access control to manage user permissions, ensuring that users have access only to the functionalities and data relevant to their assigned roles (e.g., admin, regular user). This includes the use of JWT to encode user roles within the token.

Requirement	Description
R03	The system shall implement robust session management mechanisms to ensure secure user interactions and maintain user state throughout their engagement with the application.
R04	The system must ensure that the data collected from the user (the image of the plant) is accurately captured and transmitted to the backend for analysis. This involves using proper data formats for image uploads and ensuring that images are not corrupted during transmission.
R05	The system should implement error detection and handling mechanisms to manage situations where the AI cannot confidently classify a plant or detect a disease.

3.3.3 Maintainability

Table 4 Maintainability Requirements for the Shatla Application

Requirement	Description
R01	The system should be designed using modular architecture, where components or modules are self-contained and can be independently developed, evaluated, and reused.
R02	The source code shall follow standard coding conventions and be well-documented, with clear and meaningful names for variables, functions, and classes.
R03	The system shall adhere to the principle of separation of concerns, where different functionalities are divided into distinct

Requirement	Description
	sections (e.g., UI, business logic, data access) using appropriate design patterns.
R04	The system shall support automated testing, including unit tests, integration tests, and end-to-end tests.
R05	The system's source code and documentation shall be managed using a version control system (Git) to track changes over time.

3.3.4 Performance

Table 5 Performance Metrics Requirements for Shatla Application

Requirement	Description
R01	The system shall respond to user queries within 35 seconds to ensure smooth user experience.
R02	The system shall manage at least five requests per second to support concurrent users without performance degradation.
R03	The system shall support data storage for up to 1 Gb without significantly affecting response times.
R04	The latency for loading data shall not exceed seconds for users.

3.4 Stakeholders

Stakeholders and Their Responsibilities

User

- Browse and search for plants using filters.
- View detailed plant information, including care guides and plant conditions.
- Add plants to their personal garden list for tracking.
- Edit the garden by adding or deleting plants.
- Update personal information, including profile details.
- View their profile and the plants added to their garden.
- View detailed plant care schedules, including watering, fertilization, pruning, and pest management.

Admin

- View a list of all registered admins on the platform.
- Manage the plant database by adding, updating, or removing plant information based on platform guidelines and quality standards.
- Monitor user activity related to plants, ensuring compliance with platform rules.

3.5 System Scenarios

User

Scenario 1: Browse and Search for Plants

- **As a user**, I want to browse and search for plants available on the platform, utilizing filters (e.g., plant type, seasonality, care level) and search functionality to find specific plants relevant to my needs.

Scenario 2: View Plant Details

- **As a user**, I want to view detailed information about each plant, including its characteristics (e.g., sunlight needs, water requirements, soil type), growth season, common problems, and care instructions.

Scenario 3: Add Plant to Garden

- **As a user**, I want to add specific plants to my personal garden list for easy access and future reference, so I can track their care and growth.

Scenario 4: Edit Garden

- **As a user**, I want to be able to update my garden by adding or deleting plants from the list, allowing me to effectively manage the plants I am caring for.

Scenario 5: Update Profile

- **As a user**, I want to update my personal information (such as name, email, contact details, and profile picture) to keep my profile up to date.

Scenario 6: View Profile

- **As a user**, I want to view my profile information, including personal details and the list of plants I have saved in my garden, with the option to edit this information if needed.

Scenario 7: View Plant Schedules

- **As a user**, I want to view detailed care schedules for each plant in my garden, including watering, fertilization, pruning, and any relevant reminders for proper plant care based on the plant's life cycle and seasonal requirements.

Admin

Scenario 1: View All Admins

- **As an admin**, I want to access a list of all admin users on the platform to see who has admin-level access and to manage their permissions if necessary.

Scenario 2: Manage Plants

- **As an admin**, I want to manage all plant-related information on the platform by adding, updating, or removing plants as necessary to ensure that users have access to accurate and up-to-date data, following platform policies and quality standards.

3.6 Use Case Diagram



To view Clearer pictures of diagrams, visit this link: [Shatla Diagrams](https://shatla.com)

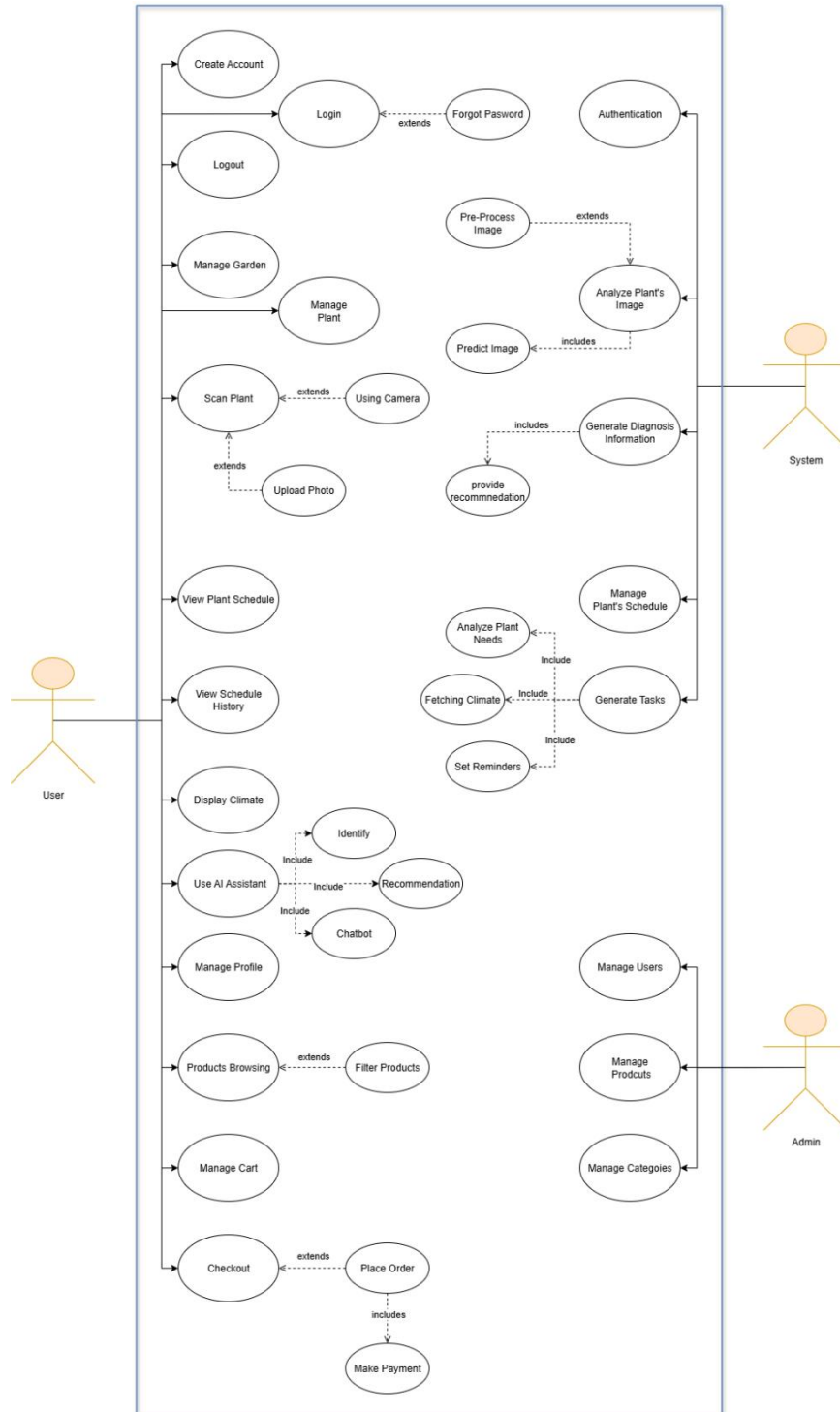


Figure 2 Use Case

3.7 Use Case Tables

3.7.1 User Account Management

Use Case ID:	UAM-01	
Use Case Name:	Create Account	
Actors:	User, System	
Pre-conditions:	The user has access to the registration page.	
Post-conditions:	A new user account has been created, and the user can log in.	
The flow of events:	User Action	System Action
	1-Navigates to the registration page.	
		2-Displays the registration form.
	3- Fill in the required fields (e.g., name, email/phone, password).	
		4-Validates input fields for accuracy and format.
	5-Submit the form.	
		6-Sends a verification code to the provided email.
	7-Enter the verification code.	
		8-Validate the code and create the account.

Exceptions:	User Action	System Action
	1-Enters an invalid email format.	
		2-Displays an error message and prompts for correction.
	3-Enters mismatched password confirmation.	
		4-Displays a validation error for the password field.
	5-Attempts to register with an existing email.	
		6-Notifies the user of duplicate account and suggests recovery.
Includes:	Email/Phone validation, password validation.	
Notes and Issues:	Strong password enforcement and duplicate account prevention.	

Use Case ID:	UAM-02	
Use Case Name:	Login	
Actors:	User, System	
Pre-conditions:	The user must have an existing account.	
Post-conditions:	The user is logged into the system.	
The flow of events:	User Action	System Action
	1-Navigates to the login page.	
		2-Displays the login form.

	3-Enter email and password.	
		4-Validates credentials.
	5-Submit the form.	
		6-Grants access and redirects to the Home page.
Exceptions:	User Action	System Action
	1-Enters incorrect email or password.	
		2-Displays an "Invalid credentials" error.
	3-Excess allowed login attempts.	
		4-Temporarily locks the account or prompts for CAPTCHA.
Includes:	Login error handling.	
Notes and Issues:	Ensure secure session management and encrypted password storage.	

Use Case ID:	UAM-03	
Use Case Name:	Logout	
Actors:	User, System	
Pre-conditions:	The user must be logged in.	
Post-conditions:	The user session is terminated.	
The flow of events:	User Action	System Action
	1-Clicks on the "Logout" button.	

		2-Confirms the action (optional).
	3-Confirms logout action.	
		4-Ends the session and redirects to the login page.
Exceptions:	User Action	System Action
	1-Refreshes the page after logout.	
		2-Redirects to the login page.
Includes:	Session invalidation.	
Notes and Issues:	Ensure all active tokens are invalidated.	

Use Case ID:	UAM-04	
Use Case Name:	Reset/Forgot Password	
Actors:	User, System	
Pre-conditions:	The user must have an existing account.	
Post-conditions:	Users can set a new password.	
The flow of events:	User Action	System Action
	1-Navigates the "Forgot Password" page.	
		2-Displays a form to enter email.
	3-Enter email and submit.	

		4-Send a reset link or OTP.
	5-Click on the reset link or enter OTP.	
		6-Validates the reset link or OTP.
	7-Enters a new password.	
		8-Validate and update the password.
Exceptions:	User Action	System Action
	1-Enter an invalid email.	
		2-Displays an error message.
	3-Uses an expired reset link or OTP.	
		4-Prompts to request a new reset link/OTP.
Includes:	Email verification, and strong password rules.	
Notes and Issues:	Manage expired or invalid reset tokens.	

Use Case ID:	UAM-05	
Use Case Name:	OTP Authentication	
Actors:	User, System	
Pre-conditions:	The user must initiate an action requiring OTP (e.g., login, account creation).	
Post-conditions:	OTP is validated, and the action is completed.	
	User Action	System Action

The flow of events:	1-Requests OTP (e.g., during login or sign-up).	
		2-Generates and sends OTP via email.
	3-Enter the OTP in the form.	
		4-Validate the OTP.
Exceptions:	User Action	System Action
	1-Enters Incorrect OTP.	
		2-Displays an error and prompts for re-entry.
	3-Resends OTP after a timeout.	
		4-Generates a new OTP and send it.
Includes:	Generate OTP, Validate OTP, and Resend OTP.	
Notes and Issues:	Ensure OTP expiration and secure transmission.	

3.7.2 Garden and Plant Management

Use Case ID:	GPM-01
Use Case Name:	Manage Garden
Actors:	User, System
Pre-conditions:	The user is logged in and on the garden management page.

Post-conditions:	The user can create, view, delete, or access a garden.	
The flow of events:	User Action	System Action
	1-The user logs into the system and navigates to the "Manage Garden" page.	
		2-The system displays a list of existing gardens (if any).
	3-The user selects an action to manage gardens (create, view, delete, access).	
		4-System provides the corresponding page and actions for each option.
Exceptions:	User Action	System Action
	1-No gardens have been created by the user yet.	
		2-System prompts the user to create a garden.
	3-The user attempts to access or manage a garden without permission.	
		4-System shows an error message or restricted access prompt.

Includes:	Create Garden, View Gardens List, Delete Garden, Access/Select Garden.
Notes and Issues:	Ensure smooth navigation between various garden management features.

Use Case ID:	GPM-02	
Use Case Name:	Create Garden	
Actors:	User, System	
Pre-conditions:	The user is logged in and is on the "Manage Garden" page.	
Post-conditions:	A new garden is being created and linked to the user's account.	
The flow of events:	User Action	System Action
	1-The user clicks the "Create Garden" button.	
		2-The system displays a garden creation form with fields for garden name, location, and description.
	3-The user enters valid details and clicks "Submit."	
		4-System validates the form and saves the garden in the database. A confirmation message is displayed.
Exceptions:	User Action	System Action
	1-The user leaves the required field blank.	

		2-System prompts the user to fill out the required field(s).
	3-The garden name is already in use.	
		4-System displays a message informing the user that the name is taken.
Includes:	From validation for garden name, location, and description.	
Notes and Issues:	Manage errors like duplicate garden names. Ensure validation for required fields.	

Use Case ID:	GPM-03	
Use Case Name:	View Gardens List	
Actors:	User, System	
Pre-conditions:	The user is logged in and has created at least one garden.	
Post-conditions:	The system displays a list of all gardens associated with the user's account.	
The flow of events:	User Action	System Action
	1-The user clicks on "View Gardens List."	
		2-System fetches all gardens associated with the user's account and displays them in a list format.
	3-The user selects a garden to view its details.	
		4-System loads the selected garden's detailed view.

Exceptions:	User Action	System Action
	1-The user has no gardens.	
		2-System displays a message that no gardens are available and prompts to create one.
Includes:	Pagination for large garden lists.	
Notes and Issues:	Ensure all garden data is fetched accurately, especially when there are multiple gardens.	

Use Case ID:	GPM-04	
Use Case Name:	Delete Garden	
Actors:	User, System	
Pre-conditions:	The user has at least one garden and selects a garden to delete.	
Post-conditions:	The selected garden is removed from the user's account and the database.	
The flow of events:	User Action	System Action
	1-The user selects a garden to delete.	
		2-System displays a confirmation prompt to ensure the user wants to delete the garden.
	3-The user confirms the deletion.	
		4-System deletes the garden from the database and shows a success message.
Exceptions:	User Action	System Action

	1-The user cancels the deletion.	
		2-The system cancels the action and returns to the garden list.
Includes:	Confirmation dialog for garden deletion.	
Notes and Issues:	Confirm the garden deletion and ensure proper feedback.	

Use Case ID:	GPM-05	
Use Case Name:	Access/Select Garden	
Actors:	User, System	
Pre-conditions:	The user has at least one garden available in their account.	
Post-conditions:	The user is taken to the selected garden's details or management page.	
The flow of events:	User Action	System Action
	1-The user selects a garden from the list.	
		2-The system displays the selected garden's details or the management interface.
Exceptions:	User Action	System Action
	1-The user tries to access a garden with insufficient permissions.	
		2-The system displays an error message and restricts access.

Includes:	Option to view garden details and manage plants.
Notes and Issues:	Ensure the correct garden is selected and all related data loads.

Use Case ID:	GPM-06	
Use Case Name:	Manage Plant	
Actors:	User, System	
Pre-conditions:	The user is logged in and has an existing garden with plants.	
Post-conditions:	The user can view, add, or delete plants within their garden.	
The flow of events:	User Action	System Action
	1-The user logs into the system and navigates to their garden.	
		2-The system displays a list of plants within the garden.
	3-The user selects a plant from the list.	
		4-System provides options to view, add, delete, or update the plant's details
Exceptions:	User Action	System Action
	1-The user tries to manage a plant that does not exist.	
		2-The system displays an error message stating that the plant cannot be found.

Includes:	Add New Plant, Delete Existing Plant, View Plant Details.
Notes and Issues:	Ensure proper handling of invalid or missing plant data.

Use Case ID:	GPM-07	
Use Case Name:	Add New Plant	
Actors:	User, System	
Pre-conditions:	The user is logged in and is on the "Manage Plant" page of an existing garden.	
Post-conditions:	A new plant has been added to the user's selected garden.	
The flow of events:	User Action	System Action
	1-The user clicks "Add New Plant."	
		2-The system displays a form with fields for plant name, species, care instructions, and an optional image.
	3-The user enters the plant's information and submits the form.	
		4-The system validates the details entered and saves the new plant in the database, associating it with the user's garden.
Exceptions:	User Action	System Action
	1-The user leaves the required fields empty.	

		2-The system displays an error message prompting the user to fill out the required fields.
	3-The plant name already exists.	
		4-System notifies the user that the plant name already exists and suggests alternatives.
Includes:	From validation for plant name, species, and care information.	
Notes and Issues:	Ensure accurate validation and avoid duplicating plant names.	

Use Case ID:	GPM-08	
Use Case Name:	Delete Existing Plant	
Actors:	User, System	
Pre-conditions:	The user has at least one plant in their garden.	
Post-conditions:	The selected plant is removed from the user's garden.	
The flow of events:	User Action	System Action
	1-The user clicks "Delete" on a plant's entry.	
		2-The system displays a confirmation dialog asking the user to confirm deletion.

	3-The user confirms deletion.	
		4-System deletes the plant from the database and updates the garden's plant list.
Exceptions:	User Action	System Action
	1-The user cancels the deletion.	
		2-The system cancels the operation and returns to the garden's plant list.
	3-The plant cannot be found.	
		4-System displays an error message stating that the plant could not be deleted (e.g., if it has been deleted already).
Includes:	Confirmation dialog for plant deletion.	
Notes and Issues:	Ensure that the user understands the impact of deleting a plant.	

Use Case ID:	GPM-09
Use Case Name:	View Plant Details
Actors:	User, System
Pre-conditions:	The user is logged in and has plants in their garden.
Post-conditions:	The user views detailed information of a selected plant.

The flow of events:	User Action	System Action
	1-The user selects a plant from the garden list.	
		2-System fetches and displays detailed information about the selected plant (e.g., name, species, care instructions, image).
Exceptions:	User Action	System Action
	3-The plant details are missing.	
		4-The system displays an error message stating that no detailed information is available.
Includes:	View all related information about a plant.	
Notes and Issues:	Display plant images and care instructions clearly.	

3.7.3 Plant Diagnosis and Recommendations

Use Case ID:	PDR-01
Use Case Name:	Image Capture for Diagnosis
Actors:	User, System
Pre-conditions:	The user is logged in and is on the plant diagnosis page.
Post-conditions:	A photo is captured and ready for analysis.

The flow of events:	User Action	System Action
	1-The user clicks the "Capture Image" button.	
		2-The system activates the camera to take a picture of the plant.
	3-The user captures the image of the plant.	
Exceptions:		4-The system processes the captured image and prepares it for the next step (image upload or analysis).
	User Action	System Action
	1-The user denies camera permissions.	
		2-System displays an error message explaining the need for camera access.
	3-The image is blurry or low-quality.	
		4-System prompts the user to retake the picture with suggestions for better lighting and focus.
Includes:	Camera interface integration.	
Notes and Issues:	Ensure proper permission for camera access, handle errors like low lighting or blurry images.	

Use Case ID:	PDR-02	
Use Case Name:	Submit Button for Image Upload	
Actors:	User, System	
Pre-conditions:	The user captured an image of the plant.	
Post-conditions:	The image is uploaded to the system for analysis.	
The flow of events:	User Action	System Action
	1-The user clicks "Submit" after capturing the image	
		2-The system uploads the image to the server for processing and analysis.
Exceptions:	User Action	System Action
	1-The image is too large for upload.	
		2-System displays an error message and suggests resizing the image.
	3-The user has no internet connection.	
		4-The system displays a message about connectivity issues and retries the upload when possible.
Includes:	Image upload functionality, file size validation.	
Notes and Issues:	Ensure the upload supports various image formats (JPG, PNG) and large file sizes.	

Use Case ID:	PDR-03	
Use Case Name:	Analyze Plant Image (Pre-process Image, Detect Disease)	
Actors:	System, AI Model	
Pre-conditions:	The image is uploaded to the system.	
Post-conditions:	The system processes the image and identifies any diseases or anomalies.	
The flow of events:	User Action	System Action
	1-The image is uploaded.	
		2-System pre-processes the image (e.g., resizing, enhancement) for disease detection.
	3-The system analyzes the image using an AI model.	
		4-The system detects the plant's health status and diseases.
Exceptions:	User Action	System Action
	1-The image is not suitable for analysis.	
		2-System returns a message asking the user to upload a clearer image.
	3-No disease detected in the image.	
		4-System notifies the user that no diseases were found.

Includes:	Image pre-processing, AI model integration.
Notes and Issues:	Ensure the AI model has been trained to detect diseases based on the dataset.

Use Case ID:	PDR-04	
Use Case Name:	Display Diagnosis Results	
Actors:	User, System	
Pre-conditions:	The plant image has been analyzed for diseases.	
Post-conditions:	The user sees the diagnosis results, including disease name and severity.	
The flow of events:	User Action	System Action
	1-The system finishes analyzing the image.	
		2-The System displays diagnosis results, including disease name, severity, and treatment options.
Exceptions:	User Action	System Action
	1-No disease detected.	
		2-The System notifies the user that no disease was detected and offers preventive care tips.
Includes:	Disease severity display, potential causes.	
Notes and Issues:	Ensure the results are understandable and provide relevant details to the user.	

Use Case ID:	PDR-05	
Use Case Name:	Save Diagnosis Results to Plant History	
Actors:	User, System	
Pre-conditions:	The diagnosis results are available.	
Post-conditions:	Diagnosis results are saved to the plant's history.	
The flow of events:	User Action	System Action
	1-The user clicks "Save" on the diagnosis results.	
		2-System saves the diagnosis data to the plant's history.
Exceptions:	User Action	System Action
	1-The user tries to save incomplete diagnosis.	
		2-System prompts the user to ensure all fields are complete before saving.
Includes:	Plant history tracking.	
Notes and Issues:	Ensure that diagnosis results are stored securely and can be retrieved later.	

Use Case ID:	PDR-06	
Use Case Name:	Provide Recommendations	
Actors:	User, System	
Pre-conditions:	The diagnosis results have been analyzed.	

Post-conditions:	The user receives recommendations for the plant.	
The flow of events:	User Action	System Action
	1-The user requests recommendations after diagnosis.	
		2-The system generates and displays tailored recommendations based on the plant's diagnosis.
Exceptions:	User Action	System Action
	1-The system is unable to generate recommendations.	
		2-The system notifies the user that recommendations could not be provided and suggests further actions.
Includes:	Care tips, preventive measures.	
Notes and Issues:	Ensure the recommendations are practical and easy to follow.	

3.7.4 Care Schedule and Notifications

Use Case ID:	CSN-01
Use Case Name:	Generate Care Schedule (Weekly)
Actors:	System, User
Pre-conditions:	Users have added plants to the garden, and plant data is available.

Post-conditions:	A weekly care schedule is generated and saved for each plant.	
The flow of events:	User Action	System Action
	1-User clicks "Generate Weekly Schedule".	
		2-The system analyzes plant care requirements and generates a care schedule.
	3-User confirms the schedule.	
		4-System saves the schedule and notifies the user.
Exceptions:	User Action	System Action
	1-User requests a schedule for a plant with incomplete data.	
		2-System prompts the user to provide missing data before generating the schedule.
	3-System fails to fetch external climate data.	
		4-The system uses default care guidelines to generate the schedule.
Includes:	Integration with climate data, plant-specific care requirements.	
Notes and Issues:	Ensure the schedule accounts for holidays, user preferences, and regional weather conditions.	

Use Case ID:	CSN-02	
Use Case Name:	Update Care Schedule Based on Diagnosis	
Actors:	System, User	
Pre-conditions:	Diagnosis results are available for a plant.	
Post-conditions:	The care schedule is updated with actions based on the diagnosis.	
The flow of events:	User Action	System Action
	1-User selects a plant with diagnosis results.	
		2-System retrieves the diagnosis and suggests schedule updates.
	3-User confirms the updates.	
		4-System saves the updated schedule.
Exceptions:	User Action	System Action
	1-User cancels the update.	
		2-The system retains the original schedule.
	3-Diagnosis data is incomplete.	
		4-The system prompts the user to complete missing details.
Includes:	Diagnosis data integration, schedule adjustments.	
Notes and Issues:	Ensure updated schedules are highlighted for user review.	

Use Case ID:	CSN-03	
Use Case Name:	View Plant Schedule	
Actors:	User	
Pre-conditions:	A care schedule has been generated for the plant.	
Post-conditions:	The user views detailed care tasks for the plant.	
The flow of events:	User Action	System Action
	1-User selects a plant from the list.	
		2-The system retrieves and displays the plant's care schedule.
Exceptions:	User Action	System Action
	1-User selects a plant with no schedule.	
		2-System notifies the user and suggests generating a schedule.
Includes:	User interface for schedule visualization.	
Notes and Issues:	Provide easy navigation and filter options for the user.	

Use Case ID:	CSN-04	
Use Case Name:	View Care Schedule History	
Actors:	User	

Pre-conditions:	Past schedules exist for the plant.	
Post-conditions:	The user views the history of care schedules.	
The flow of events:	User Action	System Action
	1-User requests a plant's schedule history.	
		2-The system retrieves and displays history from the database.
Exceptions:	User Action	System Action
	1-No historical schedules exist.	
		2-System notifies the user that no history is available.
Includes:	Database queries for historical data.	
Notes and Issues:	Ensure schedules are displayed in a chronological and user-friendly manner.	

Use Case ID:	CSN-05	
Use Case Name:	Provide Reminders	
Actors:	System, User	
Pre-conditions:	The care schedule includes tasks with deadlines.	
Post-conditions:	The user receives reminders for scheduled tasks.	
The flow of events:	User Action	System Action
	1-User configures task reminders.	

		2-System schedules notifications for the configured tasks.
Exceptions:	User Action	System Action
	1-User disables reminders.	
		2-System stops sending notifications for the disabled reminders.
Includes:	Notification system, task deadlines.	
Notes and Issues:	Ensure reminders can be customized or snoozed by the user.	

Use Case ID:	CSN-06	
Use Case Name:	Display Notifications	
Actors:	System, User	
Pre-conditions:	Notifications are set for tasks or events.	
Post-conditions:	Notifications are displayed to the user.	
The flow of events:	User Action	System Action
	1-User opens the notification center.	
		2-The system displays pending notifications.
Exceptions:	User Action	System Action
	1-No new notifications exist.	

		2-The system displays a message indicating no new notifications.
Includes:	Notification system integration.	
Notes and Issues:	Ensure notifications are actionable and easy to dismiss.	

3.7.5 User Profile Management

Use Case ID:	UPM-01	
Use Case Name:	View Profile Information	
Actors:	User	
Pre-conditions:	The user is logged into the system.	
Post-conditions:	Users view their profile details on the profile page.	
The flow of events:	User Action	System Action
	1-User selects the "Profile" option in the menu.	
		2-The system fetches the user's profile data from the database.
	3-User view the displayed profile details.	
		4-System ensures the information is up-to-date and accurate.

Exceptions:	User Action	System Action
	1-User profile data is incomplete or corrupted.	
		2-System displays an error message and prompts the user to edit missing or invalid details.
	3-Database connectivity issues occur.	
		4-System notifies the user of a temporary error and retries fetching the data.
Includes:	Data retrieval from the database, profile page UI.	
Notes and Issues:	Ensure sensitive data such as passwords are not displayed.	

Use Case ID:	UPM-02	
Use Case Name:	Edit Profile Information	
Actors:	User	
Pre-conditions:	Users are logged into the system, and their profile information exists in the database.	
Post-conditions:	Profile information is successfully updated and saved in the system.	
The flow of events:	User Action	System Action
	1-User clicks the "Edit Profile" button.	

		2-The system displays an editable form with the current profile details.
	3-Users update their information and submit the form.	
		4-The system validates the input and saves the changes to the database.
	5-User receives confirmation of a successful update.	
		6-The system updates the displayed profile with the new data.
Exceptions:	User Action	System Action
	1-User enters invalid data (e.g., incorrect email format).	
		2-System highlights error fields and prompts the user to correct them.
	3-User cancels the editing operation.	
		4-System discards unsaved changes and reloads the original profile details.
	5-Database error occurs during saving.	

		6-System notifies the user of a temporary error and retries saving the data.
Includes:	From validation, database update operations.	
Notes and Issues:	Validation rules must ensure proper data format (e.g., valid email, phone number).	

3.7.6 E-Commerce Features

Use Case ID:	ECOM-01	
Use Case Name:	Browse Products	
Actors:	User	
Pre-conditions:	The user is logged in and the product database is available.	
Post-conditions:	The user sees a list of products.	
The flow of events:	User Action	System Action
	1-User opens the "Products" section.	
		2-System fetches and displays the list of available products.
	3-User scroll through the list.	
		4-System loads additional products if pagination or lazy loading is implemented.
Exceptions:	User Action	System Action

	1-No products are available.	
		2-The system displays a "No products found" message.
	3-Database connectivity issues occur.	
		4-System notifies the user of a temporary error and retries fetching the products.
Includes:	Product listing, and pagination (if needed).	
Notes and Issues:	Ensure performance optimization for large product lists.	

Use Case ID:	ECOM-02	
Use Case Name:	Search and Filter Products	
Actors:	User	
Pre-conditions:	The user is on the product page and the product database is available.	
Post-conditions:	The user sees a filtered list of products.	
The flow of events:	User Action	System Action
	1-User enters a keyword in the search bar.	
		2- The system processes the search query and displays matching products.

	3- The user selects a filter (e.g., category).	
		3-The system applies the filter and updates the product list accordingly.
Exceptions:	User Action	System Action
	1-No matching products are found.	
		2-The system displays a "No matching products found" message.
	3-Invalid search or filter input is provided.	
		3-The system displays a validation error and prompts the user to correct it.
Includes:	Search query processing, filter options (e.g., category, price range).	
Notes and Issues:	Ensure filters work together logically without conflicts.	

Use Case ID:	ECOM-03	
Use Case Name:	View Product Details	
Actors:	User	
Pre-conditions:	The user is viewing a list of products.	
Post-conditions:	The user views detailed information about a selected product.	
	User Action	System Action

The flow of events:	1-User clicks on a product from the list.	
		2-The system retrieves and displays product details (e.g., name, price, description, images).
Exceptions:	User Action	System Action
	1-Product details are unavailable or corrupted.	
		2-System displays an error message and redirects the user back to the product list.
Includes:	Product details page.	
Notes and Issues:	Ensure detailed product information is accurate and up to date.	

Use Case ID:	ECOM-04	
Use Case Name:	Shopping Cart	
Actors:	User	
Pre-conditions:	The user has selected products to add to the cart.	
Post-conditions:	The user's selected items are added to the cart.	
The flow of events:	User Action	System Action
	1-User clicks "Add to Cart" for a product.	
		2-The system adds the product to the shopping cart and updates the cart count.

	3- The user updates the quantity of an item.	
		4-The system recalculates the total cost and updates the cart.
Exceptions:	User Action	System Action
	1-Products are out of stock.	
		2-The system notifies the user and prevents adding the product to the cart.
	3-Database error during cart update.	
		4-The system displays an error and retrieves the operation.
Includes:	Add to cart, update cart quantity, remove from cart.	
Notes and Issues:	Ensure the cart is updated in real-time and persists across sessions.	

Use Case ID:	ECOM-05	
Use Case Name:	View Cart Summary	
Actors:	User	
Pre-conditions:	The user has items in the shopping cart.	
Post-conditions:	The user views a summary of selected products.	
The flow of events:	User Action	System Action
	1-User clicks "View Cart".	

		2-System retrieves cart items and displays them along with quantities and total price.
Exceptions:	User Action	System Action
	1-The cart is empty.	
		2-The system displays a message indicating the cart is empty.
	3-Database errors occur during retrieval.	
		4-System notifies the user of a temporary error and retries fetching the data.
Includes:	Cart summary page, total price calculation.	
Notes and Issues:	Ensure accurate calculation of totals and user-friendly display.	

3.7.7 AI

Use Case ID:	AI-01	
Use Case Name:	AI Chatbot for Plant Help	
Actors:	User	
Pre-conditions:	The user is logged in and has access to the AI chatbot feature.	
Post-conditions:	The user receives plant-related assistance via chatbot.	
	User Action	System Action

The flow of events:	1- User opens the Chatbot screen.	
		2-System loads the chatbot interface and previous chat history.
	3-User types a question about plants.	
		4-System processes the query using AI and retrieves a contextual response.
	5-User receives and reads the reply.	
		6-System displays the answer with any related tips or links.
Exceptions:	User Action	System Action
	1-Chatbot service is unavailable.	
		2-System displays an error message and suggests retrying later.
	3-Query is unclear or unsupported.	
		4-System prompts user to rephrase or offer example questions.
Includes:	Natural language processing, chat history, contextual plant knowledge retrieval.	

Use Case ID:	AI-02	
Use Case Name:	Plant Identification	
Actors:	User	
Pre-conditions:	The camera and gallery permissions are granted.	
Post-conditions:	The user receives the plant's name and health status.	
The flow of events:	User Action	System Action
	1-User opens the Identification screen.	
		2-System activates the camera and displays scan animation.
	3 - User captures or uploads plant images.	
		4 - System processes the image(s) and identifies the plant.
	5 - User views the identification result.	
		6 - System displays the plant name, health status, and confidence score.
Exceptions:	User Action	System Action
	1-Image is unclear or low quality.	
		2-System asks the user to retake the photo or select a better one.

	3-Plant is not found in database.	
		4-System displays “Not identified” and offers tips for better images.
Includes:	Camera/gallery integration, AI image classification, error handling for image quality.	

Use Case ID:	AI-03	
Use Case Name:	Plant Recommendation	
Actors:	User	
Pre-conditions:	The user has access to the recommendation feature.	
Post-conditions:	The system suggests a list of suitable plants based on user preferences.	
The flow of events:	User Action	System Action
	1-User opens the Recommendation screen.	
		2-System loads the preference form or previous inputs.
	3 - User selects plant-growing preferences.	
		4 - System processes preferences and retrieves suitable plant list.
	5 - User reviews the plant suggestions.	

		6 - System displays recommended plants with images and descriptions.
Exceptions:	User Action	System Action
	1-No preferences selected.	
		2-System prompts user to select at least one criterion.
	3-No matching plants found.	
		4-System displays a message and suggests adjusting preferences.
Includes:	Preference form, filtering engine, plant detail viewer.	

3.7.8 Climate Data

Use Case ID:	CLIM-01	
Use Case Name:	Fetch and Display Climate Data	
Actors:	User, System	
Pre-conditions:	The app has permission to access location data and an active internet connection.	
Post-conditions:	Climate data is displayed to the user.	
The flow of events:	User Action	System Action
	1-User accesses the climate section.	

		2-System requests location data and sends it to the climate API.
	3-User waits for data.	
		4-The system retrieves climate data and displays it in a user-friendly format.
Exceptions:	User Action	System Action
	1-Location services are disabled.	
		2-The system prompts the user to enable location services.
	3-The Internet connection is unavailable.	
		4-System notifies the user and suggests retrying later.
	5-Climate API fails to respond.	
		6-System displays an error message and falls back to cached data if available.
Includes:	Location services, API integration for climate data.	
Notes and Issues:	Ensure the API handles location errors gracefully and updates climate data frequently.	

3.7.9 Additional Functionalities

Use Case ID:	FUNC-01	
Use Case Name:	Predictive Care Suggestions	
Actors:	User, System	
Pre-conditions:	Users have added plants to their gardens, and the system has adequate historical data for analysis.	
Post-conditions:	Predictive care suggestions are generated and displayed to the user.	
The flow of events:	User Action	System Action
	1-User selects a plant or garden section.	
		2-The system retrieves historical care data and plant-specific conditions.
	3-User views care suggestions.	
		4-The system generates predictive care tasks based on plant health and environmental factors.
Exceptions:	User Action	System Action
	1-Insufficient historical data.	
		2-System notifies the user and provides generic care tips instead.
	3-AI model fails to generate predictions.	

		4-The system logs the issue and displays a fallback suggestion message.
Includes:	Historical plant data analysis, AI model predictions.	
Notes and Issues:	Ensure the AI model is trained with sufficient data for reliable predictions.	

Use Case ID:	FUNC-02	
Use Case Name:	Plant Growth Tracking	
Actors:	User, System	
Pre-conditions:	Users have input or uploaded growth data for their plants (e.g., photos, height, or other metrics).	
Post-conditions:	Growth trends and insights are displayed to the user.	
The flow of events:	User Action	System Action
	1-User inputs plant growth data (e.g., height, photos).	
		2-The system stores the data and links it to the specific plant record.
	3-User views growth trends.	
		4-System analyzes the data and displays visual trends, such as graphs or charts.
Exceptions:	User Action	System Action
	1-Growth data entry is incomplete.	

		2-System prompts the user to complete missing fields.
	3-Data analysis fails.	
		4-System displays error messages and logs the issue for review.
Includes:	Data entry, growth analytics, visualizations.	
Notes and Issues:	Allow users to upload images and provide measurements for better insights. Ensure growth trends are accurate and easy to understand.	

Use Case ID:	FUNC-03	
Use Case Name:	Admin Dashboard – Product and Sales Management	
Actors:	Admin	
Pre-conditions:	Admin is logged in with appropriate access rights.	
Post-conditions:	Admin views and manages categories, products, customers, and sales analytics.	
The flow of events:	User Action	System Action
	1-Admin logs in and opens the dashboard.	
		2-System displays overview: categories, products, users, sales.
	3-Admin selects "Product List" tab.	
		4-System shows product list with filters and CRUD options.

	5 - Admin views "Sales Analytics" tab.	
		6 - System generates charts for sales trends, top products, revenue.
Exceptions:	User Action	System Action
	1-Data fails to load.	
		2-System displays a retry button and fallback “no data” message.
	3-Admin enters invalid input.	
		4-System shows validation messages for each field.
Includes:	Secure admin login, CRUD operations, dynamic chart rendering, search & filter functions.	

Shatla

Shatla



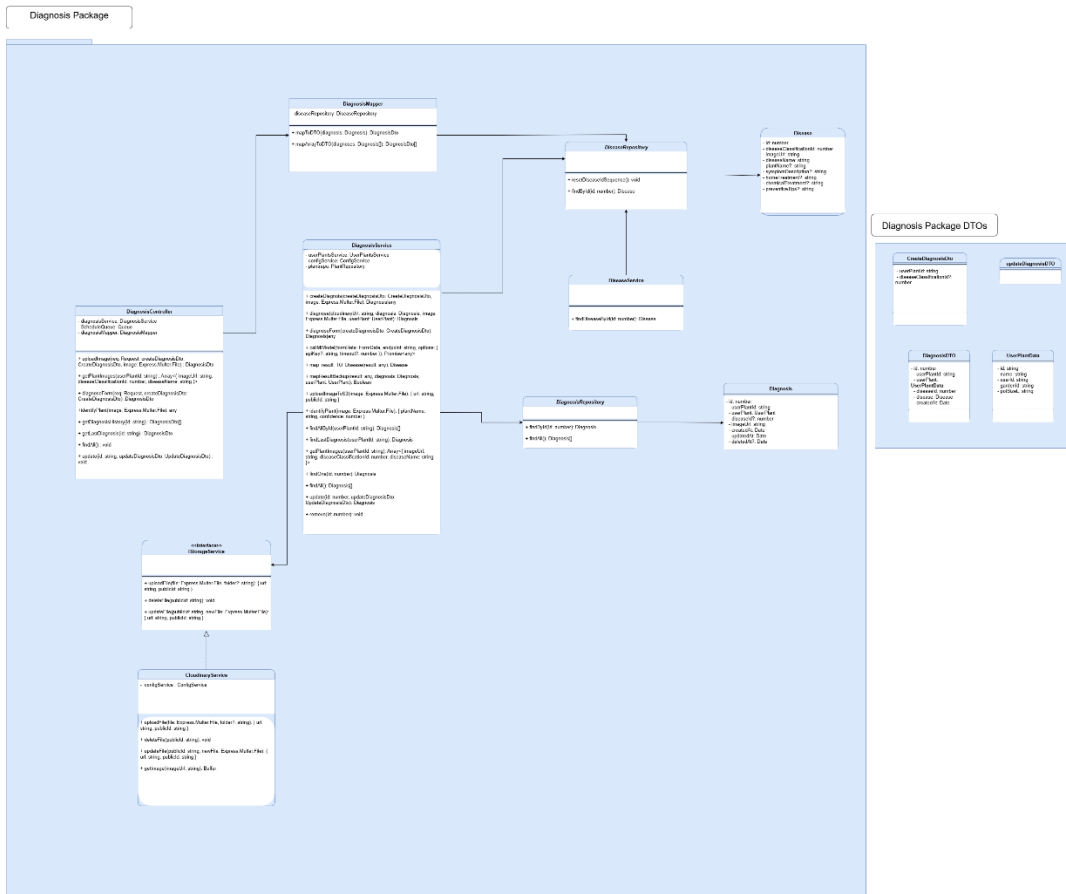


Figure 5 Diagnosis Package

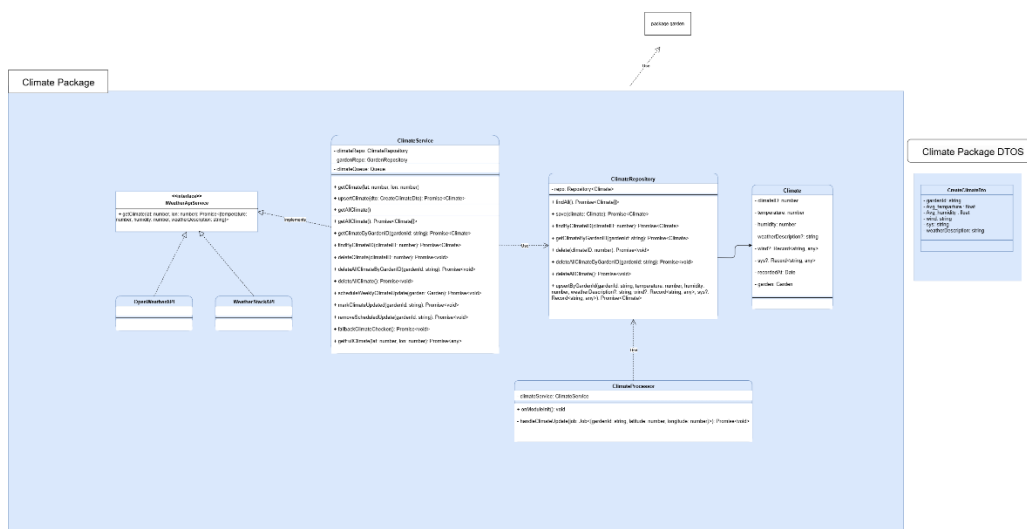


Figure 6 Climate Package

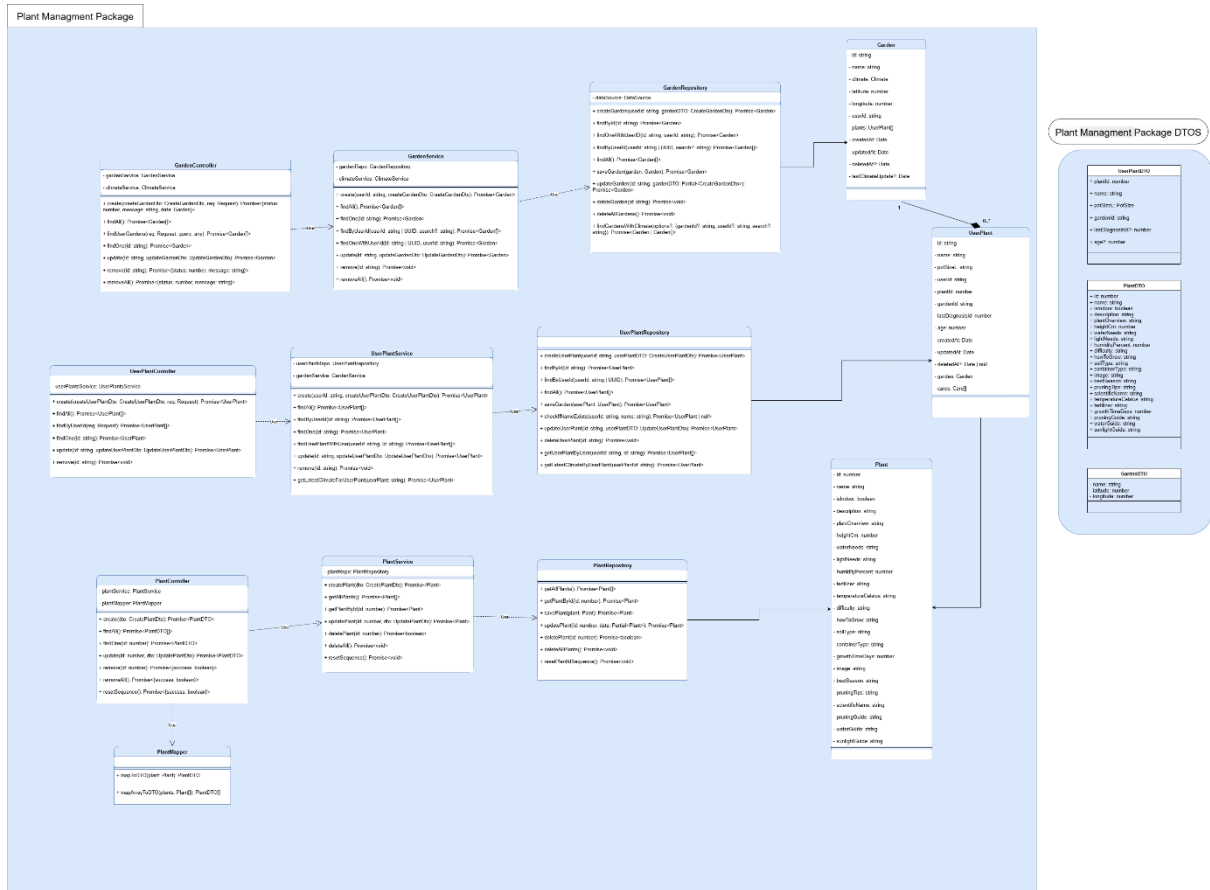


Figure 7 Plant Gardener Package

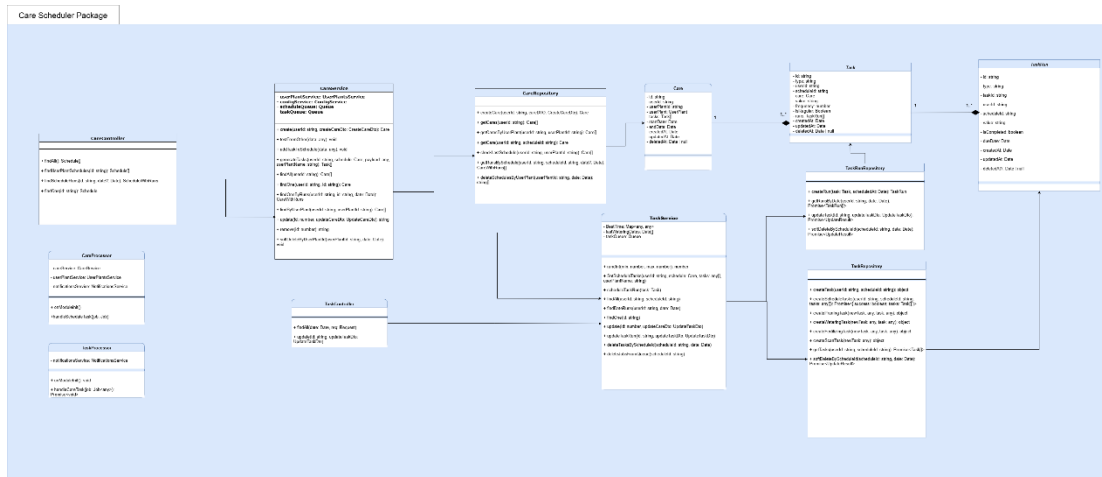


Figure 8 Care Scheduler Package

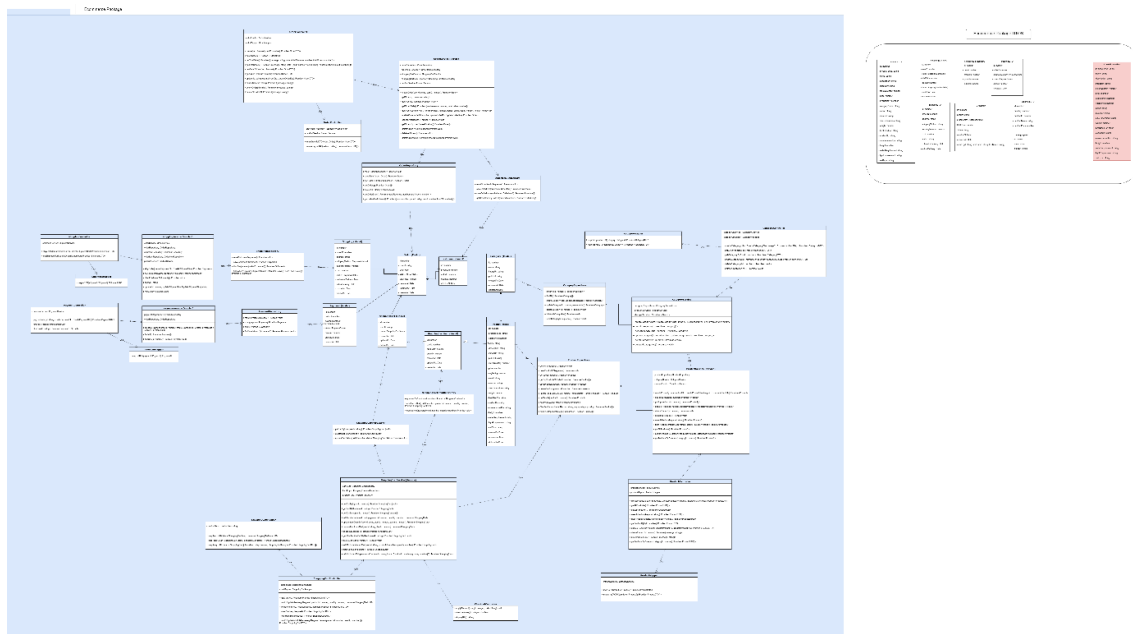


Figure 9 E-commerce Package

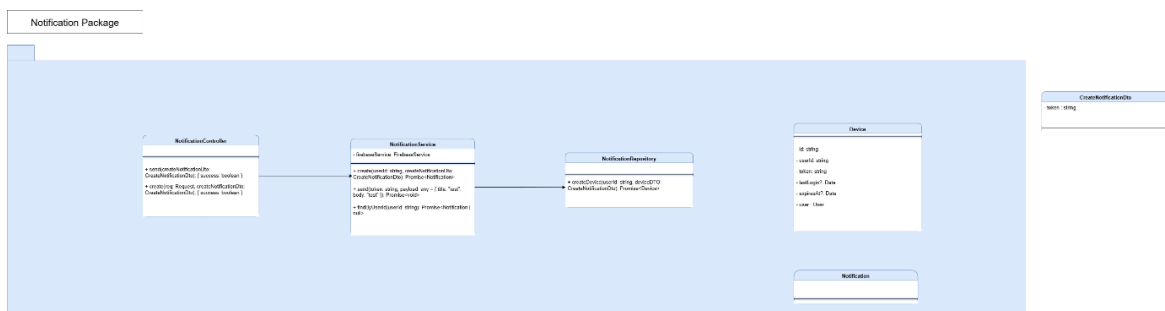


Figure 10 Notification package

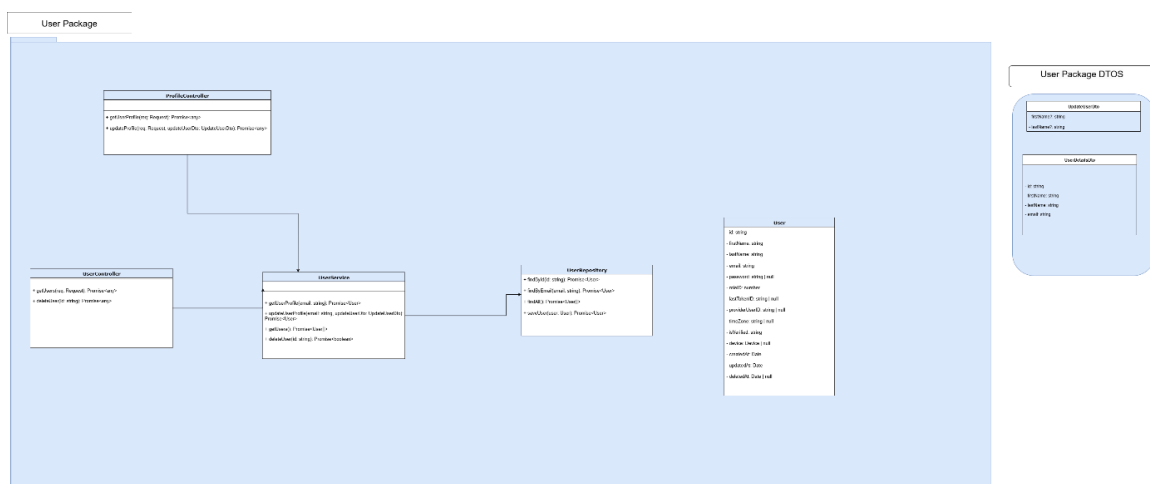


Figure 11 User Package

3.9 Sequence Diagrams

To view Clearer pictures of diagrams, visit this link: [Shatla Diagrams](https://shatla.com).



Disease Diagnosis Sequence Design

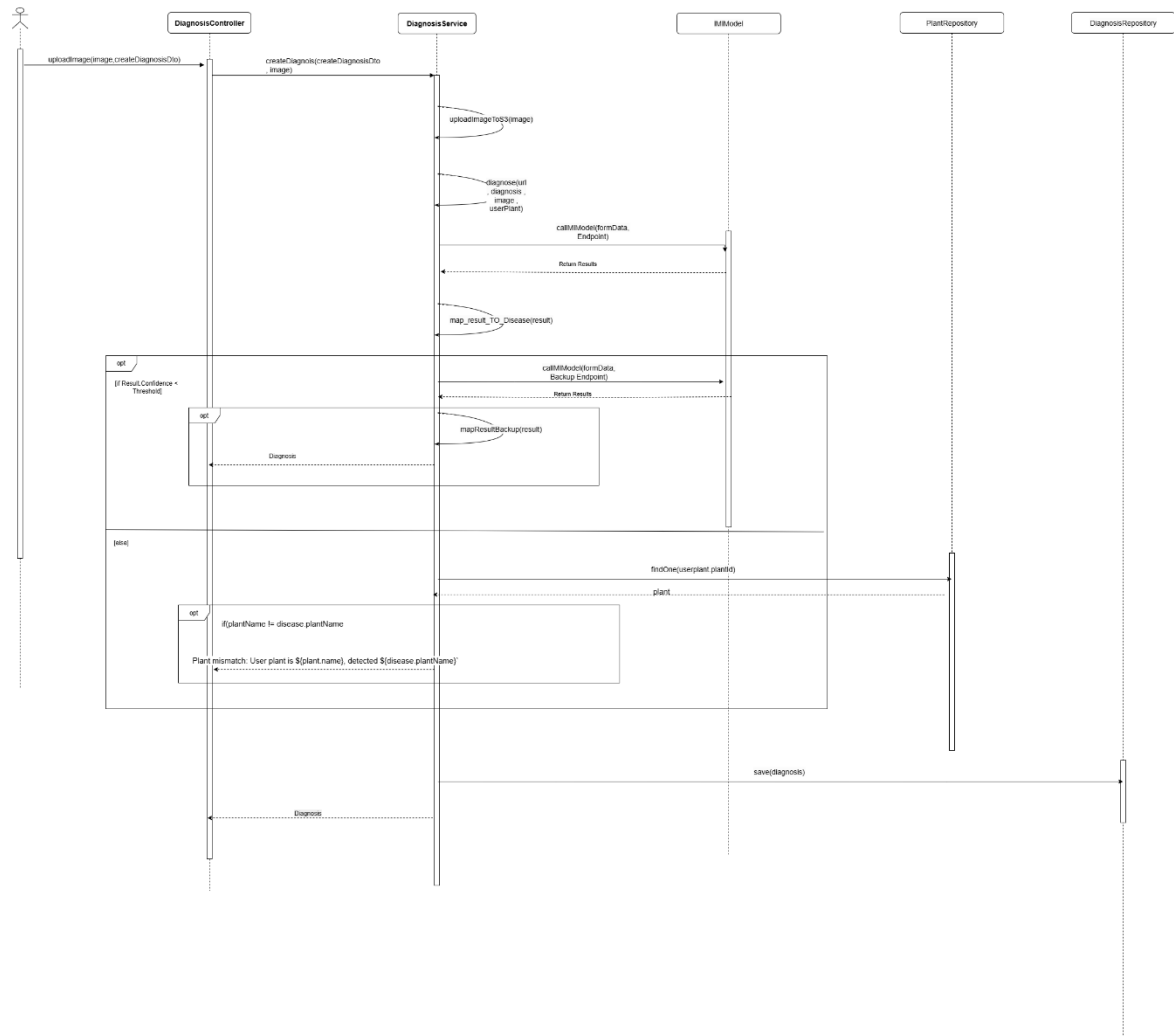


Figure 12 Disease Diagnosis Sequence

Disease Diagnosis Sequence Analysis

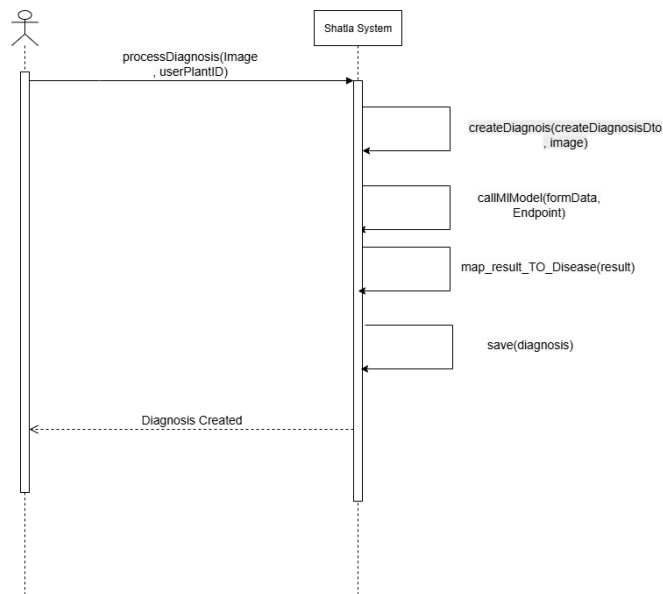


Figure 13 Disease Diagnosis Sequence Analysis

Care Schedule Sequence Design

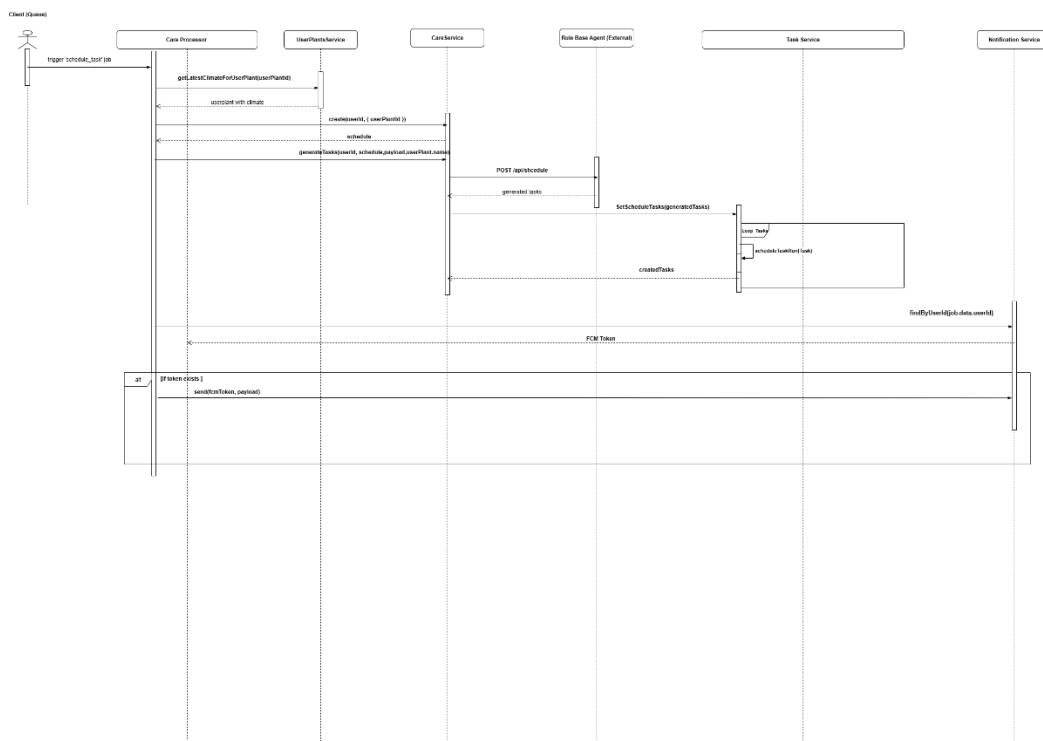


Figure 14 Care Schedule Sequence

Ecommerce Shopping Sequence Design

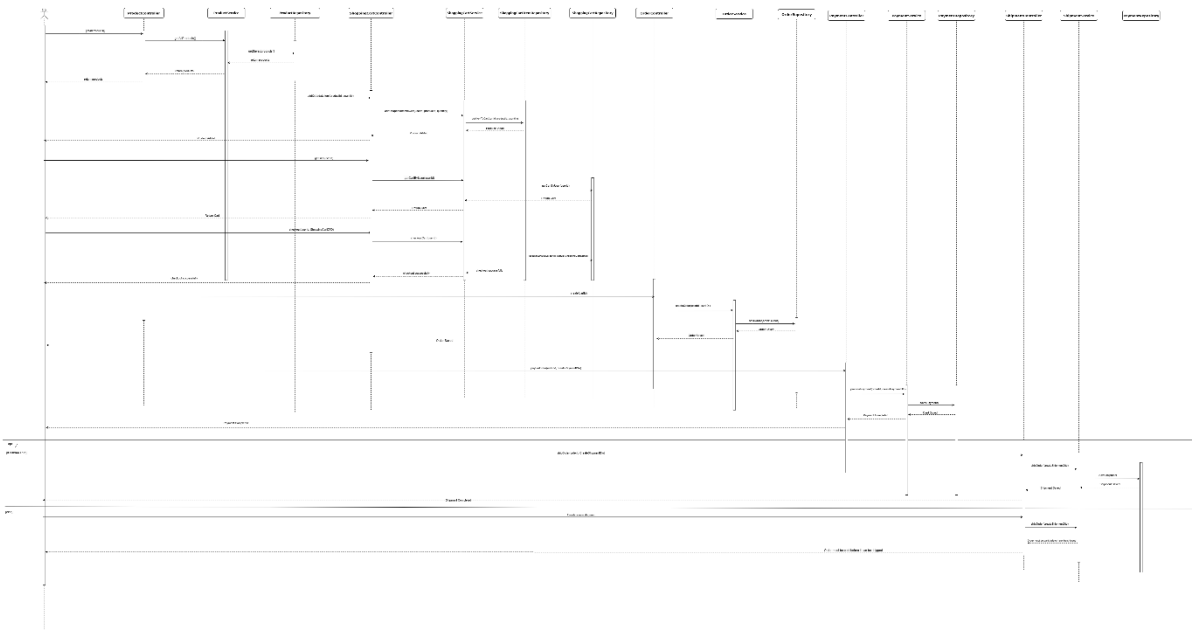


Figure 15 Ecommerce Shopping Sequence

Ecommerce Shopping Sequence Analysis

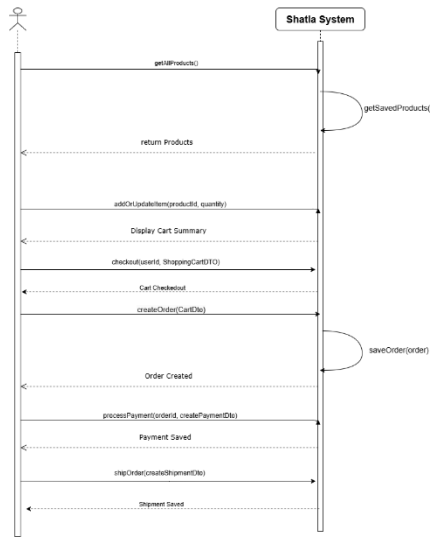


Figure 16 Ecommerce Shopping Sequence Analysis

3.10 ERD



To view Clearer pictures of diagrams, visit this link: [Shatla Diagrams](https://shatla.com).

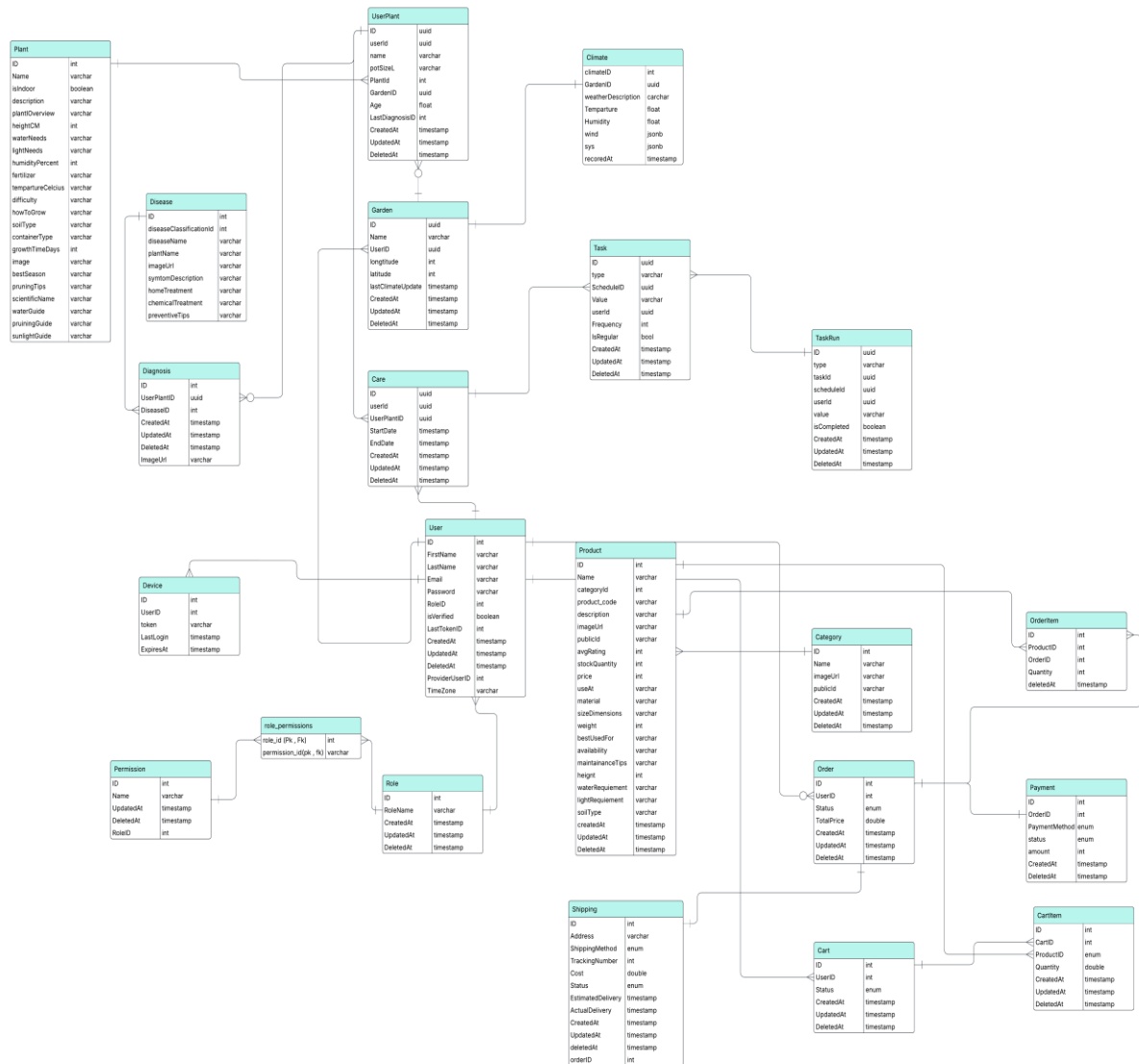


Figure 17 ERD

Chapter 4 Shatla Design and Implementation

This chapter delves into the design and implementation of the "**Shatla**" mobile application, detailing the tools, technologies, and methodologies employed to ensure seamless user experience. It includes the system architecture, workflow diagrams, and an explanation of the components and their integration.

4.1 System Architecture

The backend of the system is built using **Nest.js**, selected for its modular and highly scalable framework, making it ideal for building efficient and reliable RESTful APIs. Nest.js also offers robust support for TypeScript, enhancing maintainability and reducing runtime errors.

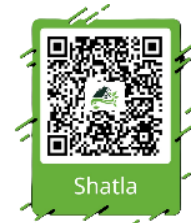
The front-end leverages **Flutter**, a powerful framework for building cross-platform mobile applications. Flutter was chosen for its ability to deliver a native performance experience and its hot-reload feature, which expedites the development process and improves productivity.

For artificial intelligence functionalities, **Python with FastAPI** is used. FastAPI provides high performance and an intuitive interface for developing and deploying machine learning endpoints. This integration ensures that plant diagnosis features, such as disease detection and care recommendations, operate efficiently.

The database system is powered by **PostgreSQL**, chosen for its reliability, scalability, and advanced support for structured data. PostgreSQL stores all critical data, including user profiles, plant schedules, diagnosis histories, and e-commerce records. Version control is managed using **Git**, ensuring collaboration among developers, and maintaining a history of all project changes. Project management is handled via **Jira**,

facilitating task tracking and sprint planning. Design and prototyping are conducted with **Figma**, while diagrams are created using **draw.io** and **lucid**.

System Architecture Diagram



To view Clearer pictures of diagrams, visit this link: [Shatla Diagrams](#)

The following diagram illustrates the high-level architecture of the "Shatla" system:

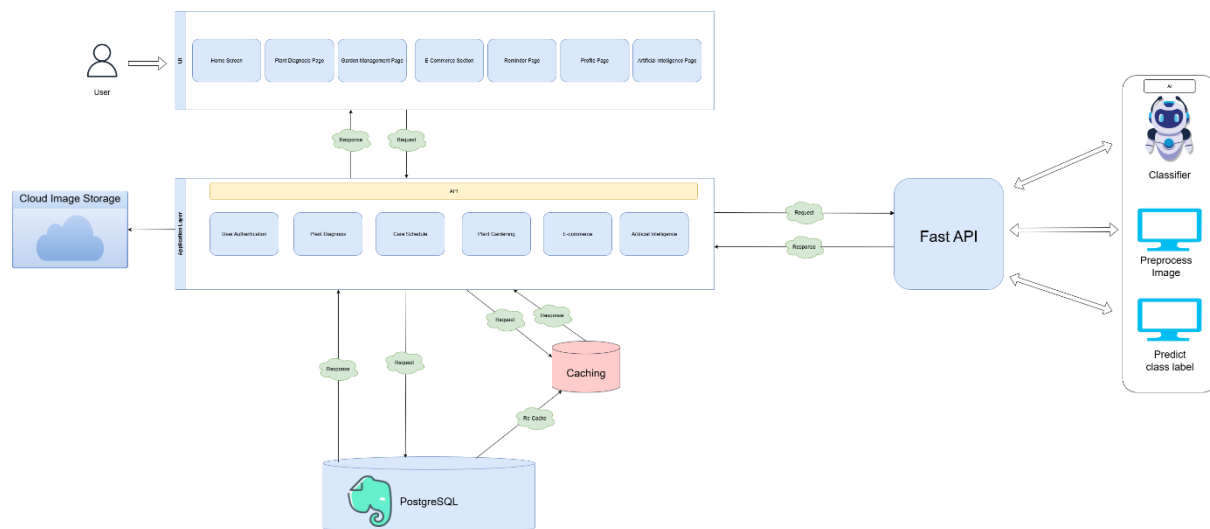


Figure 18 System Architecture

Workflow for Plant Diagnosis

The plant diagnosis workflow integrates user input, image analysis, and AI-based recommendations. The following steps outline the workflow:

1. **Image Capture:** The user captures or uploads a plant image.
2. **Preprocessing:** The system preprocesses the image to ensure it meets the AI model's input requirements.

3. **Disease Detection:** The AI model analyzes the image, detecting potential diseases.
4. **Results Display:** The diagnosis results, including detected diseases and severity, are presented to the user.
5. **Recommendations:** Based on the results, actionable care tips and recommendations are provided to the user.
6. **History Logging:** The results are saved in the user's plant history for future reference.

Workflow Diagram for Plant Diagnosis

To view Clearer pictures of diagrams, visit this link: [Shatla Diagrams](#)



Below is the workflow diagram showcasing the diagnosis process:

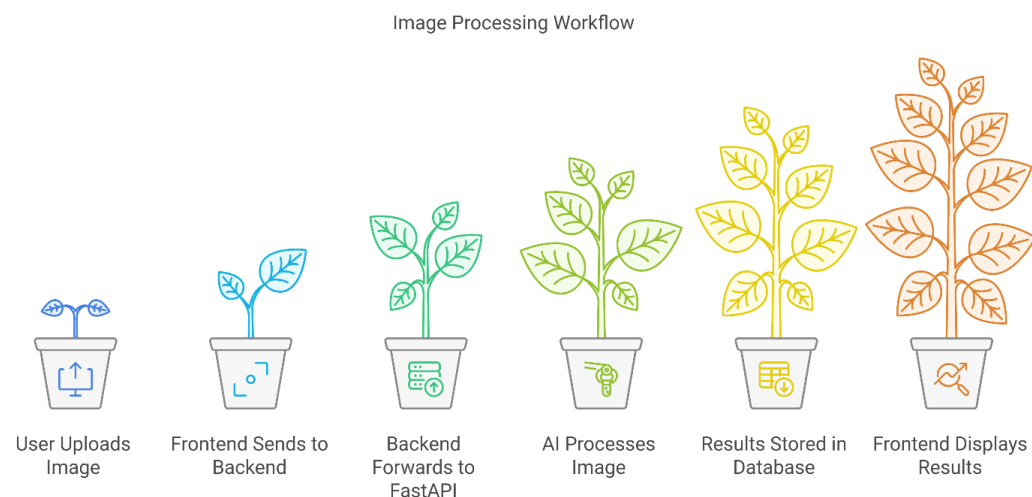


Figure 19 Diagnosis Workflow

This architecture and workflow ensure that "Shatla" provides a seamless experience, empowering home gardeners with the tools and insights needed to care for their plants effectively.

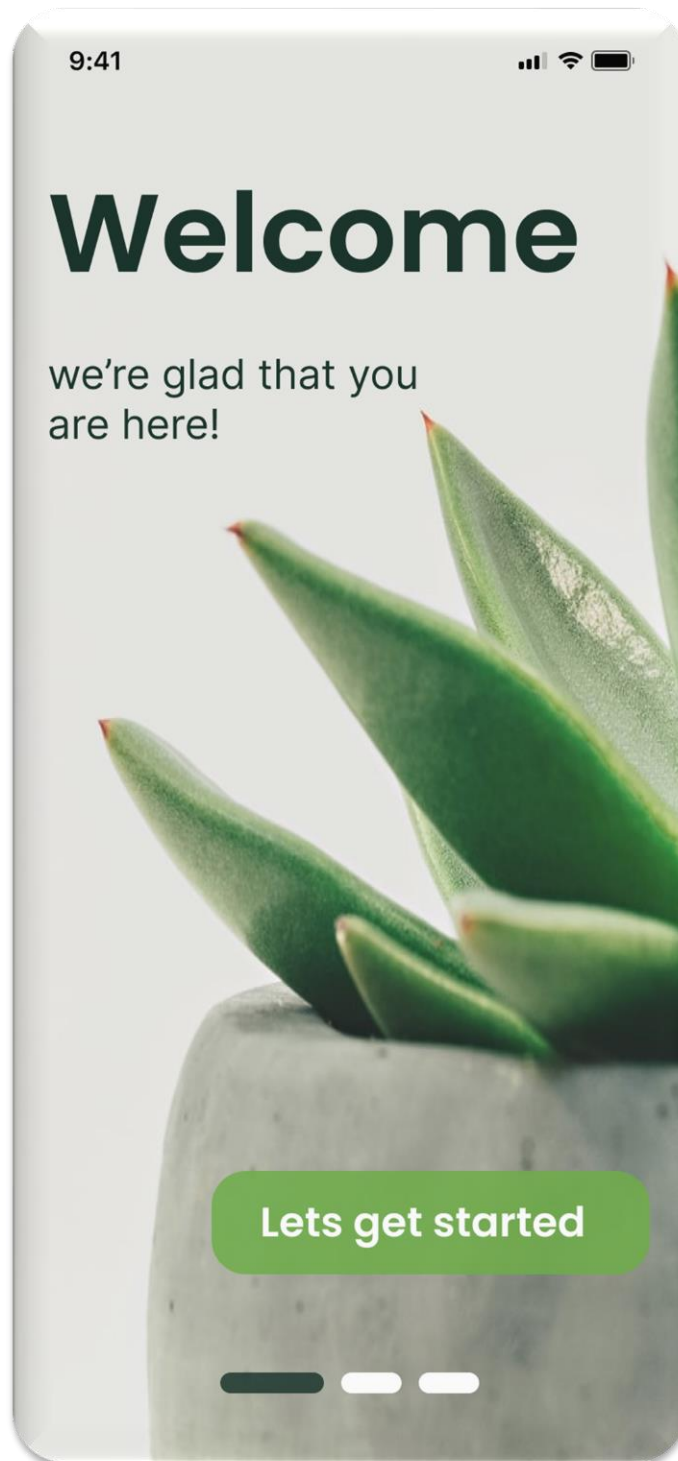
4.2 Prototype Screens

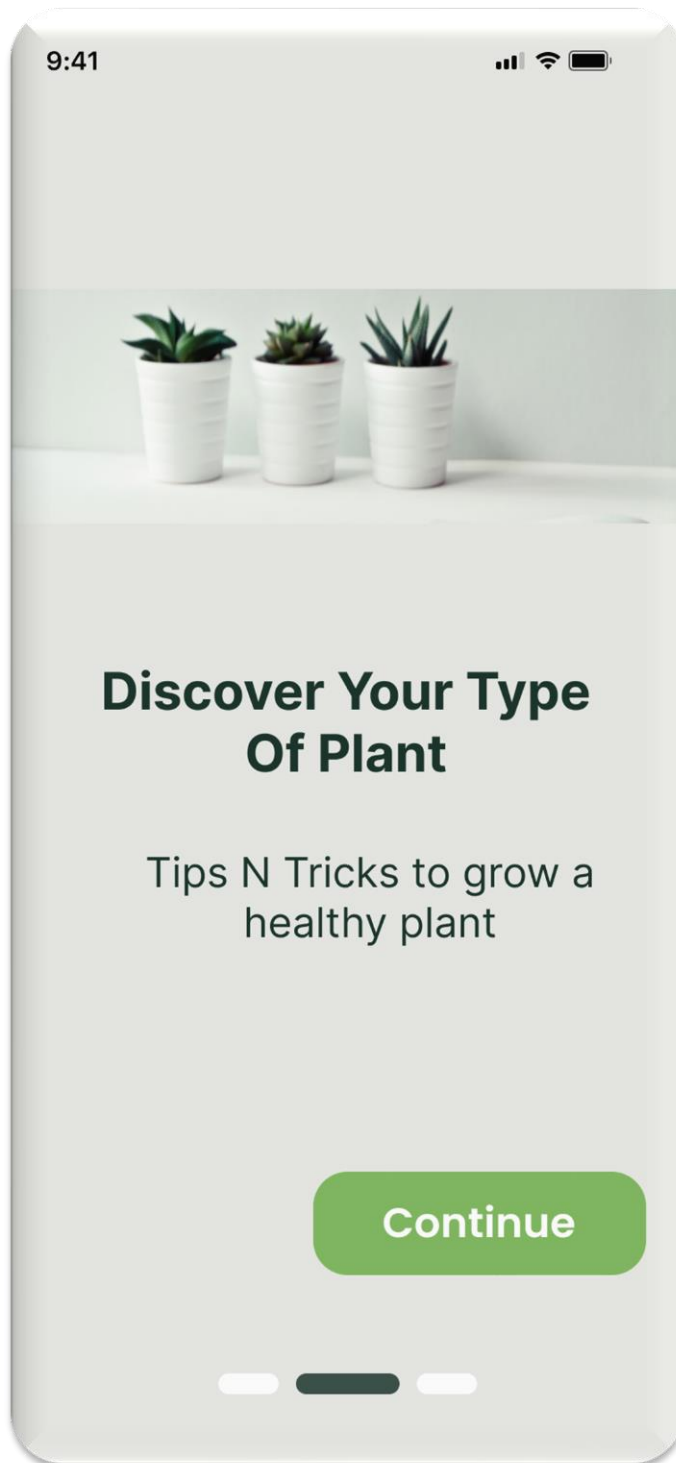
4.2.1 Mobile Screen

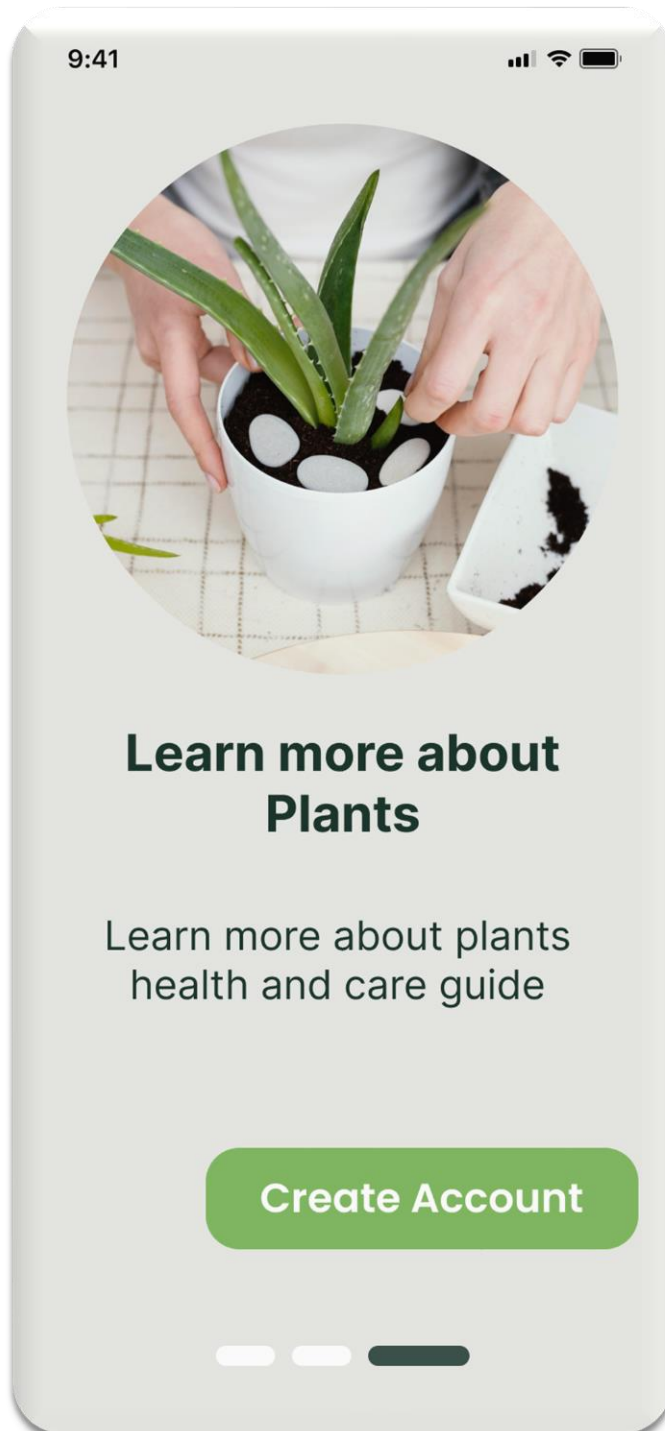
splash screen



Welcome page







Login

9:41



Welcome back! Glad to see you, Again!



[Forgot Password?](#)

Login

Don't have an account? [Register Now](#)

Or Login with

 Google

Register

9:41

SHATLA

Hello! Register to get started

Full Name

Username

Email

Phone Number

Password

Confirm password

Register

Already have an account? [Login Now](#)

Or Register with

 Google

Forgotten Password

9:41

<



Forgot Password?

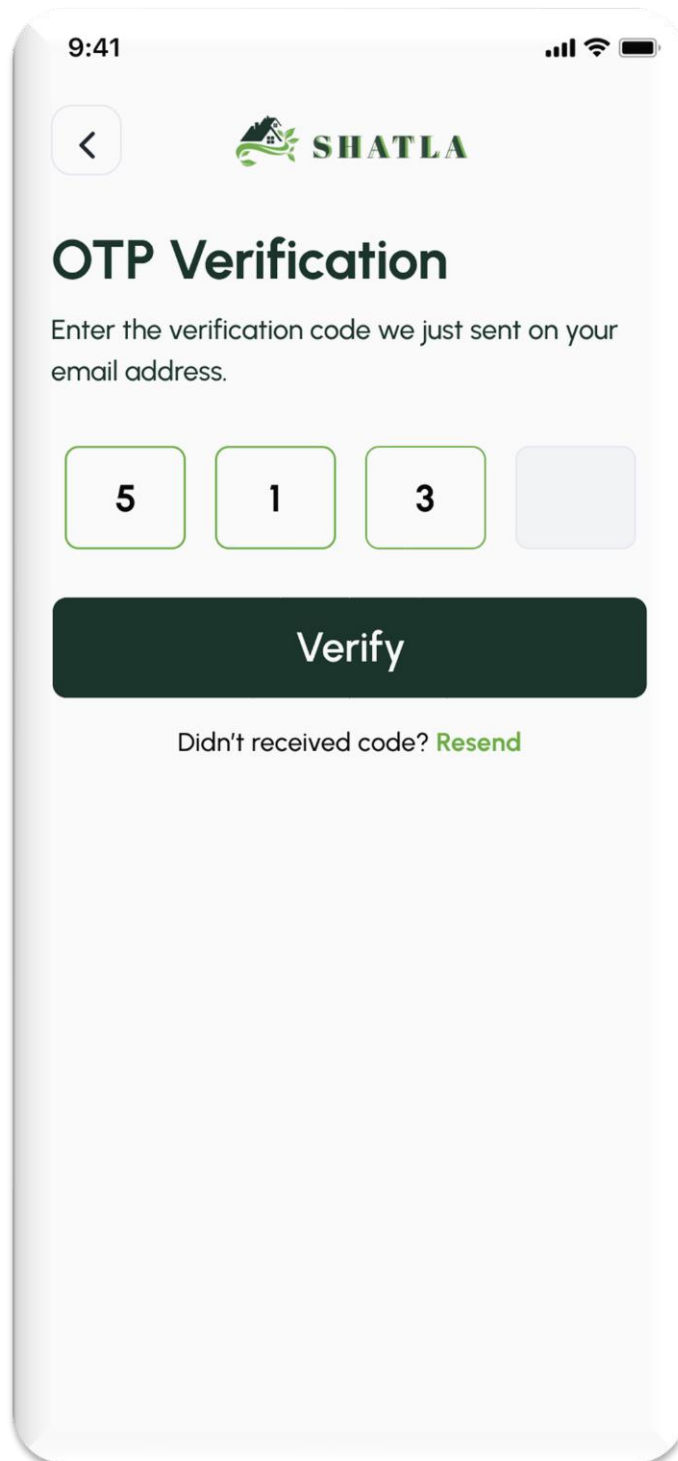
Don't worry! It occurs. Please enter the email address linked with your account.

Enter your email

Send Code

Remember Password? [Login](#)

OTP Verification



9:41

 SHATLA

OTP Verification

Enter the verification code we just sent on your email address.

5

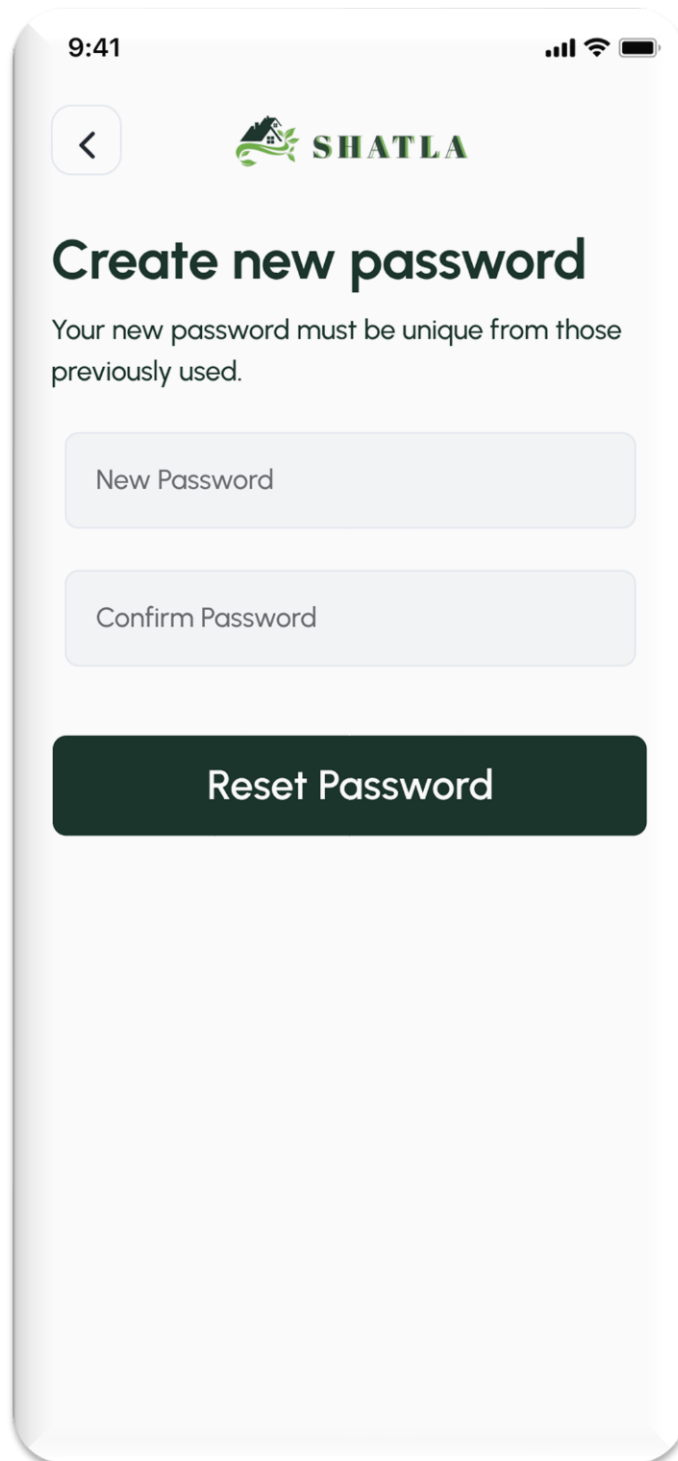
1

3

Verify

Didn't received code? [Resend](#)

Creating new password



The image shows a mobile application interface for creating a new password. At the top, the status bar displays the time 9:41, signal strength, Wi-Fi, and battery icons. Below the status bar is a navigation bar with a back arrow icon on the left and the SHATLA logo in the center. The main heading is "Create new password" in a large, bold, dark green font. Below the heading is a subtext: "Your new password must be unique from those previously used." There are two input fields: "New Password" and "Confirm Password", both with light gray borders and rounded corners. Below these fields is a large, dark green button with the text "Reset Password" in white. The background of the app is a light gray color.

9:41

 SHATLA

Create new password

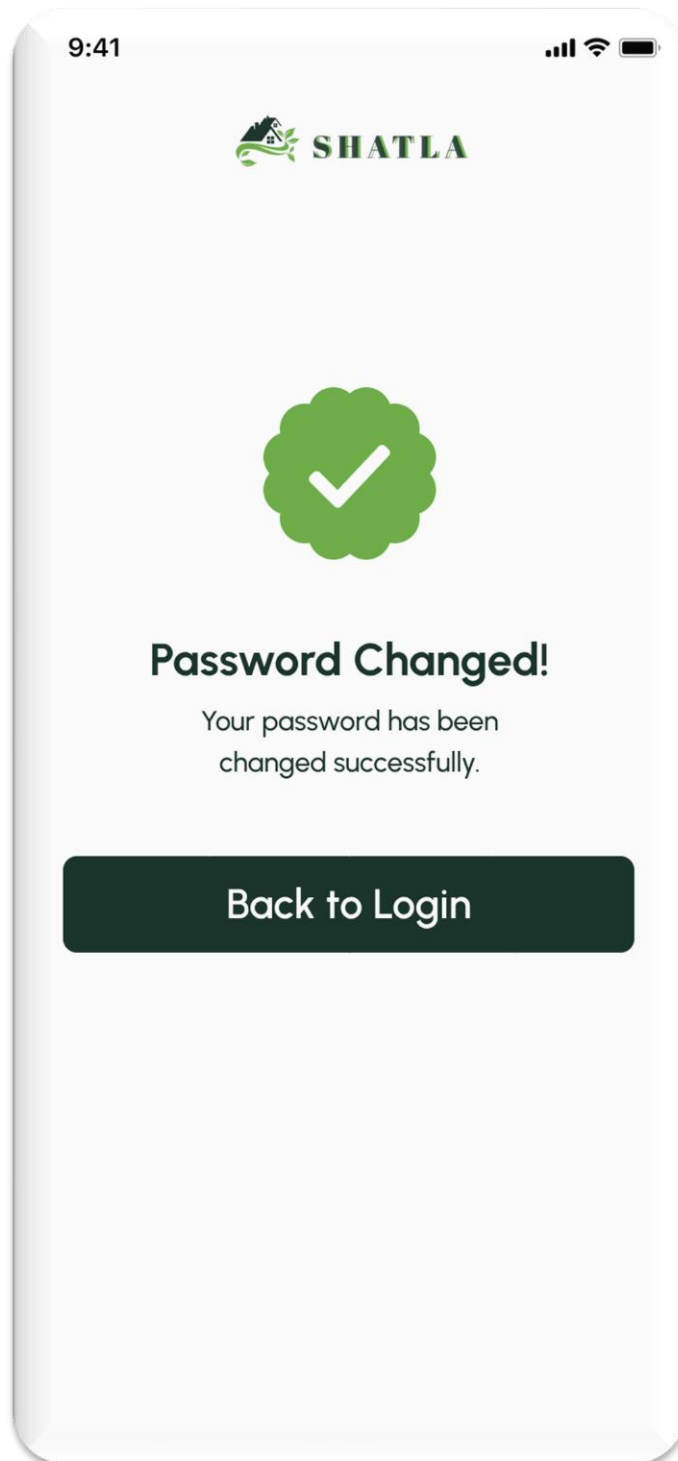
Your new password must be unique from those previously used.

New Password

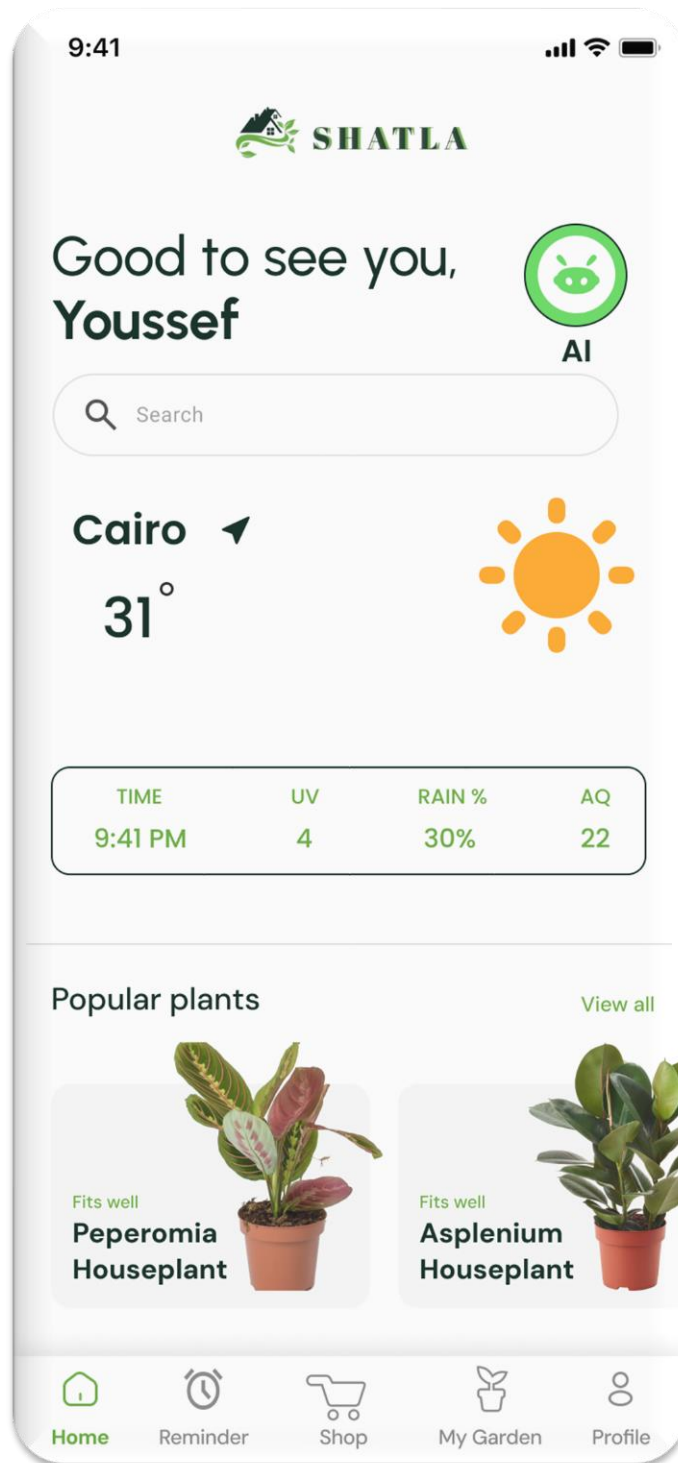
Confirm Password

Reset Password

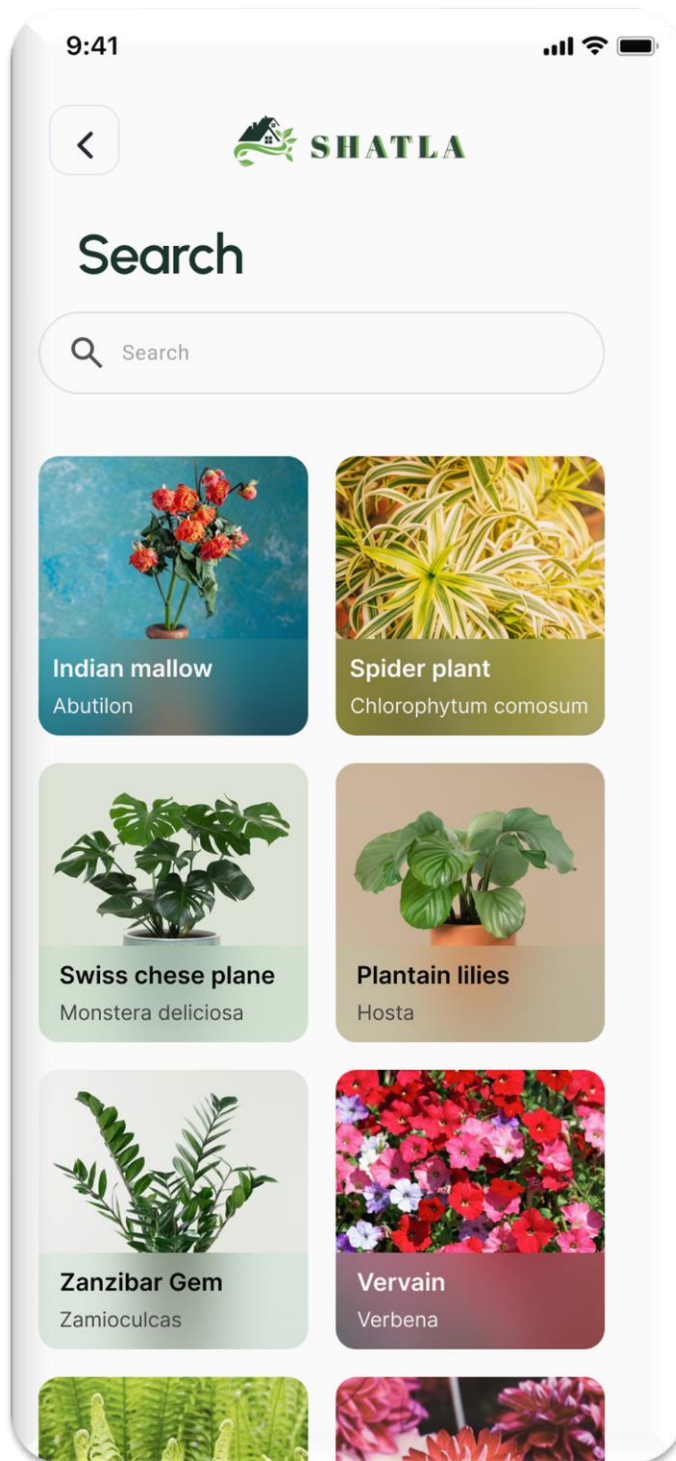
Password Changed



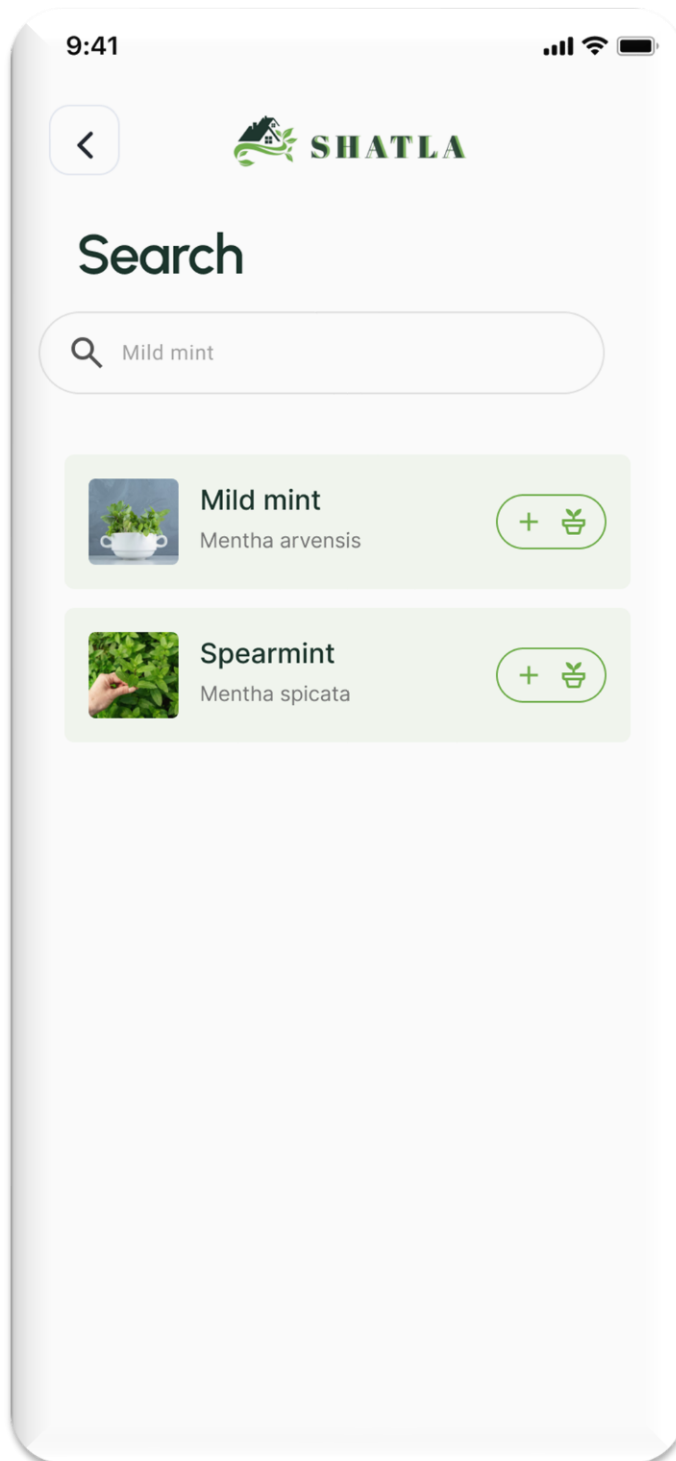
Homepage



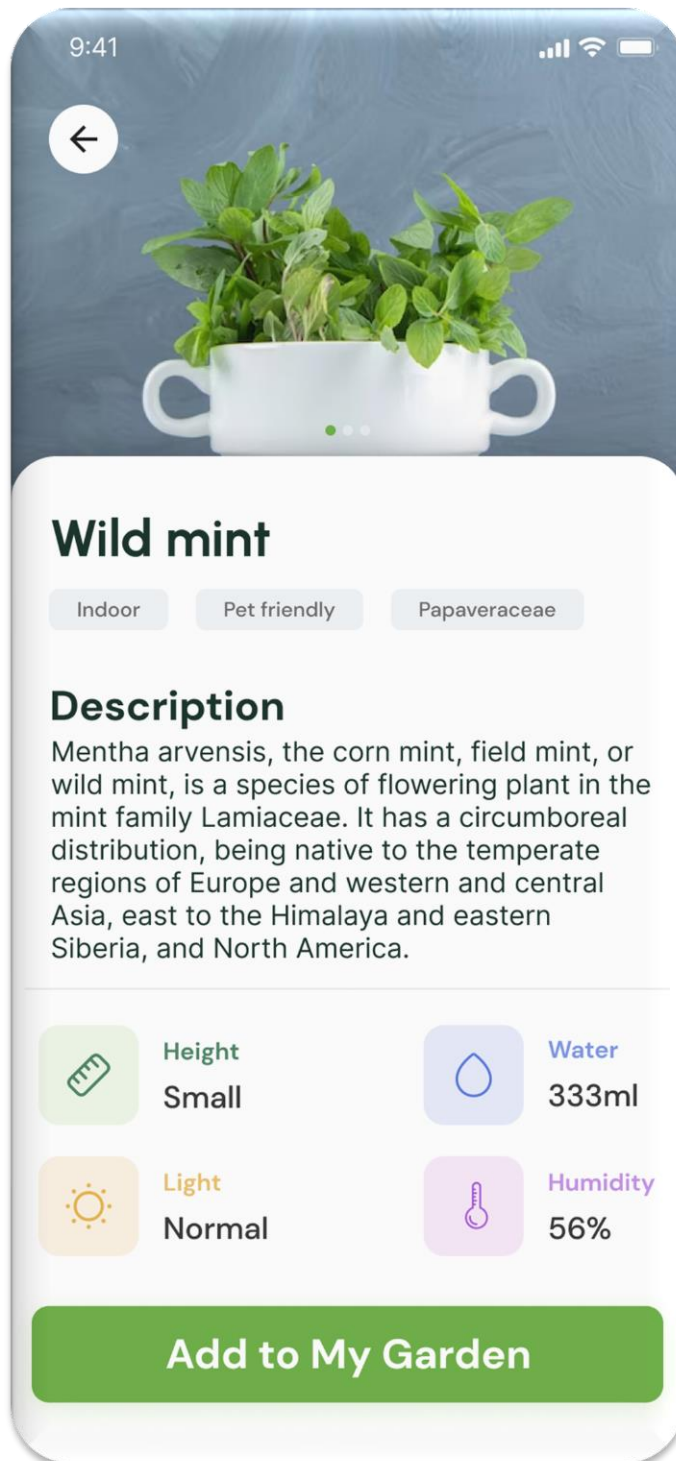
Search for plants



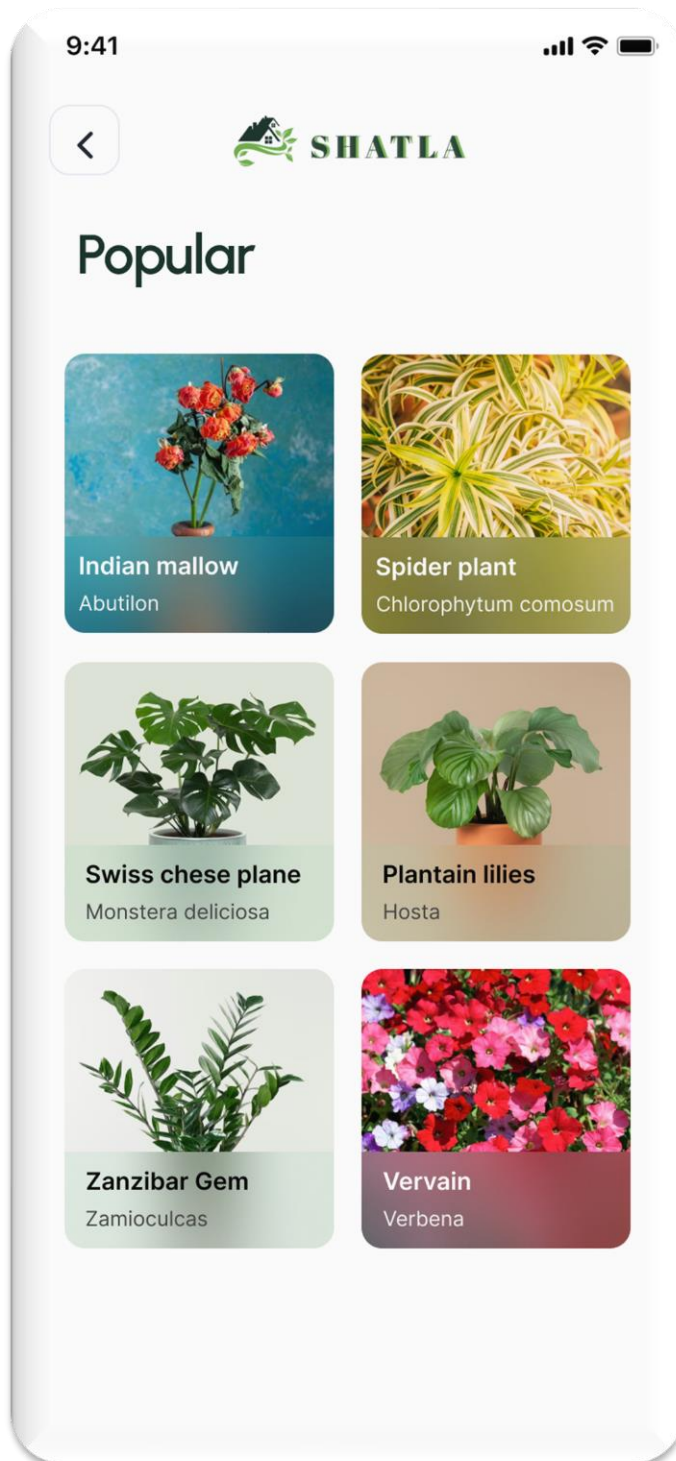
Searching



Plant details



Popular plants



AI Splash



AI Onboarding

You AI Assistant

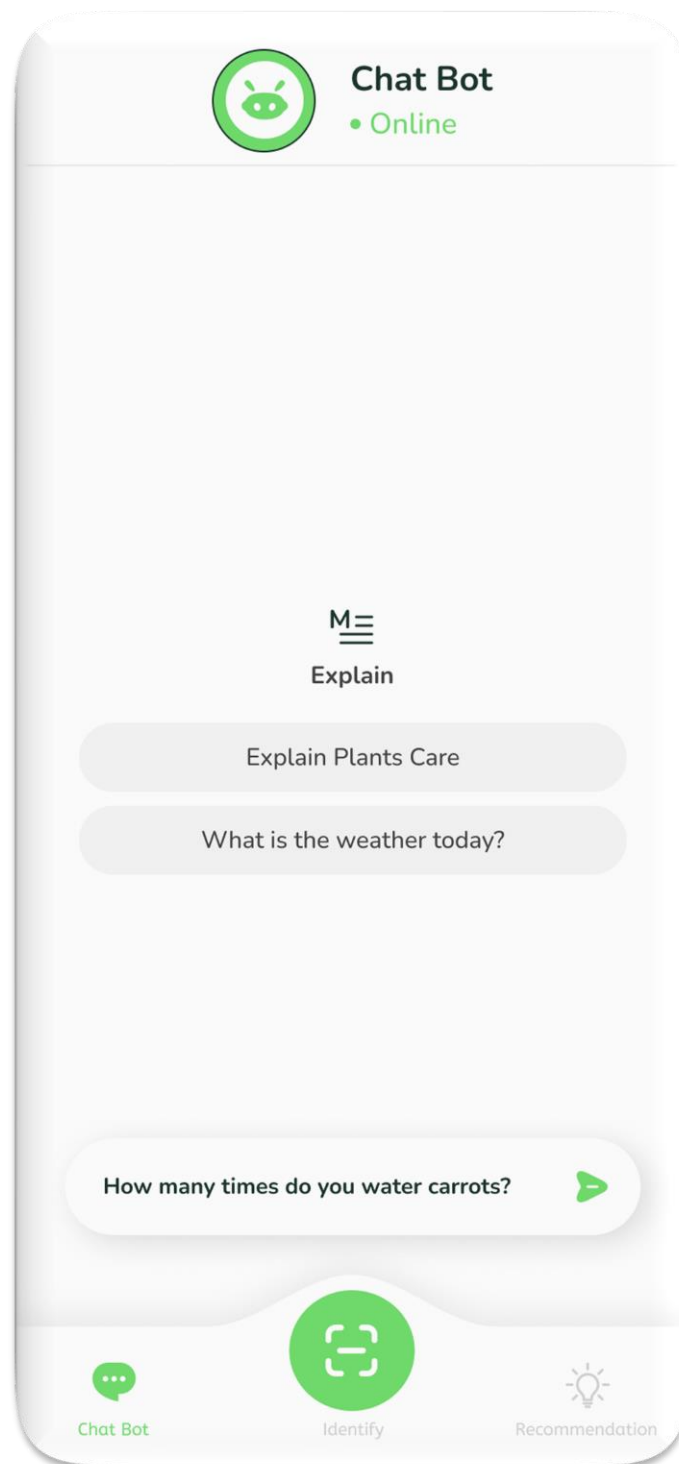
Using this software, you can ask you questions and receive articles using artificial intelligence assistant



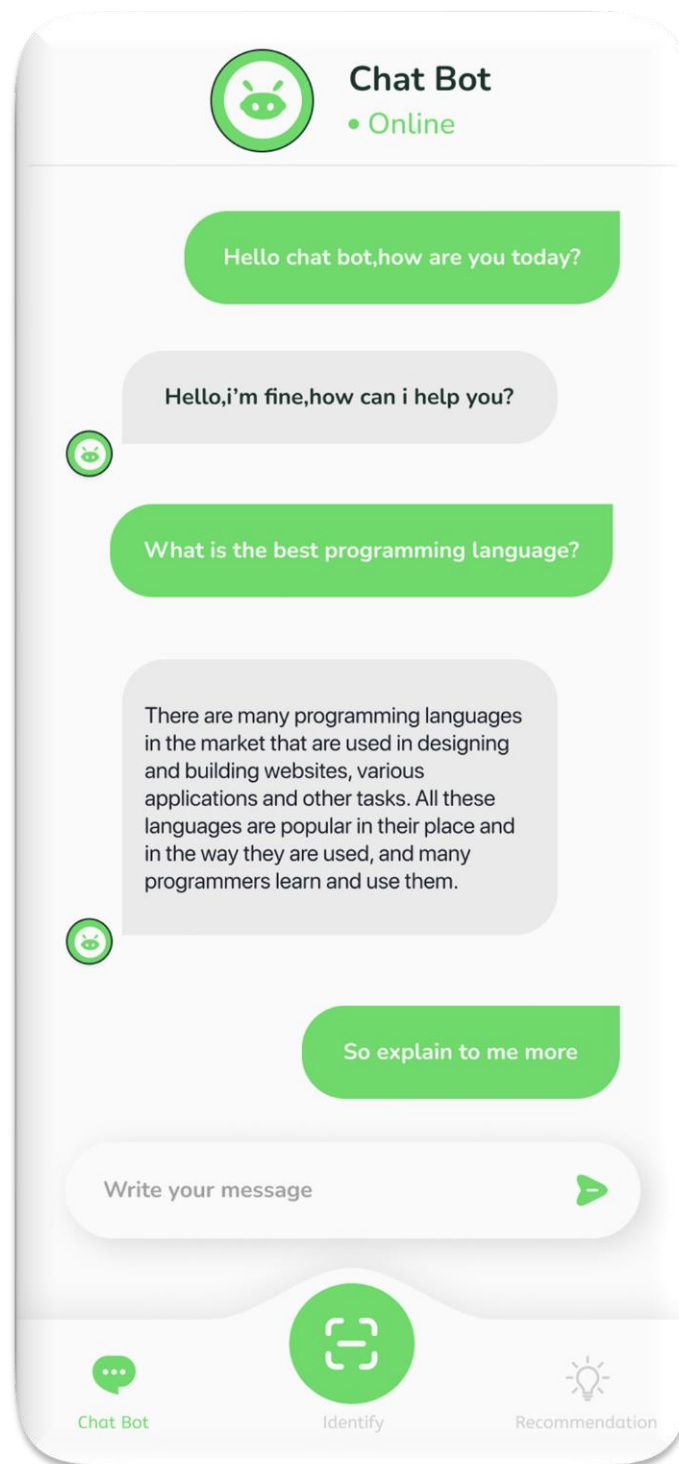
Continue



Chat Bot



Chat Bot Messaging



Recommendation

 **SHATLA**

Let's Find the Right Plant for You

Let's Find the Right Plant for You:

 Is it indoor?
[Yes] [No]

 Water Needs
[Low] [Medium] [High]

 Light Needs
[Full Sunlight] [Indirect Sunlight] [Partial Shade]
[Shade]

 Fertilizer
[Weekly] [Monthly] [None]

 Difficulty
[Easy] [Medium] [Hard]

 Soil Type
[Sandy] [Loamy] [Clay] [Peaty] [Chalky]
[Silty]

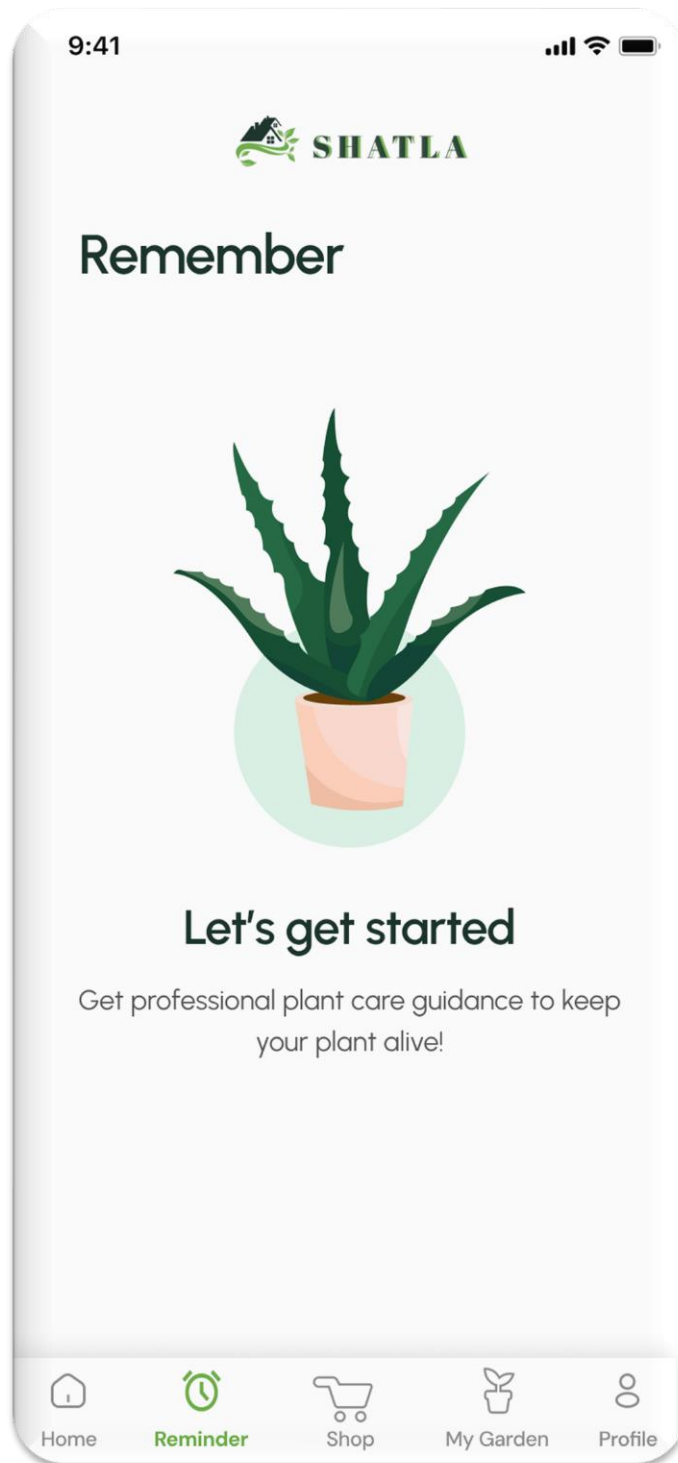
 Best Season
[Spring] [Summer] [Autumn] [Winter]
[All Year]

 Chat Bot

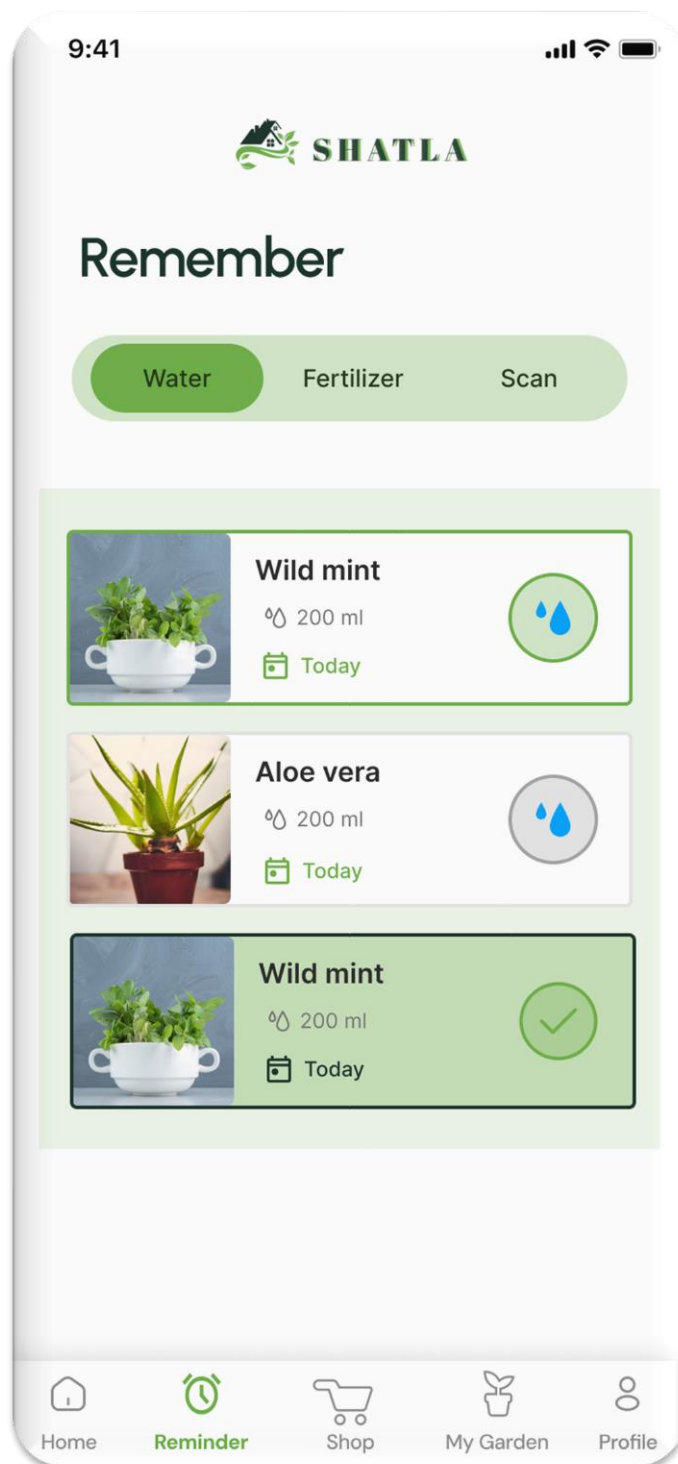
 Identify

 Recommendation

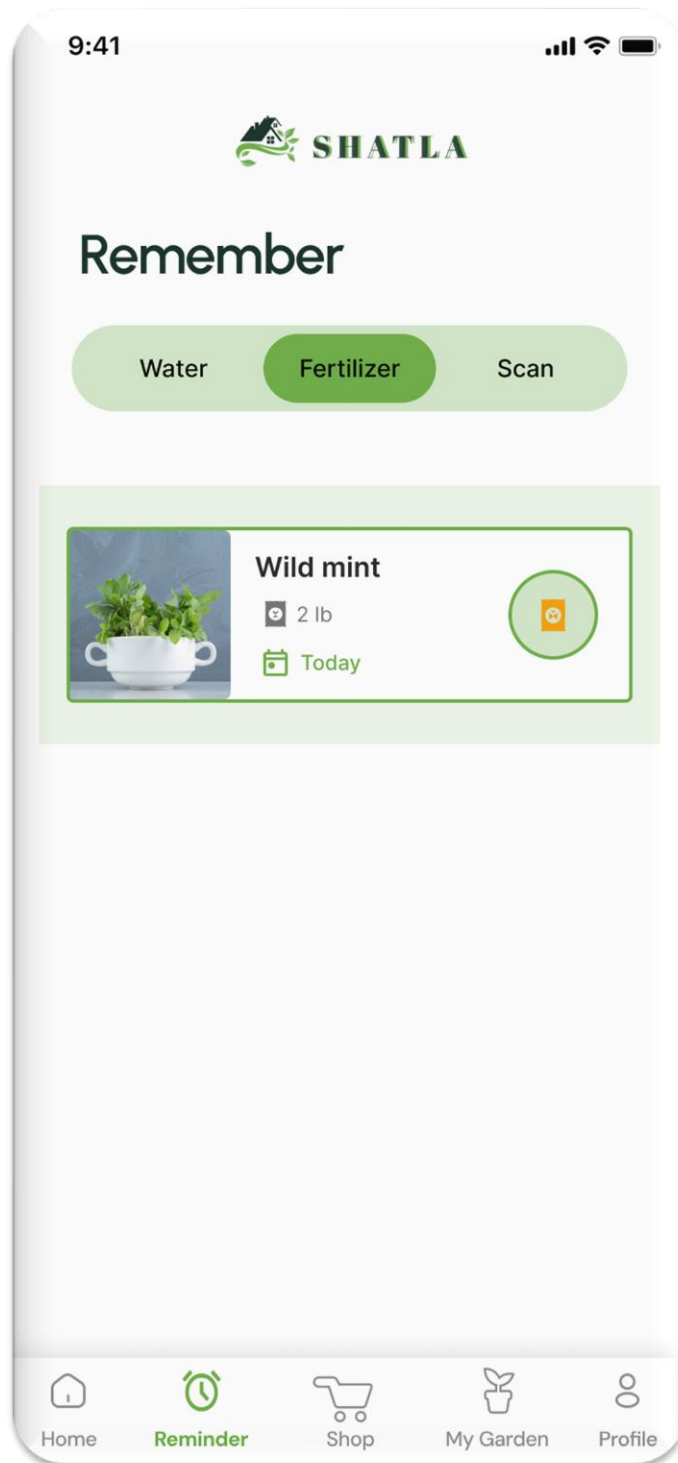
Reminder



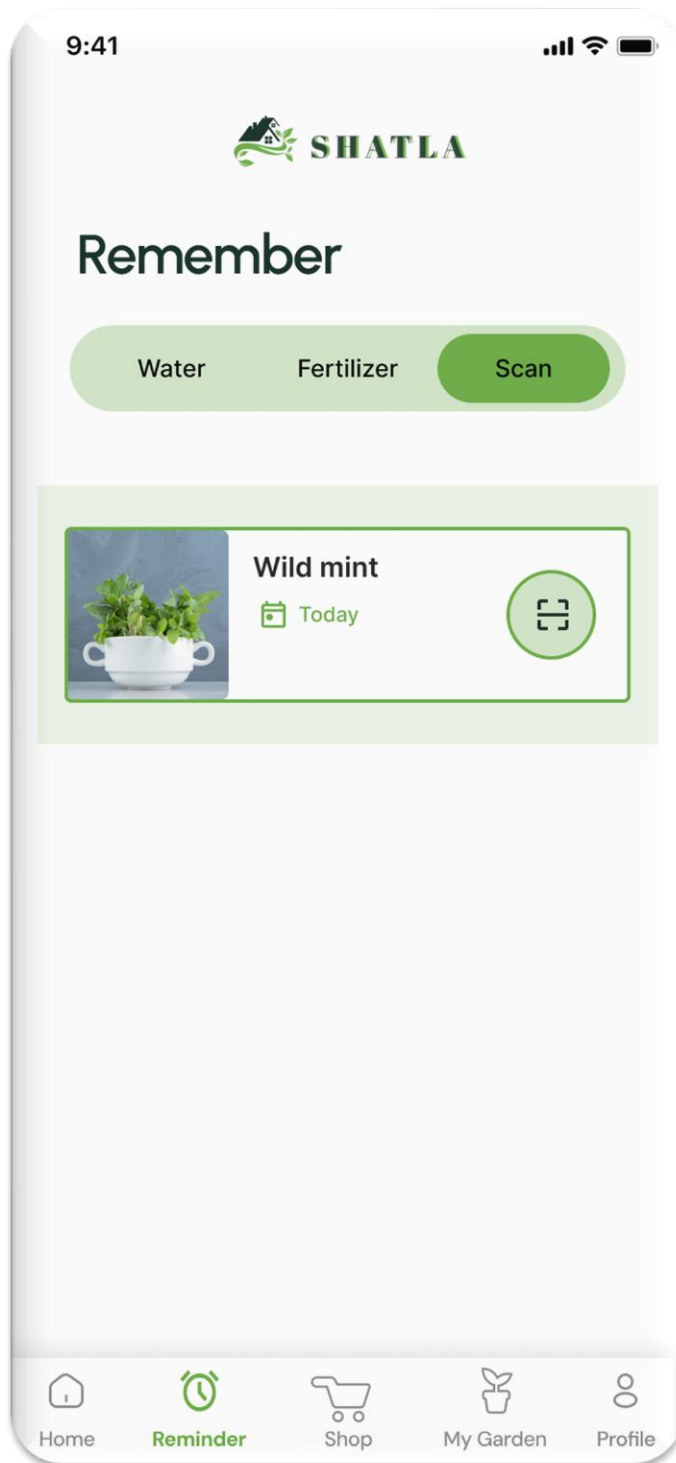
Reminder-Water



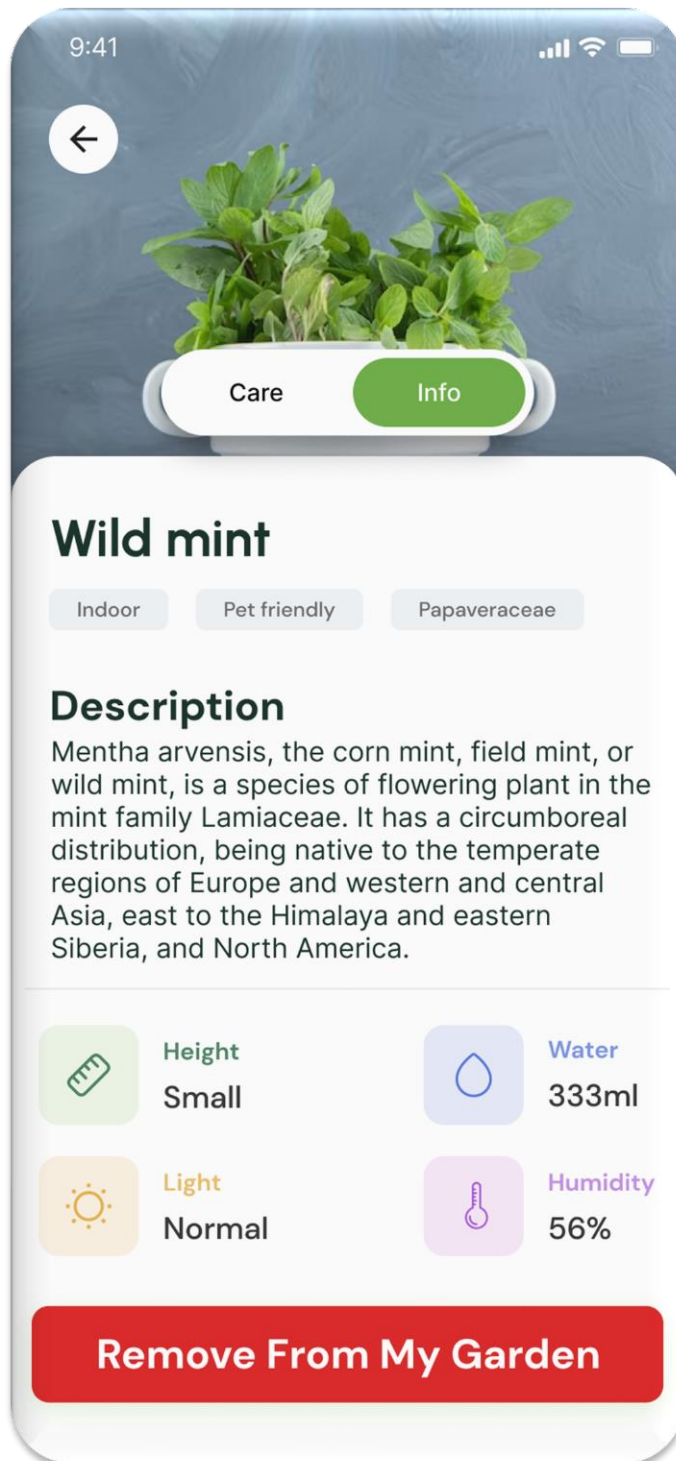
Reminder-Fertilizer



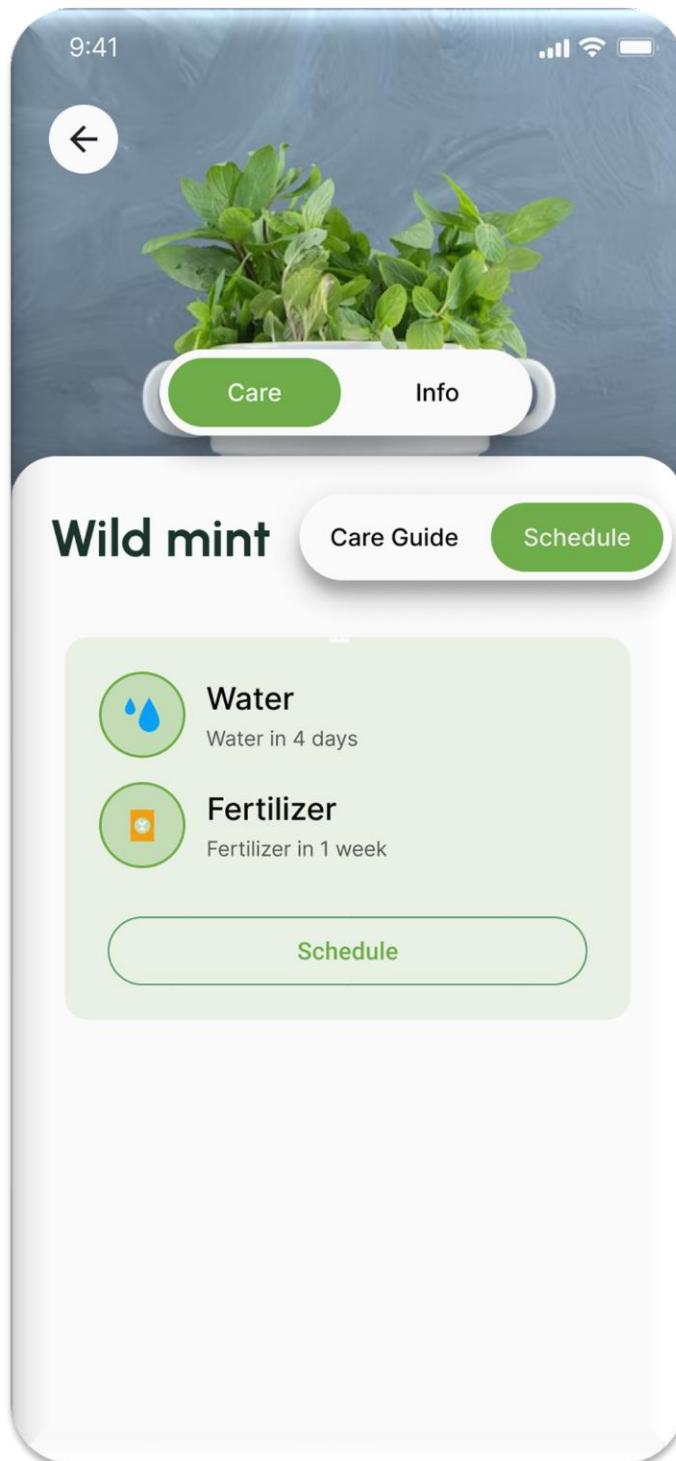
Reminder-Scan



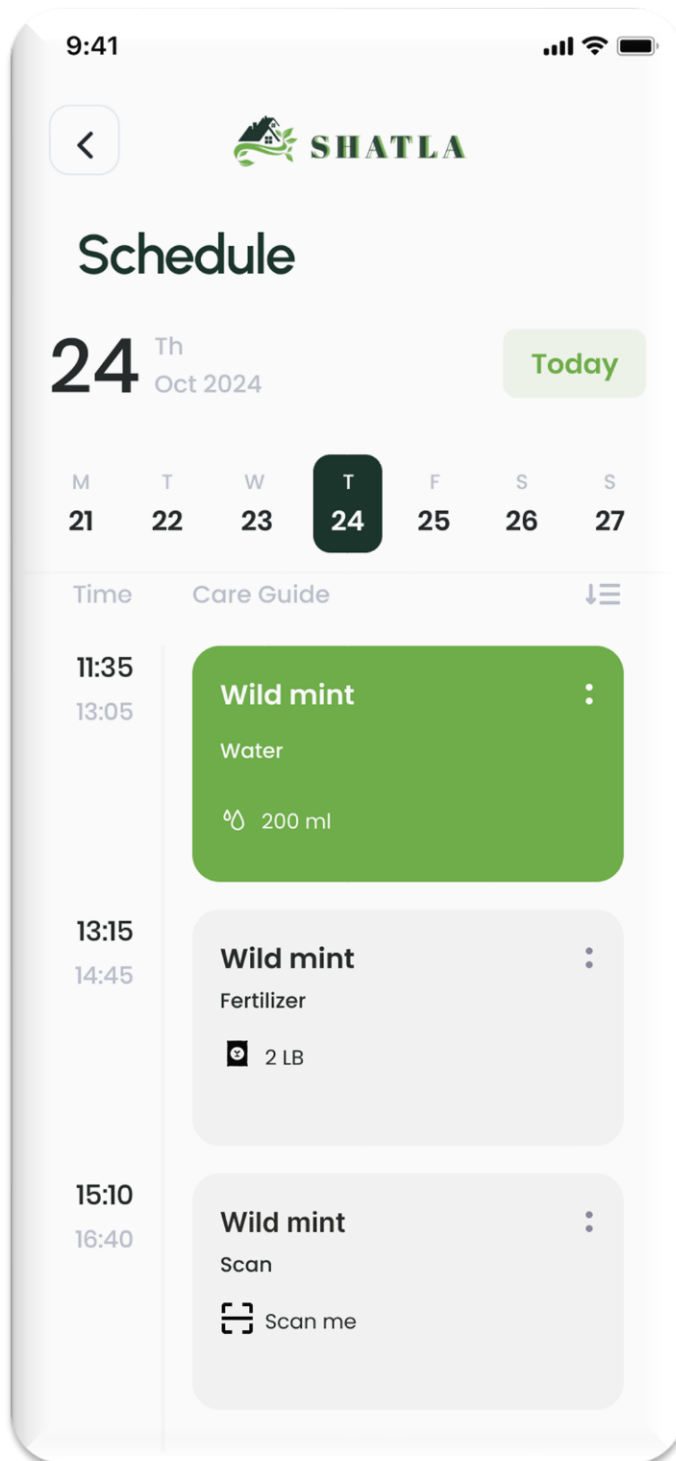
Care Details



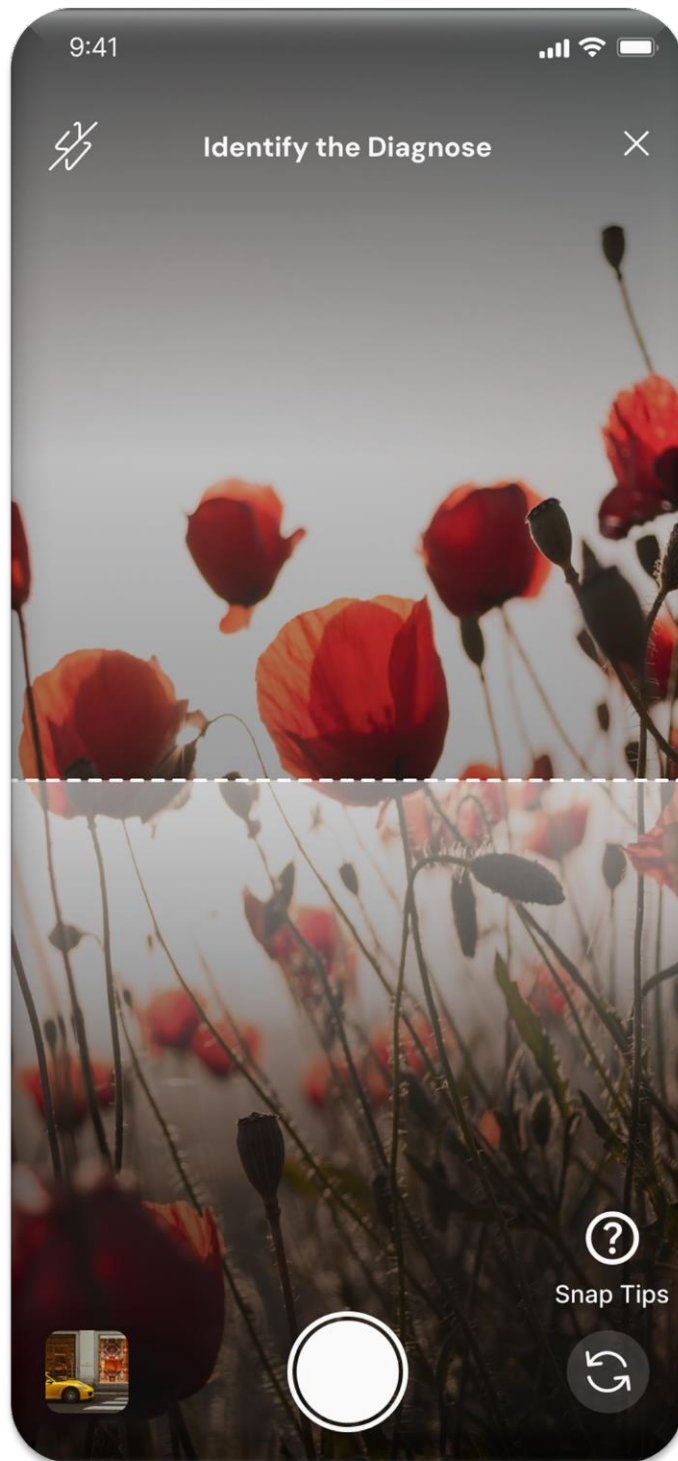
Care Frequency



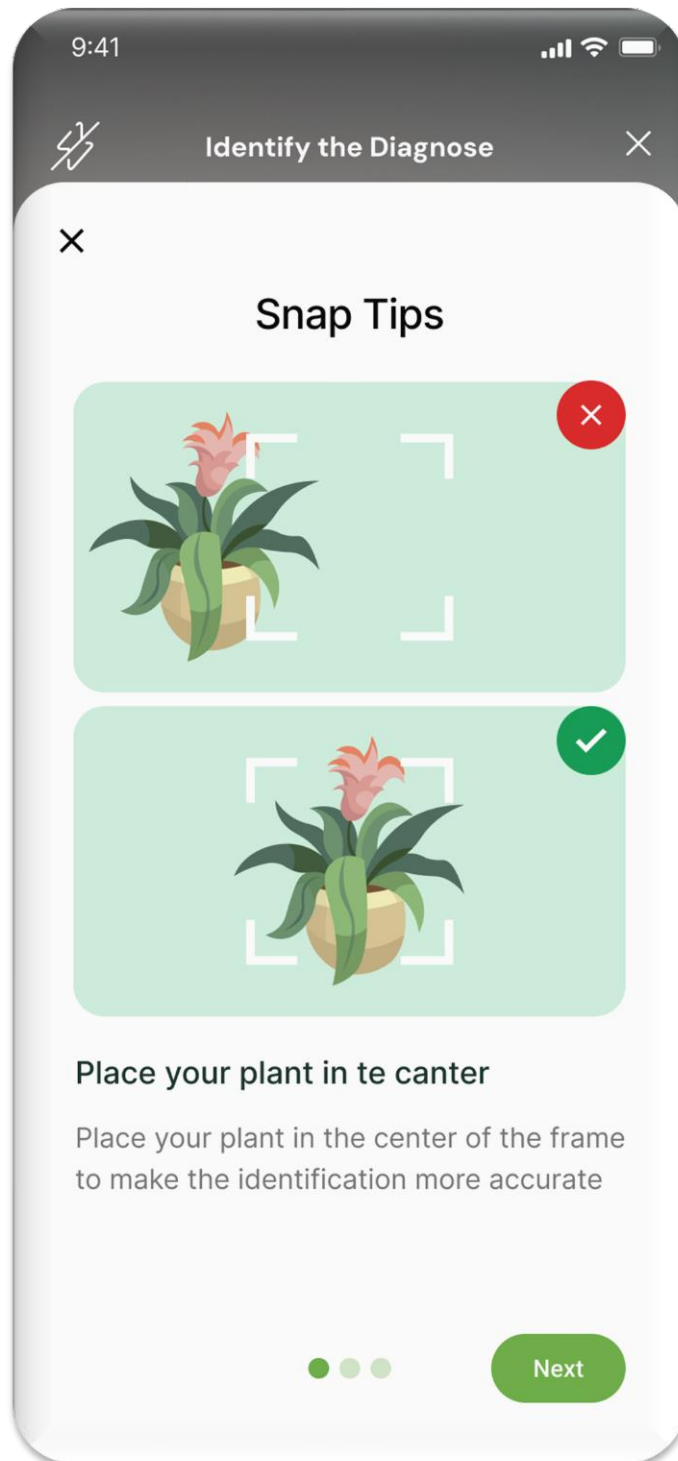
Schedule



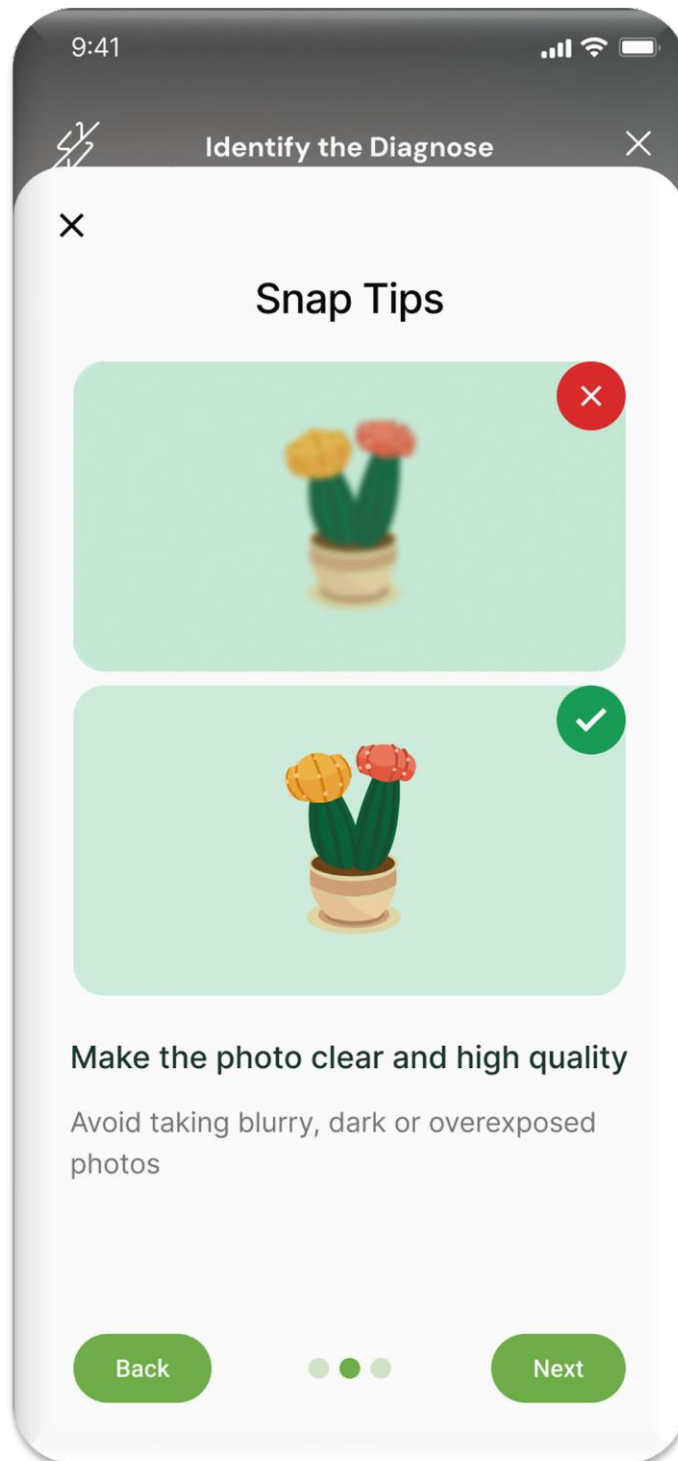
Scan



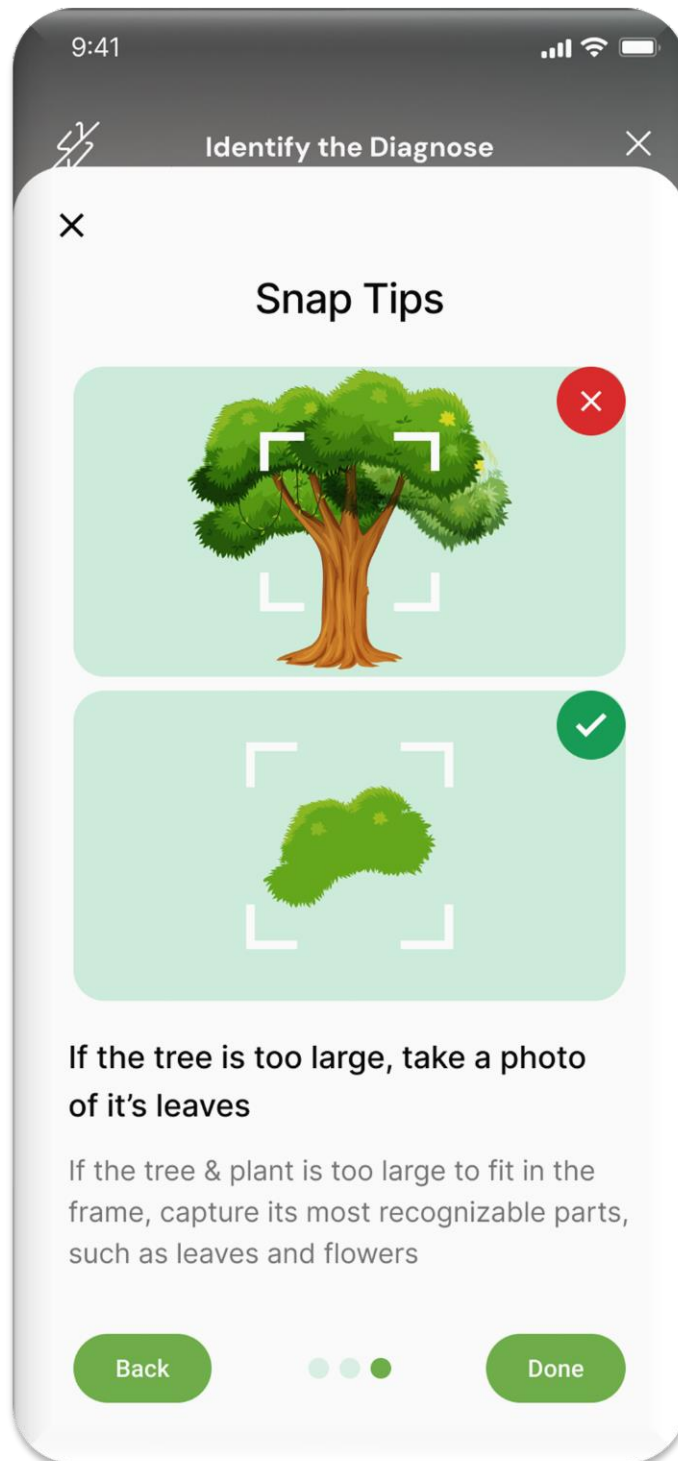
Scan-1



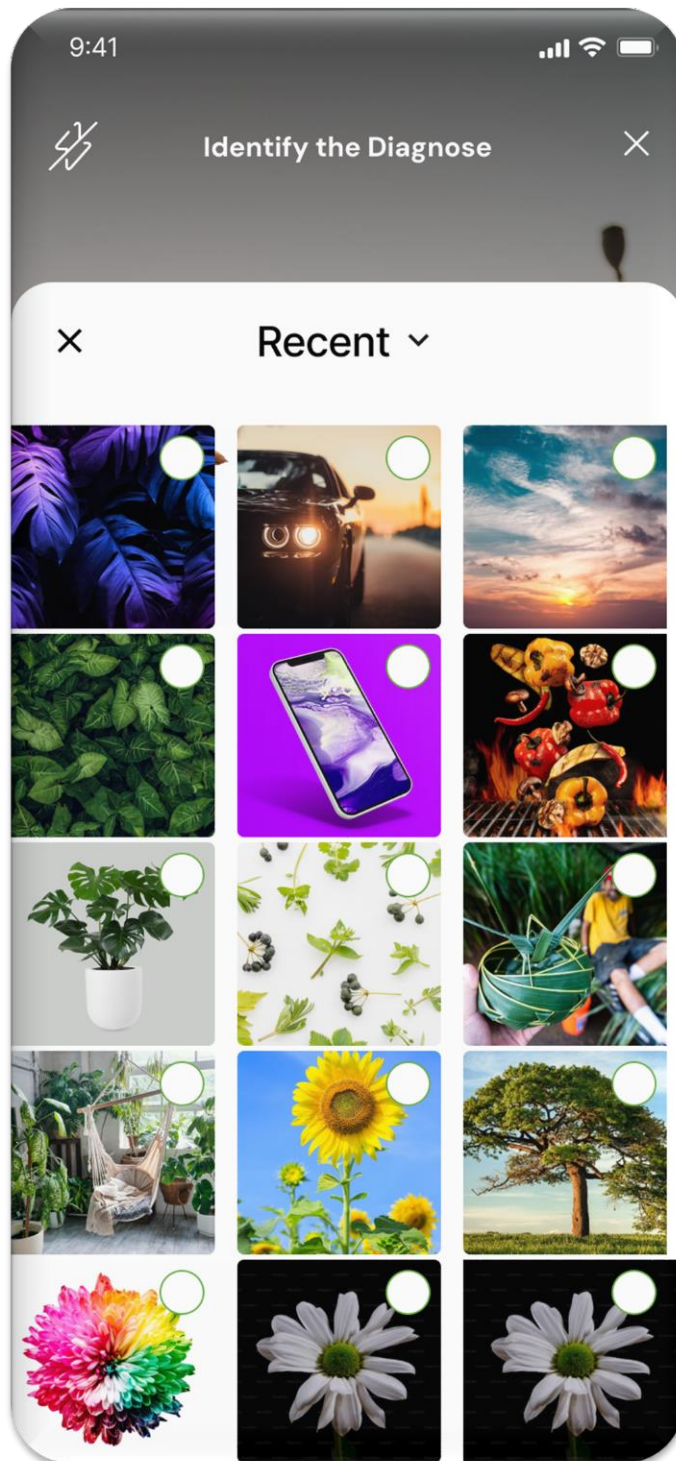
Scan-2



Scan-3



Scan-4



Diagnosis Form



Can You Help Us Identify This Plant's Condition?

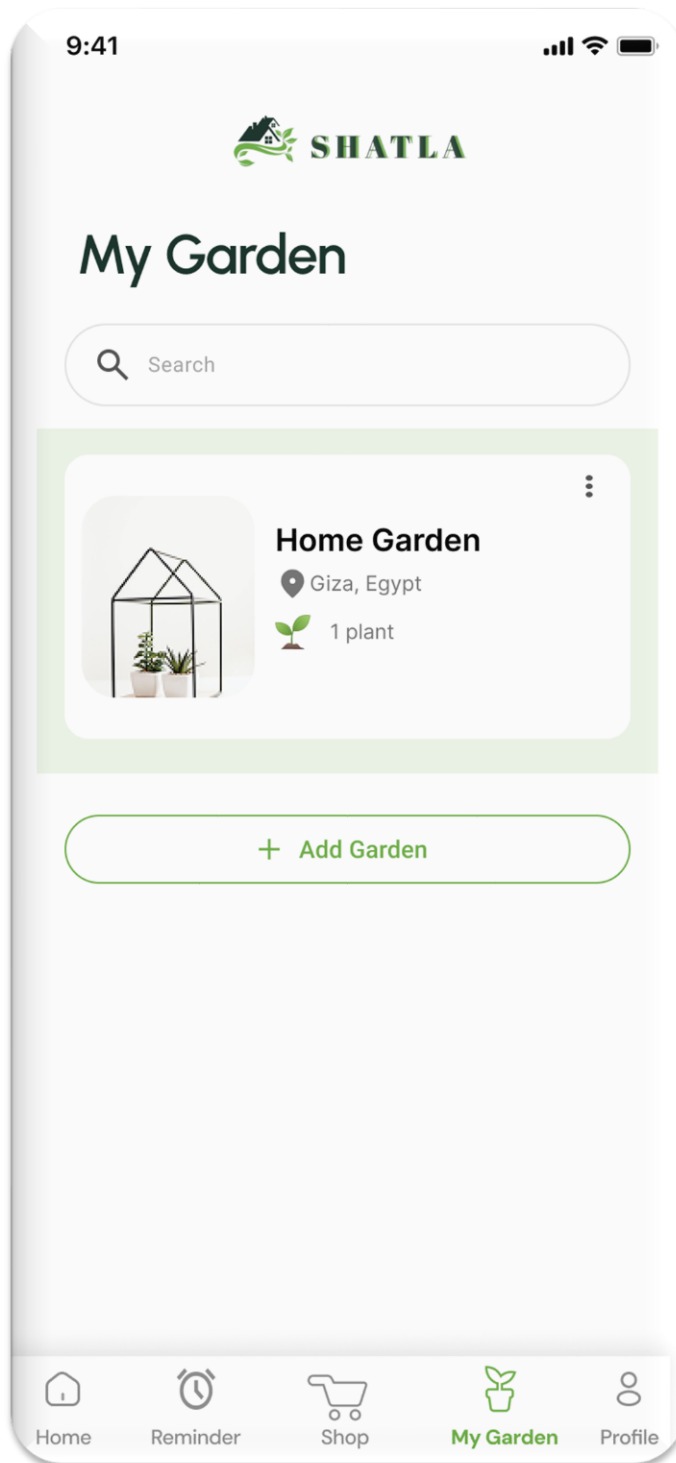
Our system wasn't sure.
Please select the closest match.

Which image looks like your plant?

image options side by side 

Submit Diagnosis

My Garden



Add Garden



Add New Garden



Garden Name

Home Garden

Location

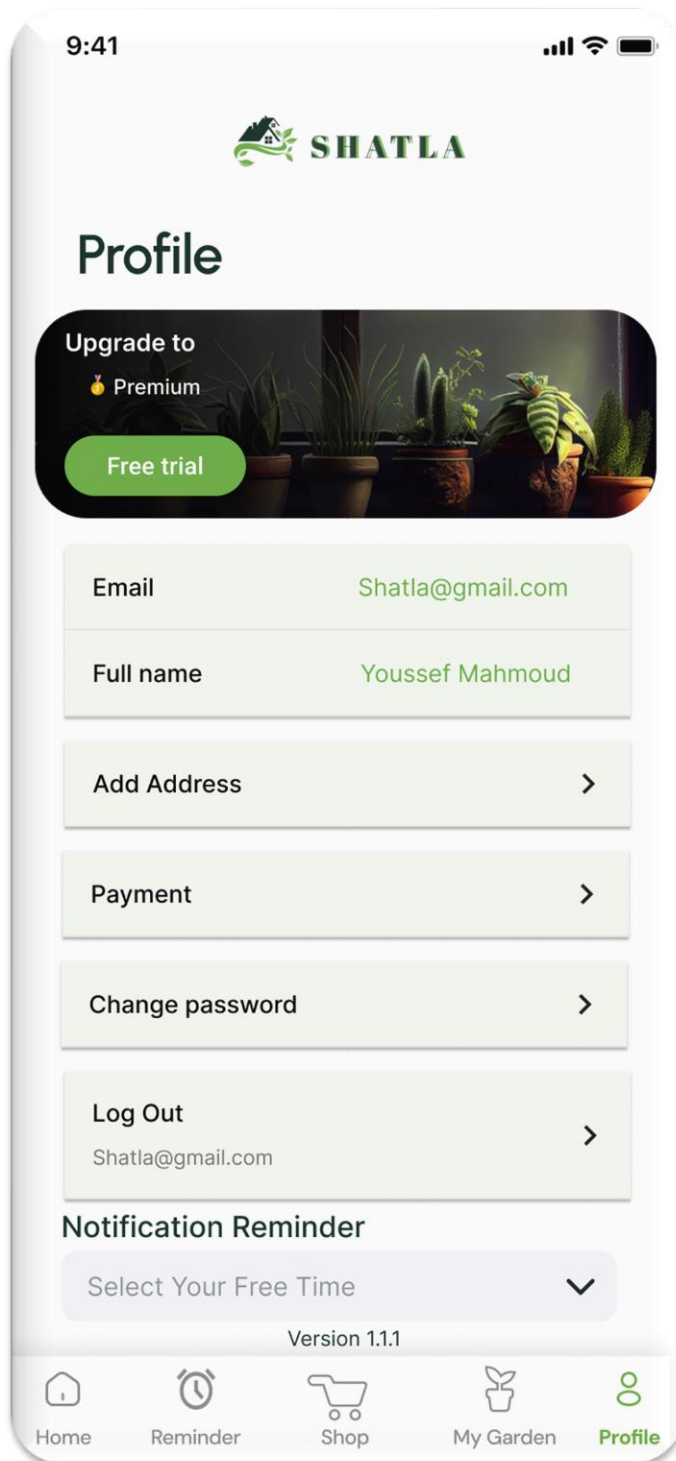
Giza, Dokki

130,1233 / 120.963

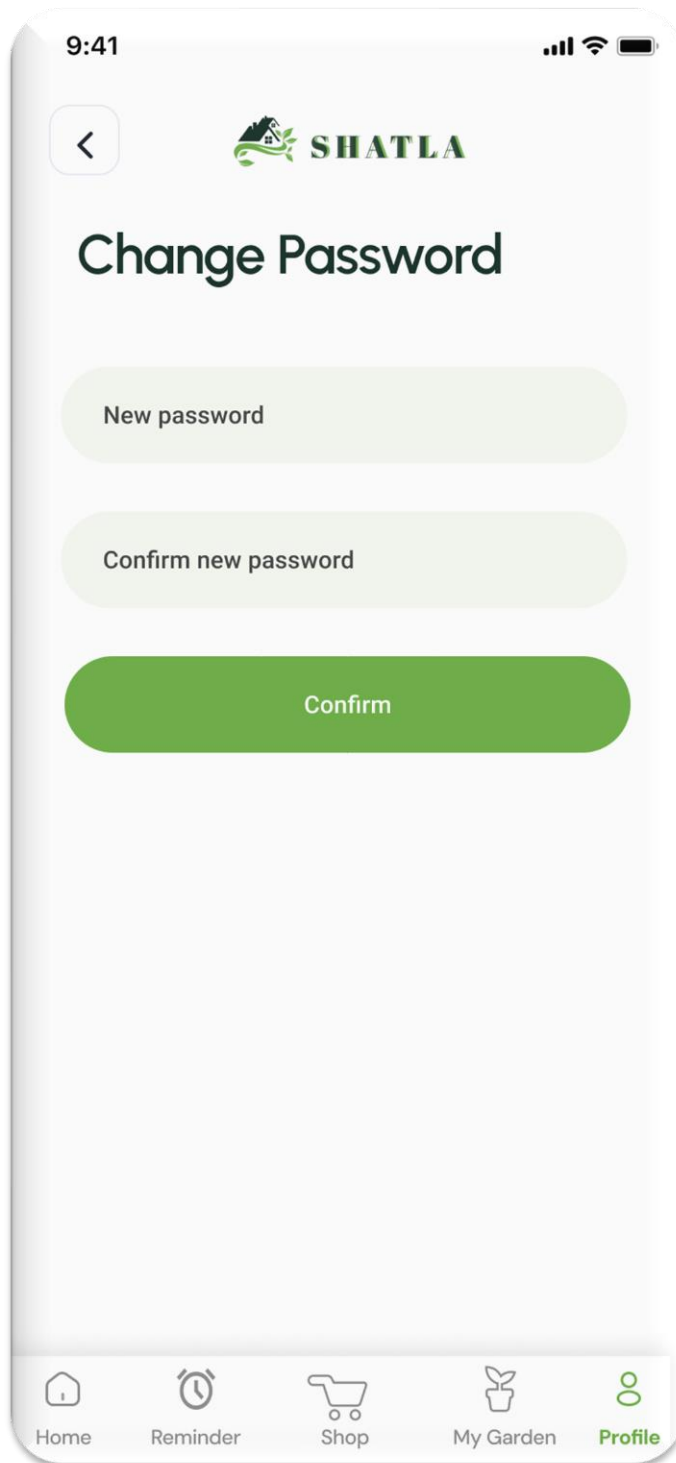
+ Add plant

Save Garden

Profile



Profile-Change password



The image shows a mobile application interface for changing a password. At the top, the status bar displays the time 9:41, signal strength, Wi-Fi, and battery icons. Below the status bar is a navigation bar with a back arrow on the left and the SHATLA logo in the center. The main heading is "Change Password". There are three input fields: "New password", "Confirm new password", and a green "Confirm" button. At the bottom is a tab bar with five icons: Home, Reminder, Shop, My Garden, and Profile (which is highlighted in green).

9:41

SHATLA

Change Password

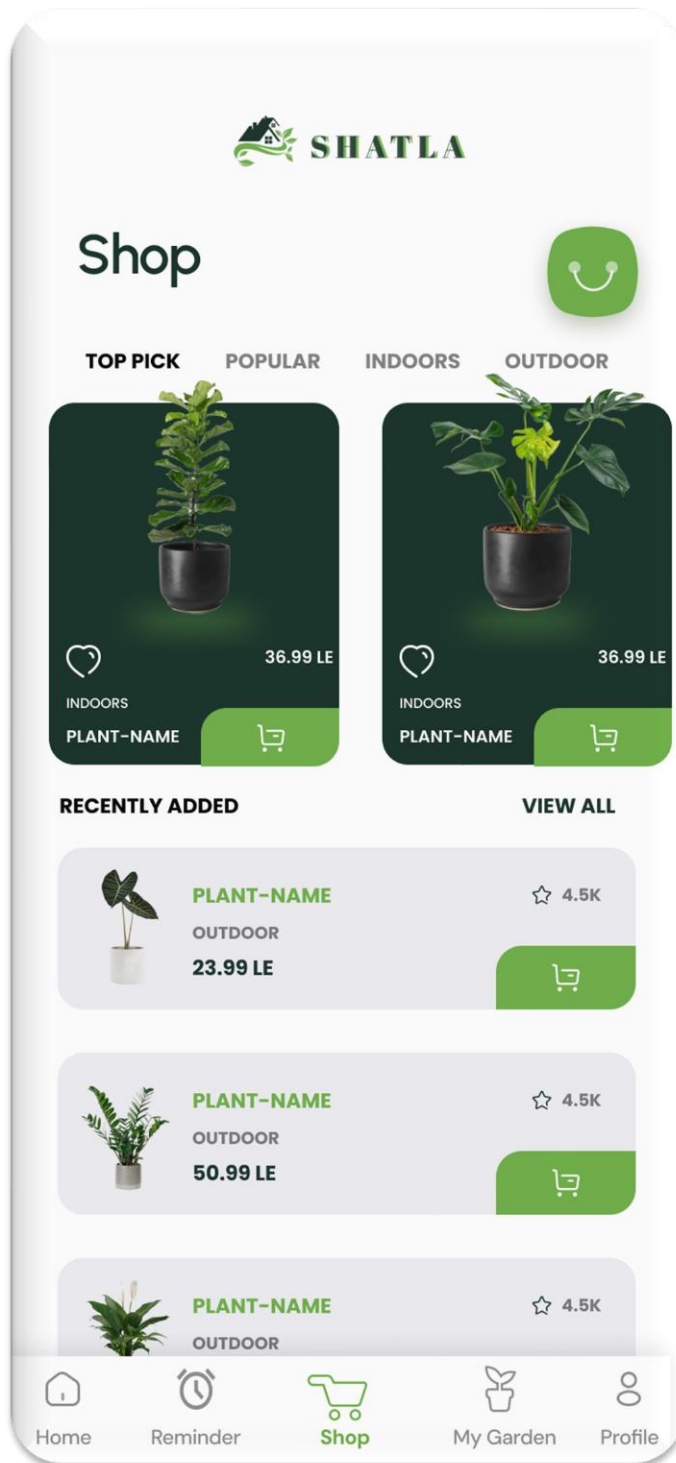
New password

Confirm new password

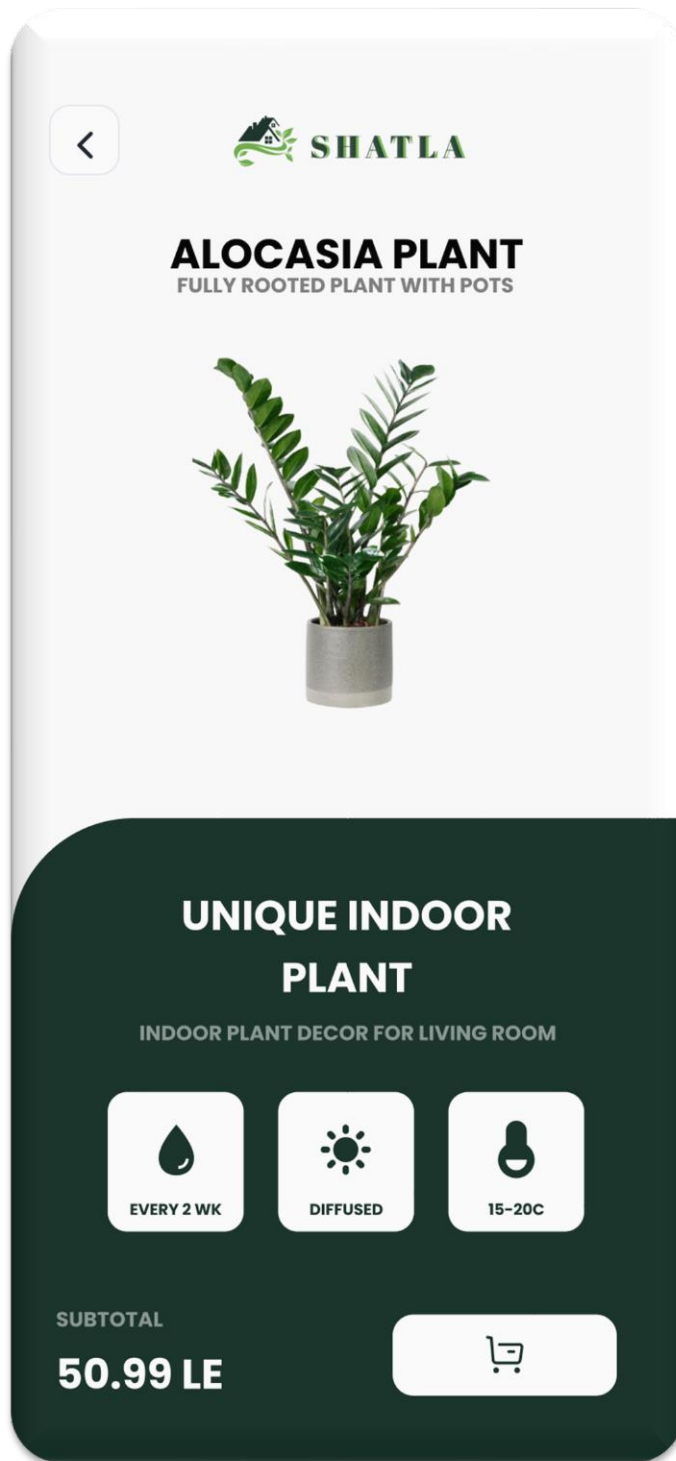
Confirm

Home Reminder Shop My Garden Profile

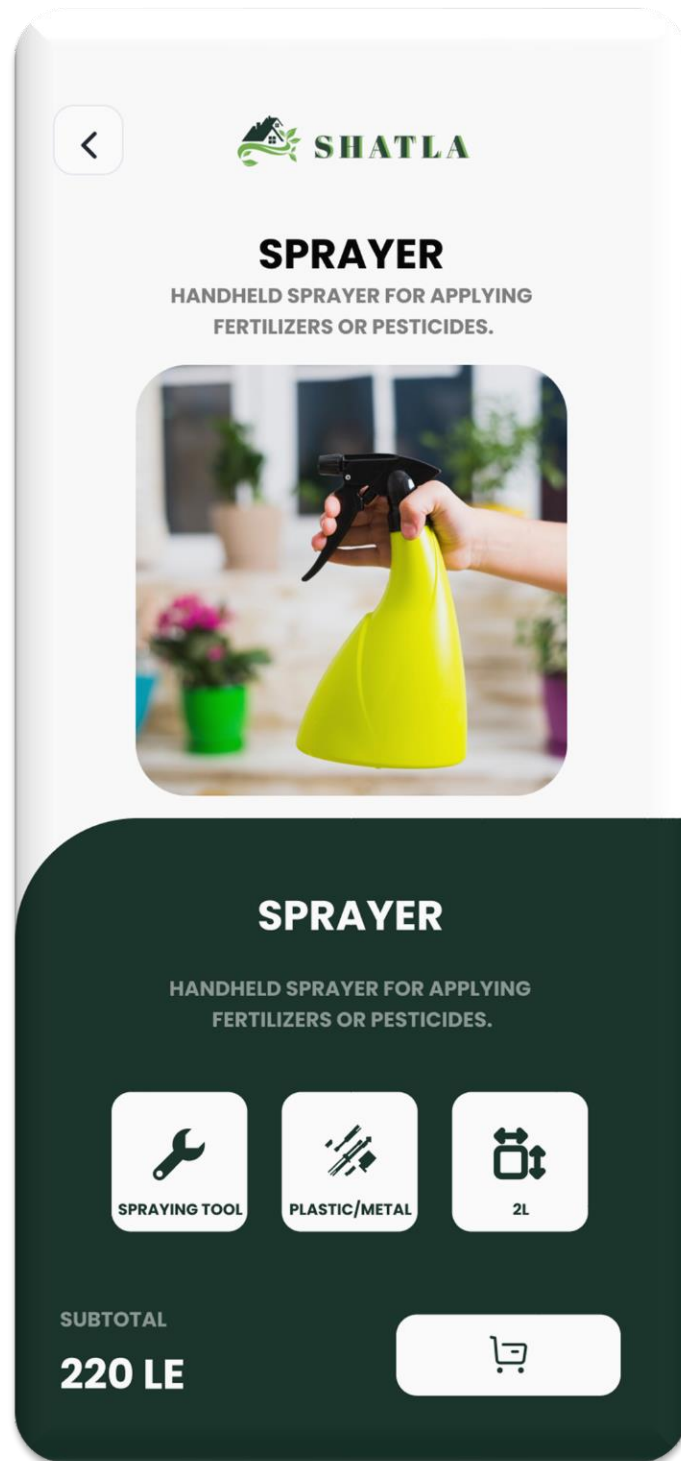
Shop



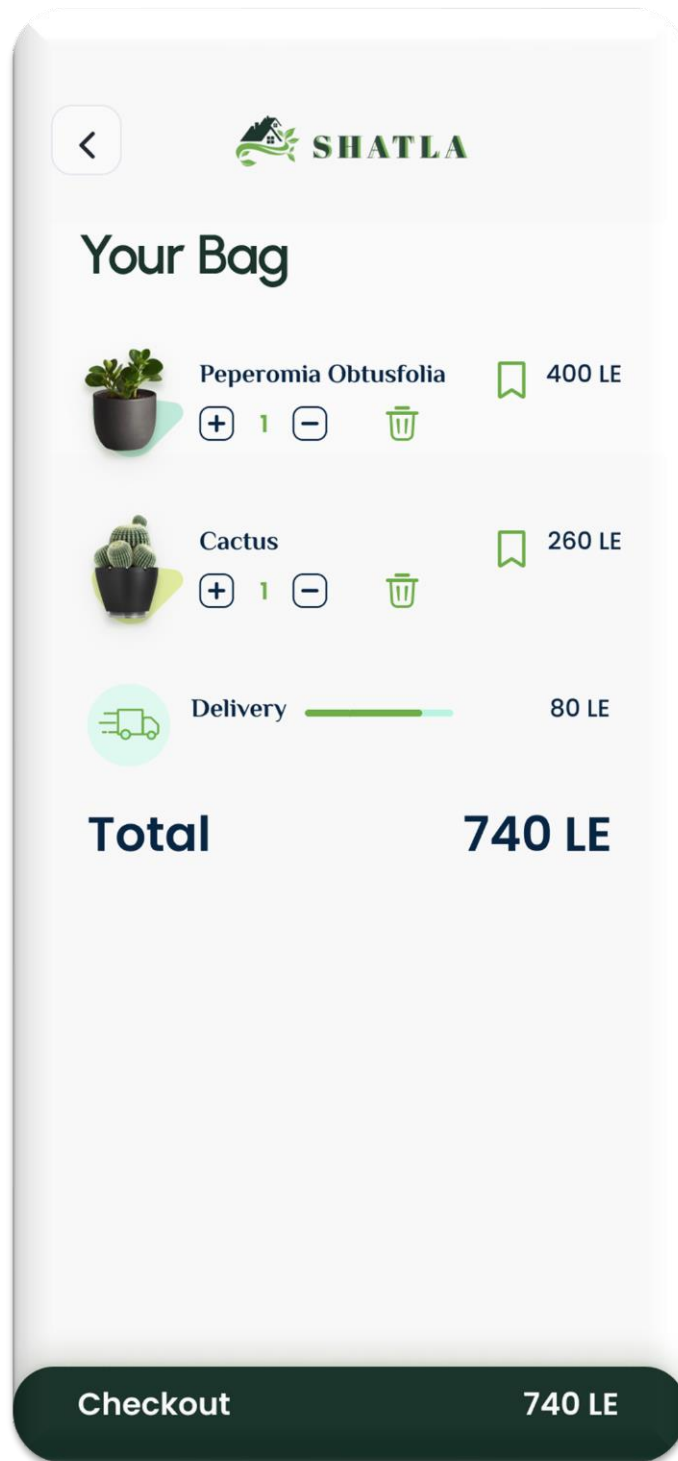
Shop-Details Plant



Shope-Details Tools



Shop-Checkout



Add Address



Address

Name

Youssef Ezatt

Country **City**

Giza Giza

Phone Number

+20 01002256894

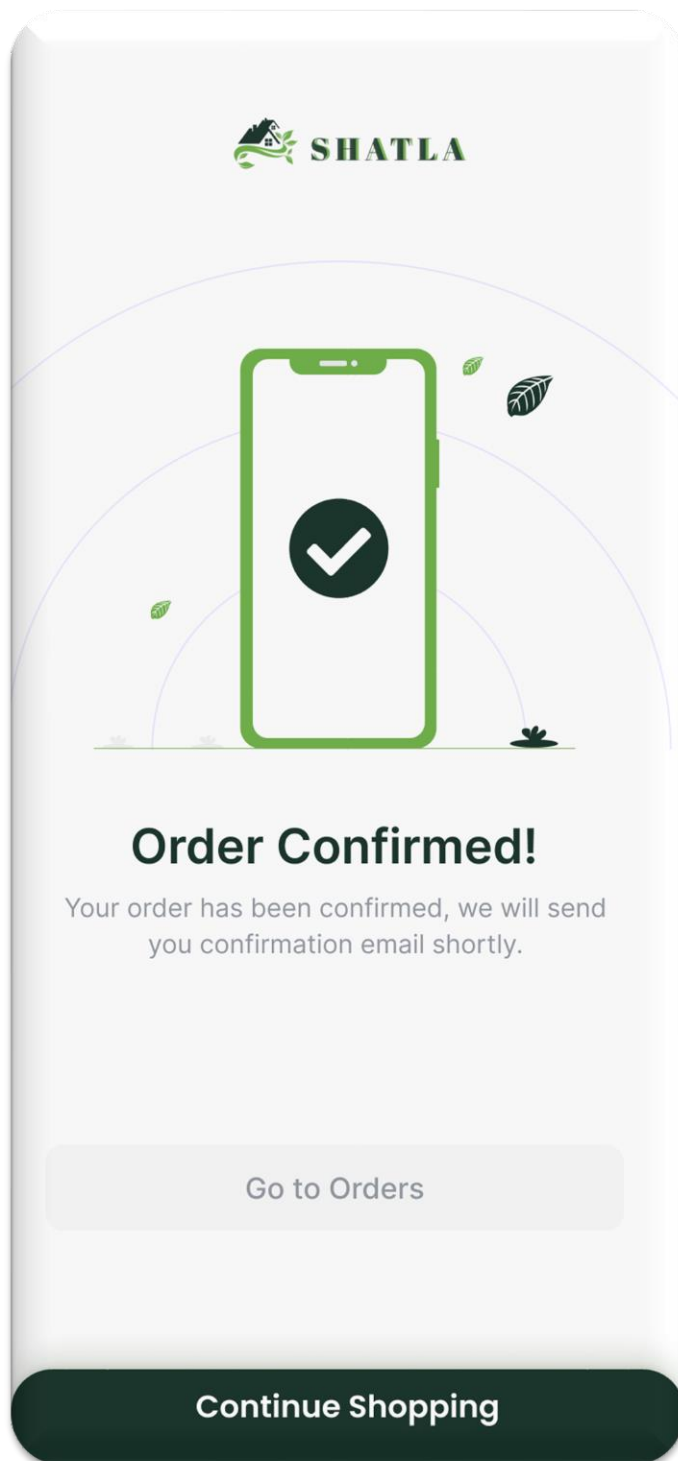
Address

Giza, Dokki

Save as primary address ☒

Save Address

Order Confirmed



Payment

Payment

Youssef Ezaat

Visa Classic

5 2 5 4 * * * * * 7 6 9 0

\$3,763.87

M

Vis

5

\$3

 Add new card

Card Owner

Youssef Ezzat

Card Number

5254 7634 8734 7690

EXP

24/24

CVV

7763

Save card info

☒

Save Card

Add Payment



Add New Card



Card Owner

Mrh Raju

Card Number

5254 7634 8734 7690

EXP

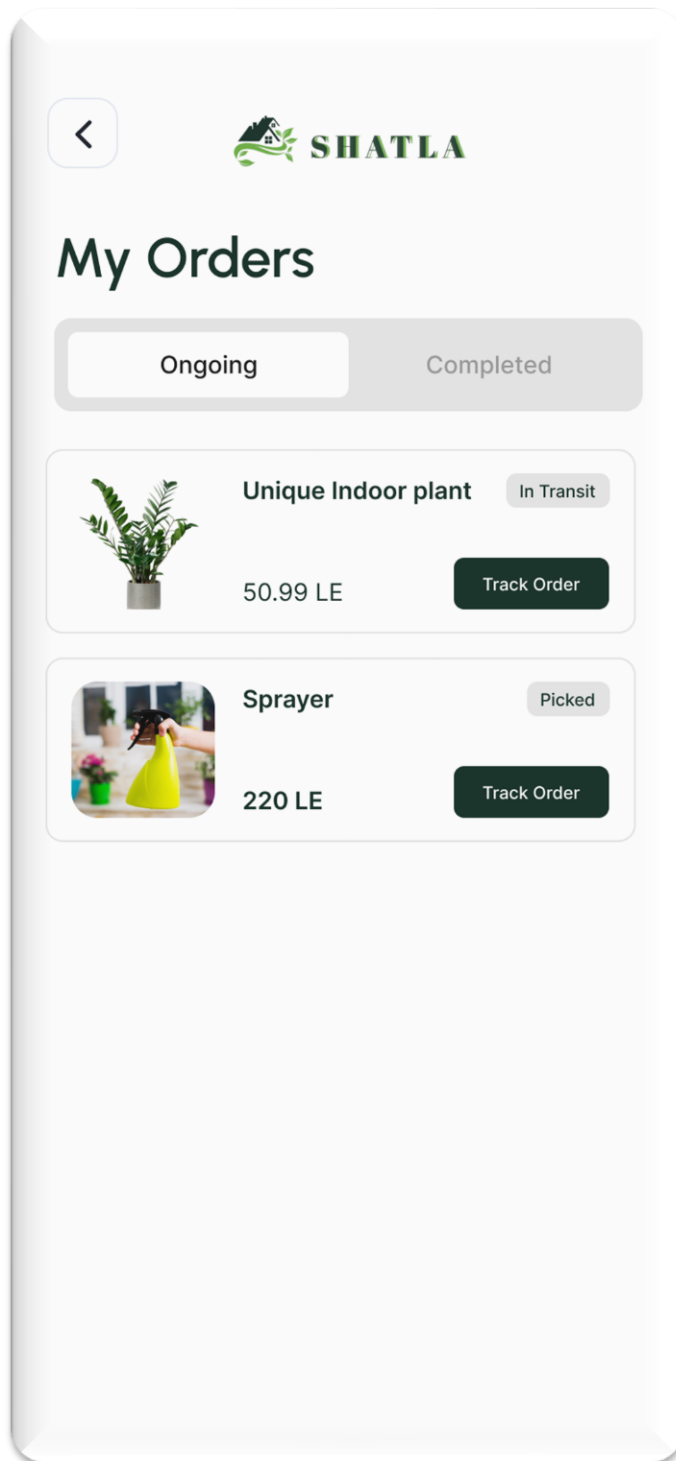
24/24

CVV

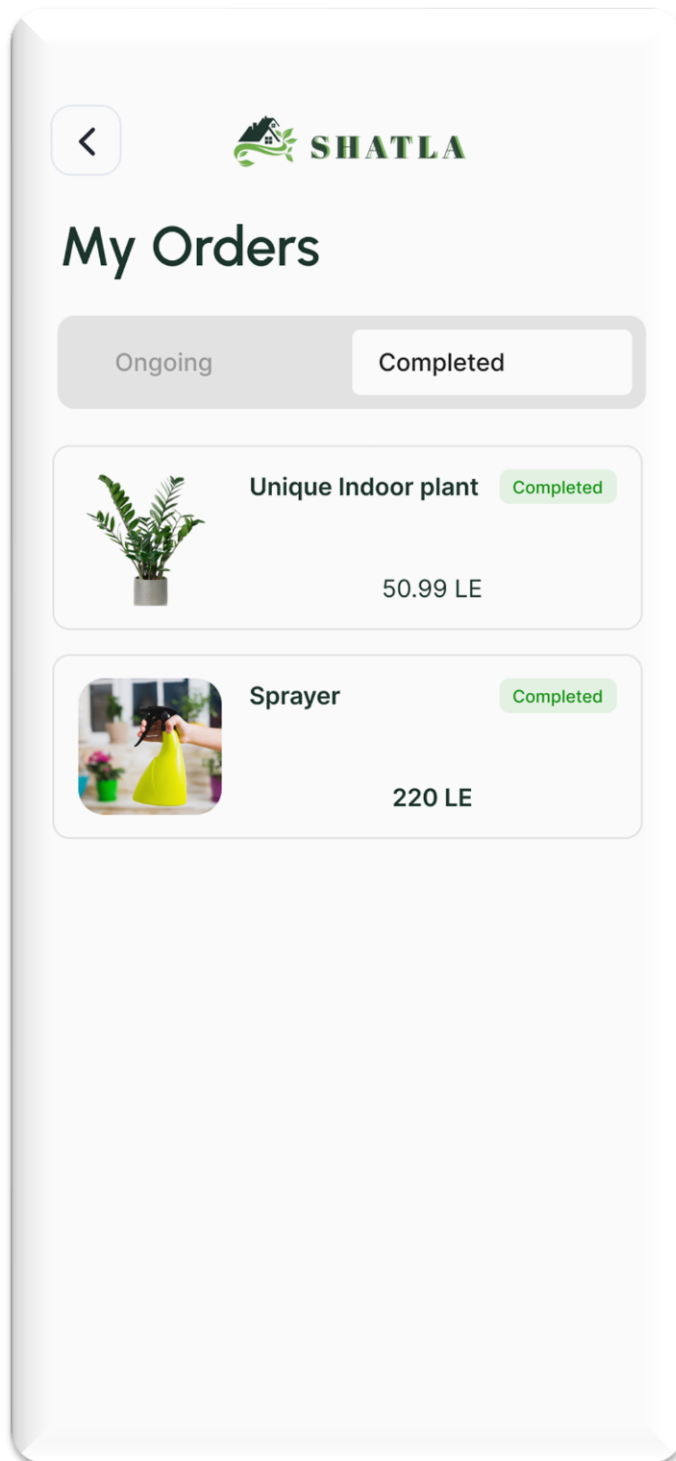
7763

Add Card

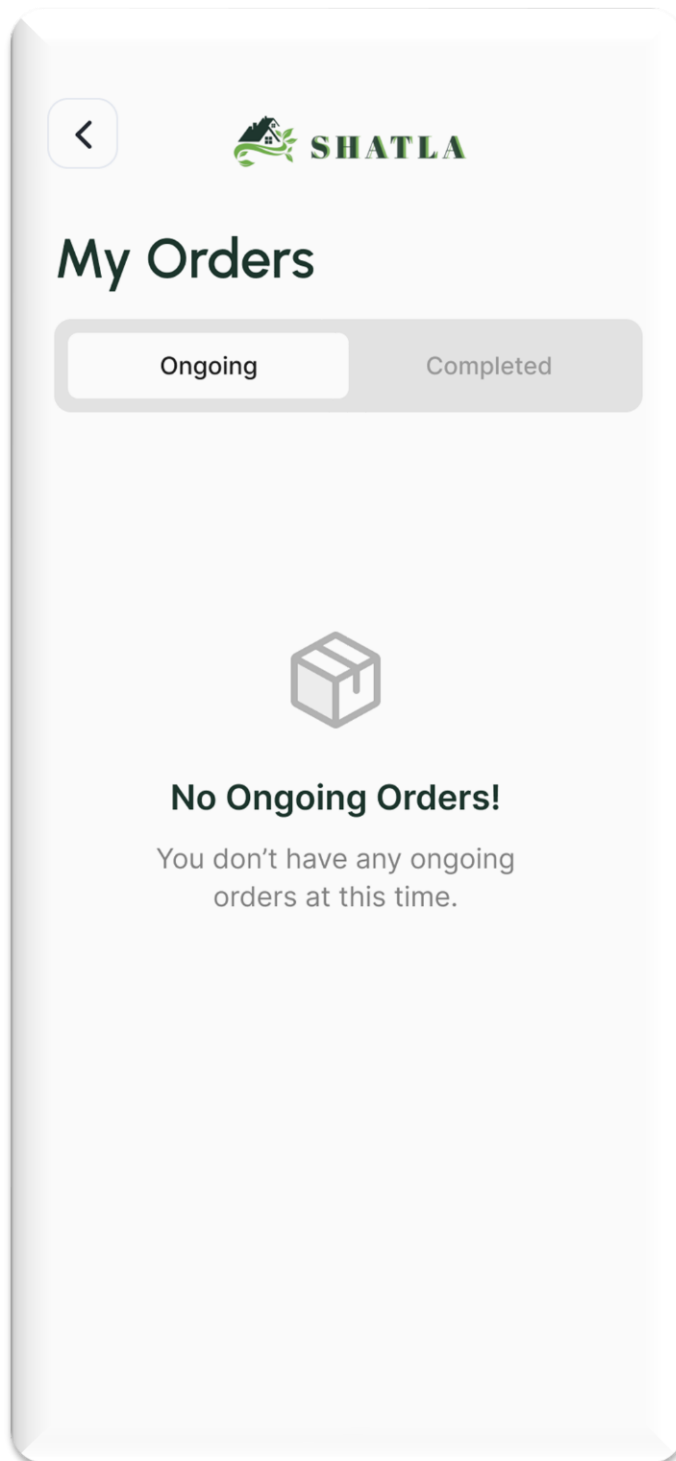
My Order Ongoing



My Order Completed

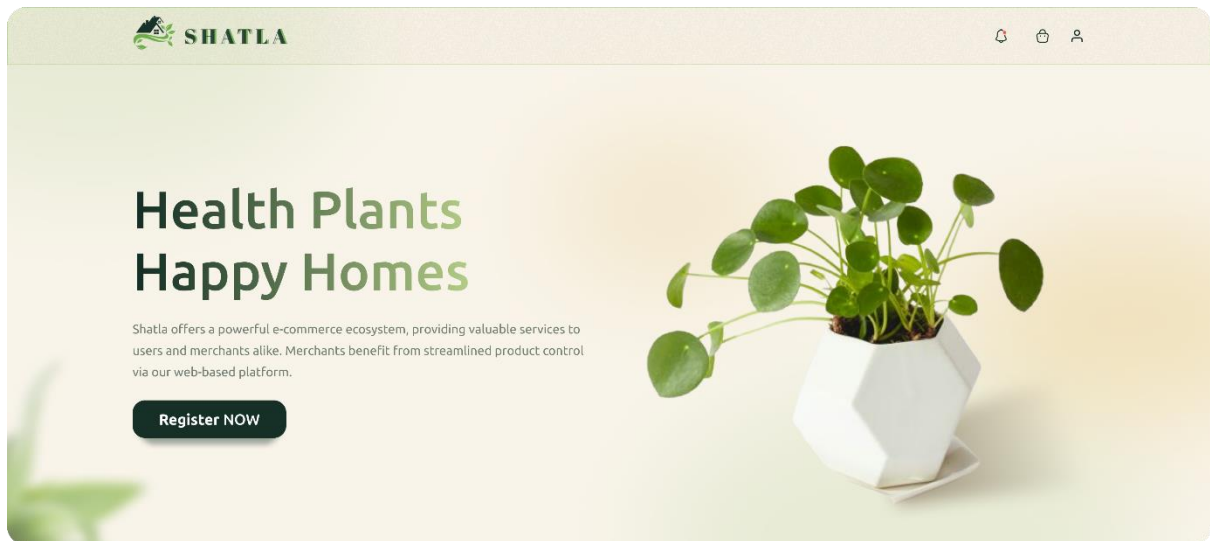


My Order Empty



4.2.2 Web Screen

Home



Login

The image shows the login screen of the SHATLA website. At the top, there is a header with the SHATLA logo. The main content area has a light green background. In the center, there is a white box containing the "Sign In" form. The form has the title "Sign In" in bold. Below the title, there are two links: "New?" and "Create an Account". The form includes two input fields: "Email" with the placeholder text "Enter Email Address" and "Password" with the placeholder text "Enter Password". Below the password field, there is a checkbox labeled "Keep me signed in". A dark green button with the text "Create Account" is positioned below the checkbox. Below the button, there is a link that says "Forgot your password?". A horizontal line separates the "Sign In" section from the "Or sign in using:" section. Below this line, there is a button with the Google logo and the text "Continue with Google".

Create Account



Create an Account

[Have an Account?](#) [Sign In](#)

Email

Password

Create Account

By creating account, you agree to our [Terms of Service](#)

Or create an account using:

 [Continue with Google](#)

Confirm Email



Confirm Email

Check Your Email and Enter Confirmation Code

Confirmation Code

Confirm Email

Haven't received your code?

[Resend Code](#)

Password Reset



Password Reset

We Will Help You Reset your Password

Email

Reset Password

Remembered your Password?

Back to Sign In

Send OTP





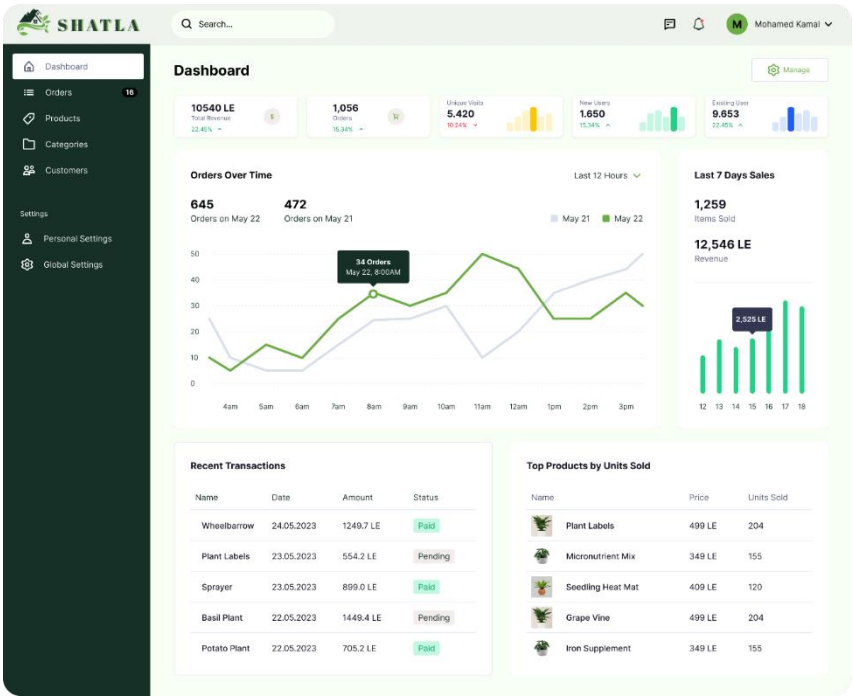
Almost There!

Check your email inbox and confirm your account

Didn't receive any mail?

Resend Confirmation

Dashboard



Orders

SHATLA

Search...

Messages

Notifications

Mohamed Kamal

Dashboard

Orders16

Products

Categories

Customers

Settings

Personal Settings

Global Settings

Orders

Export

+ Add Order

Filter

Search...

Filter

Search...

☐	Order	Date	Customer	Payment status	Order Status	Total
☒	#12512B	May 5, 4:20 PM	Tom Anderson	Paid	Ready	499 LE
☒	#12523C	May 5, 4:15 PM	Jayden Walker	Paid	Ready	343.6 LE
☒	#51232A	May 5, 4:15 PM	Inez Kim	Paid	Ready	551 LE
☒	#23534D	May 5, 4:12 PM	Francisco Henry	Paid	Shipped	297.4 LE
☐	#51323C	May 5, 4:12 PM	Violet Phillips	Paid	Shipped	230.6 LE
☐	#35622A	May 5, 4:12 PM	Rosetta Becker	Paid	Shipped	874.4 LE
☐	#34232D	May 5, 4:10 PM	Dean Love	Paid	Ready	445.5 LE
☐	#56212D	May 5, 4:08 PM	Nettie Tyler	Paid	Ready	367.9 LE
☐	#76543E	May 5, 4:08 PM	Lora Weaver	Paid	Shipped	287.8 LE
☐	#12512B	May 5, 4:05 PM	Vincent Cannon	Paid	Shipped	964.6 LE
☐	#12523C	May 5, 4:05 PM	Nettie Palmer	Paid	Received	255.3 LE
☐	#23534D	May 5, 4:04 PM	Miguel Harris	Pending	Ready	505.4 LE
☐	#12523C	May 5, 4:04 PM	Angel Conner	Pending	Ready	634.7 LE
☐	#51232A	May 5, 4:03 PM	Rosalie Singleton	Pending	Received	916.3 LE

←

1

2

3

4

5

6

...

24

→

274 Results

Products

SHATLA

Dashboard

Orders10

Products

Categories

Customers

Settings

Personal Settings

Global Settings

Search...

Messages

Notifications

Mohamed Kamal

Products

ExportAdd Product

Filter

Search...

Filter

Search

☐	Product	Inventory	Color	Price	Rating
☒	 Grape Vine Plant	96 in stock	Green	499 LE	5.0 (32 Votes)
☒	 Pepper Plant	56 in stock	Green	349 LE	4.8 (24 Votes)
☒	 Aloe Vera Plant	78 in stock	Green	409 LE	5.0 (54 Votes)
☒	 Rosemary Plant	32 in stock	Green	499 LE	4.5 (31 Votes)
☒	 Peach Tree	32 in stock	Green	349 LE	4.9 (22 Votes)
☐	 Grape Vine Plant	96 in stock	Green	499 LE	5.0 (32 Votes)
☐	 Pepper Plant	56 in stock	Green	349 LE	4.8 (24 Votes)
☐	 Aloe Vera Plant	Out of Stock	Green	409 LE	5.0 (54 Votes)
☐	 Rosemary Plant	Out of Stock	Green	499 LE	4.5 (31 Votes)
☐	 Peach Tree	Out of Stock	Green	349 LE	4.9 (22 Votes)

←

123456...24

→

146 Results

Add Product



Search...



M

Mohamed Kamal

Dashboard

Orders16

Products

Categories

Customers

Settings

Personal Settings

Global Settings

← Back

Add Product

CancelSave

Information

Product Name

Grape Vine

Product Description

Product description

Images

Add File

Or drag and drop files

Price

Product Price

Enter price

Discount Price

Price at Discount

Add tax for this product

Different Options

This product has multiple options

Option 1

Size

Size

Value

S X M X L X XL X

Add More

Shipping

Weight

Enter Weight

Country

Select Country

This is digital item

Categories

Plants

Tools

Manure

Tags

Add Tags

Enter tag name

Tools X Plant X Summer Collection X

SEO Settings

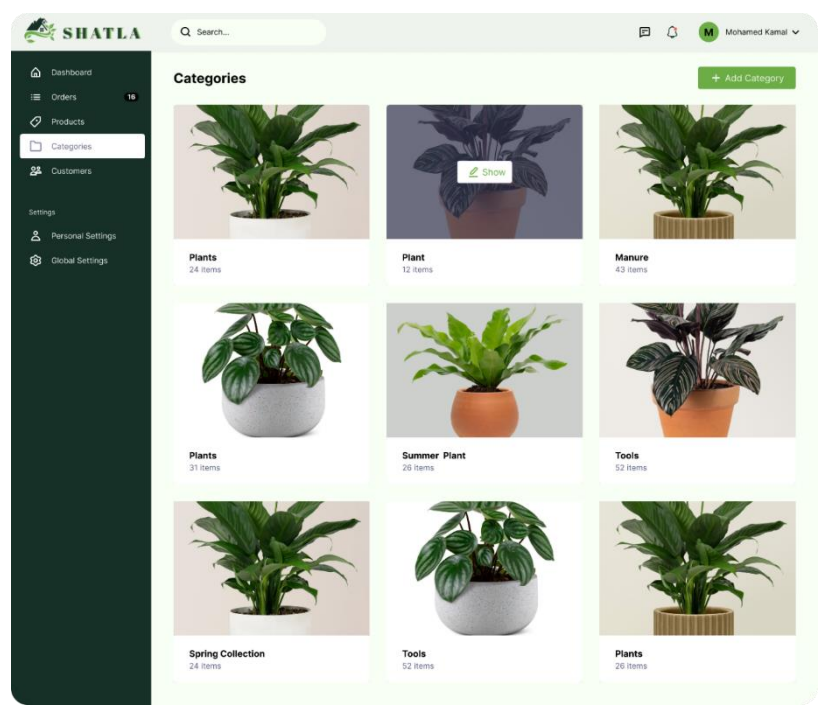
Title

Description

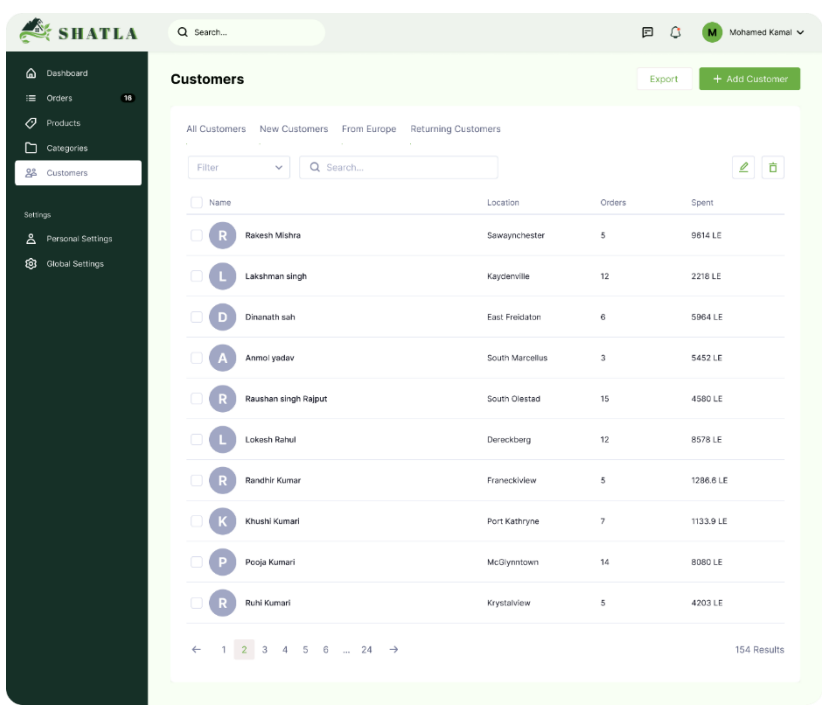
CancelSave

164

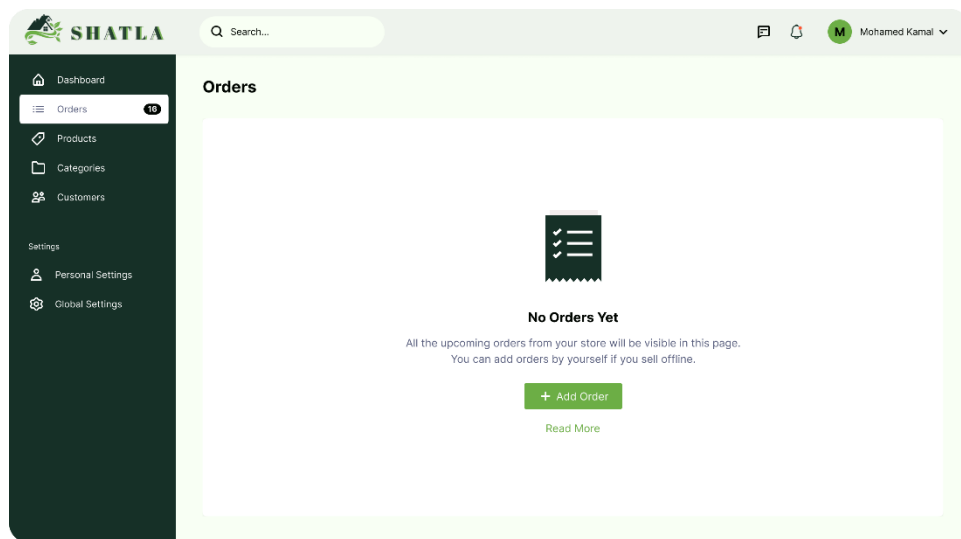
Categories



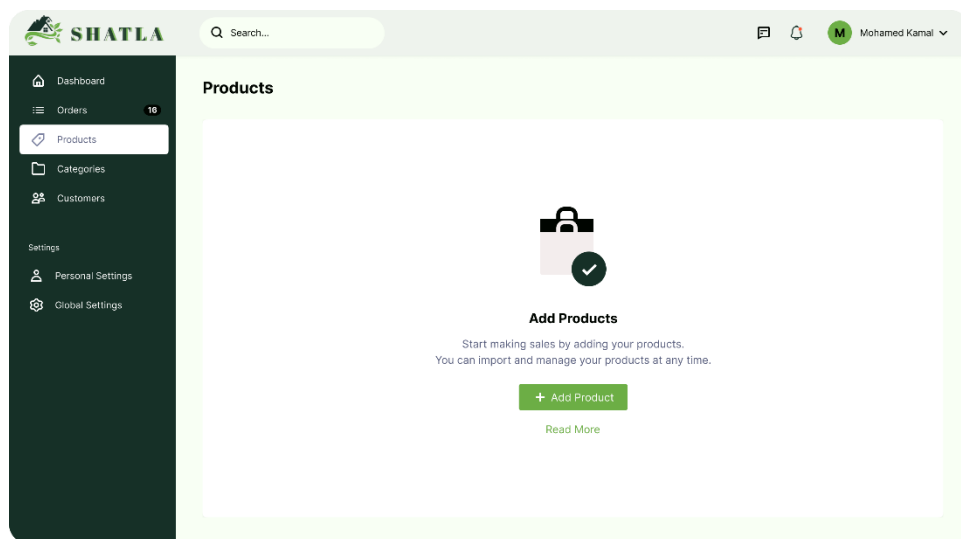
Customers



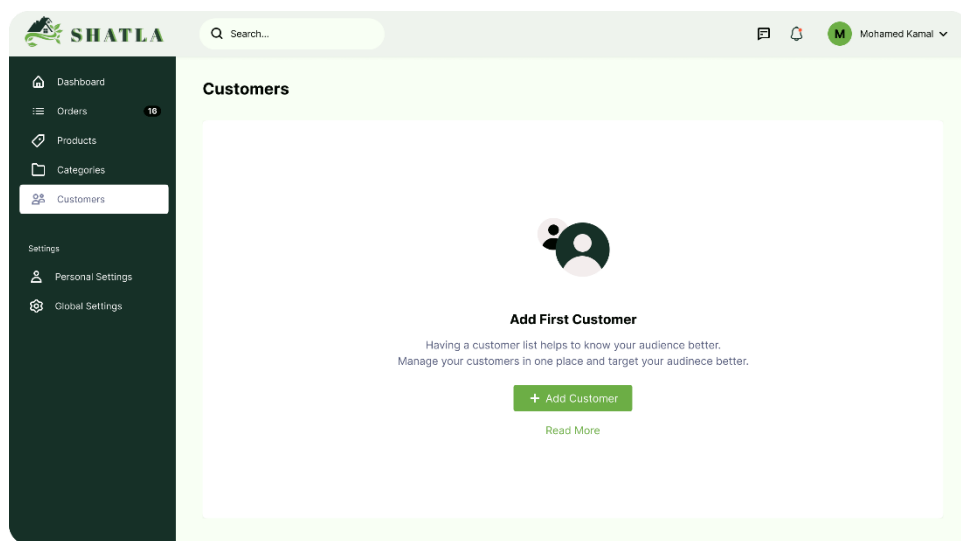
Order Empty



Product Empty

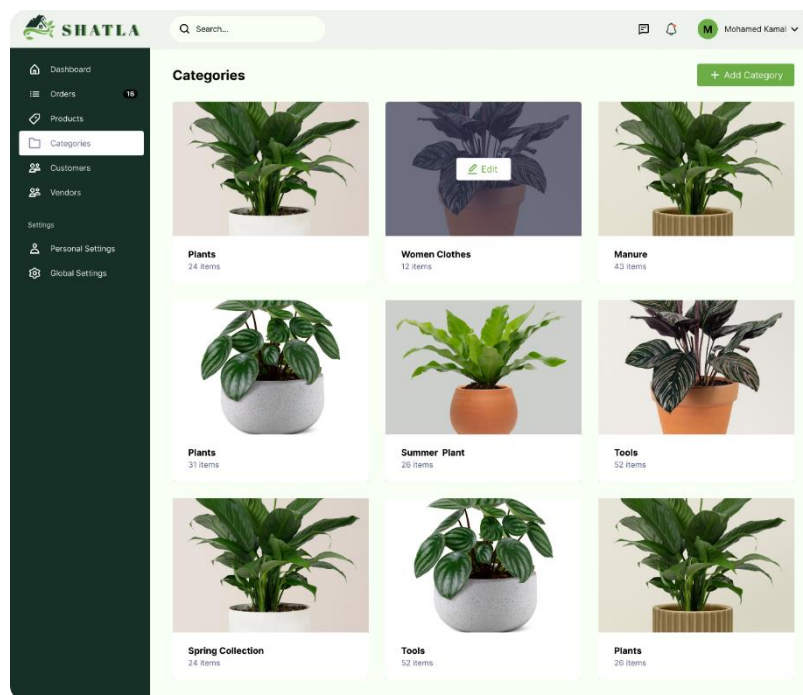


Customer Empty

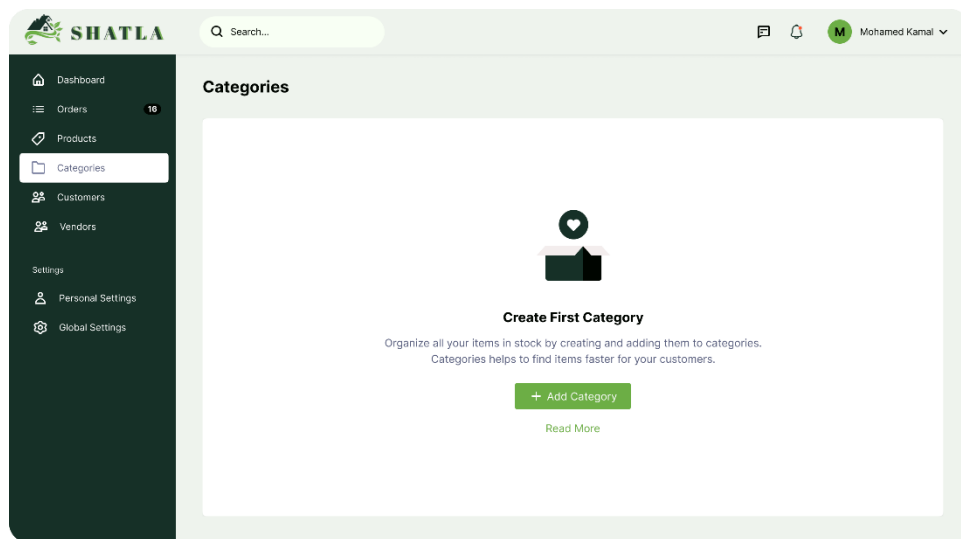


4.2.3 Admin Screen

Editing Category



Empty Category



4.3 Tools and Programming Languages with Selection Reasons

This section outlines the tools and programming languages used for developing the project, alongside the rationale behind their selection. Each tool was chosen carefully based on its strengths, compatibility, and ability to support the project's objectives.

Backend Development (Nest.js)

Programming Language:

- **TypeScript** was chosen for its strong typing, maintainability, and enhanced developer productivity. Its static typing helps in catching errors early during the development phase and ensures clean, maintainable code.

Framework:

- **Nest.js** was chosen for the following reasons:
 1. **Modular and Scalable Architecture:**
Nest.js follows a modular architecture that ensures the application is scalable, maintainable, and easy to extend with new features over time.
 2. **Built-in Dependency Injection:**
Its dependency injection system simplifies managing service lifecycles and promotes code reusability.
 3. **TypeScript Integration:**
As a framework built on TypeScript, Nest.js leverages the full potential of TypeScript, enhancing type safety and productivity.
 4. **Compatibility with REST and GraphQL APIs:**
Nest.js provides out-of-the-box support for building robust REST APIs and GraphQL APIs, making it suitable for seamless communication with other system components.

5. Active Community and Ecosystem:

The extensive community support and rich ecosystem of plugins and modules streamline development and troubleshooting processes.

IDE:

- **Visual Studio Code** was used as the primary Integrated Development Environment (IDE) for backend development. It offers extensive support for TypeScript, debugging tools, and Nest.js plugins that accelerate the coding process.

Admin Dashboard Development (Next.js)

Programming Language:

- **TypeScript** was chosen to ensure type safety, improve code readability, and catch potential bugs at compile time. Its static typing provides a solid foundation for building scalable and maintainable frontends, especially important in complex admin interfaces.

Framework:

- **Next.js** was selected for its powerful full-stack capabilities and the following key benefits:
 1. **Server-Side Rendering (SSR) and Static Site Generation (SSG):**

Next.js enables optimal performance through automatic static optimization and support for SSR. This is especially beneficial for admin dashboards where data must be fresh and responsive.
 2. **Built-in Routing System:**

Next.js provides a file-based routing system out of the box, which simplifies the development of structured and organized admin pages.
 3. **API Routes for Backend Integration:**

The ability to create API endpoints directly within the Next.js project helps

manage lightweight server-side operations, authentication, and data fetching efficiently.

4. **Optimized Performance and SEO:**

The framework includes automatic code-splitting, image optimization, and prefetching mechanisms, improving user experience and load times — even for complex dashboards.

5. **Developer Experience:**

Next.js offers fast refresh, clear error reporting, and full TypeScript support, making development more productive and enjoyable.

6. **Active Ecosystem and Community:**

A large community, regular updates, and rich plugin support (e.g., Tailwind CSS, React Query, Zustand) ensure smooth development and quick troubleshooting.

IDE:

- **Visual Studio Code** was used as the primary IDE for admin dashboard development. It provides excellent TypeScript support, integrated terminal, Git integration, and a vast range of extensions (e.g., ESLint, Prettier, Tailwind IntelliSense) tailored to Next.js development.

Frontend Development (Flutter)

Programming Language:

- **Dart** was chosen as the programming language for frontend development due to its:
 1. **Fast Development:** Hot-reload capabilities in Dart speed up the development process, enabling developers to see changes in real-time.
 2. **Native Performance:** Dart compiles native machine code, ensuring smooth performance across platforms.

Framework:

- **Flutter** was chosen for the following reasons:
 1. **Cross-Platform Development:** Flutter enables a single codebase to build apps for Android, iOS, and the web, reducing development time and effort.
 2. **Rich UI Components:** The framework includes pre-built widgets for creating visually appealing and highly customizable user interfaces.
 3. **High Performance:** With a rendering engine built in C++, Flutter ensures high-performance applications that feel native.
 4. **Active Community Support:** Flutter's growing popularity and active community provide abundant resources, tutorials, and plugins to streamline development.

IDE:

- **Android Studio** and **Visual Studio Code** were used for Flutter development due to their robust debugging tools, Flutter-specific plugins, and support for cross-platform app development.

AI Development (FastAPI with Python)

Programming Language:

- **Python** was chosen for implementing AI functionalities because of its vast ecosystem of libraries and tools for machine learning and data processing.

Framework:

- **FastAPI** was chosen for the following reasons:
 1. **High Performance:** FastAPI is one of the fastest web frameworks available, making it suitable for serving AI predictions in real time.

2. **Ease of Integration:** FastAPI integrates seamlessly with Python libraries like TensorFlow, PyTorch, and Scikit-learn for machine learning tasks.
3. **Automatic Documentation:** FastAPI automatically generates interactive API documentation using OpenAPI standards, simplifying testing and collaboration.
4. **Asynchronous Support:** FastAPI supports asynchronous programming, making it efficient in handling concurrent API calls.

Libraries Used:

- **NumPy and Pandas:** For data preprocessing and manipulation.
- **TensorFlow/PyTorch:** For building and training machine learning models.
- **OpenCV:** For image pre-processing tasks in plant diagnosis.

IDE:

- **Google Colab** was used for training models due to its access to high-performance GPUs and collaborative features.
- **PyCharm** was used for developing the FastAPI server and integrating AI models.

Database System (PostgreSQL)

Reasons for Selection:

1. **Reliability and Scalability:** PostgreSQL is an exceptionally reliable relational database system, capable of handling large datasets efficiently.
2. **Advanced Features:** PostgreSQL supports advanced features such as JSONB for storing unstructured data and indexing for optimized queries.
3. **Cross-Platform Compatibility:** PostgreSQL is a cross-platform and integrates seamlessly with Nest.js and FastAPI.

4. **Open Source:** Being open-source, PostgreSQL ensures cost efficiency and access to a wide array of community plugins.

Database System (Redis)

Reasons for Selection:

1. **High Performance and Low Latency:**

Redis is an in-memory data store known for its lightning-fast read and write operations, making it ideal for coaching, real-time analytics, and session management.

2. **Versatile Data Structures:**

Redis supports a rich set of data types such as strings, hashes, lists, sets, and sorted sets, enabling flexible storage solutions tailored to specific use cases like queues, leaderboards, and pub/sub systems.

3. **Seamless Integration with Nest.js and Next.js:**

Redis integrates smoothly with both backend (Nest.js) and frontend (Next.js) layers, particularly for managing authentication tokens, rate limiting, and temporary data storage.

4. **Scalability and Pub/Sub Capabilities:**

Redis supports horizontal scaling via clustering and offers built-in publish/subscribe functionality, which is essential for building real-time applications and microservice communication.

5. **Open Source and Widely Supported:**

As an open-source tool with a large, active community, Redis benefits from extensive documentation, frequent updates, and a broad ecosystem of client libraries and extensions.

Version Control (Git)

Reasons for Selection:

1. **Collaboration:** Git allows multiple team members to work on the project simultaneously without conflicts.
2. **Branching and Merging:** Git's branching system ensures that experimental features can be developed independently and merged when ready.
3. **Code Integrity:** Git maintains a history of code changes, enabling rollback to previous versions when needed.
4. **Repository Hosting:** GitHub was used for hosting the repository, facilitating seamless code sharing and collaboration.

Project Management (Jira)

Reasons for Selection:

1. **Task Tracking:** Jira enables effective tracking of project tasks, deadlines, and progress.
2. **Agile Workflow Support:** The project followed Agile methodology, and Jira provided tools for creating sprints, epics, and tracking user stories.
3. **Collaboration:** Jira fosters collaboration among team members with its comment and notification features.

Diagramming (draw.io and lucid)

Reasons for Selection:

1. **Ease of Use:** draw.io provides an intuitive interface for creating flowcharts, UML diagrams, and system architecture designs.
2. **Collaboration:** Being web-based, draw.io allows multiple team members to collaborate on diagrams in real time.

3. **Export Options:** Diagrams can be exported in various formats like PNG, PDF, or SVG for inclusion in documentation.

Design and Prototyping (Figma)

Reasons for Selection:

1. **Real-Time Collaboration:** Figma allows team members to work on the same design project simultaneously, ensuring efficient teamwork.
2. **Interactive Prototyping:** Figma enables the creation of interactive prototypes to visualize user workflows and gather feedback.
3. **Responsive Design:** The tool supports responsive design, ensuring UI compatibility across various devices and screen sizes.

Chapter 5 System Testing and Evaluation

5.1 Ecommerce Package

#	Functionality	Input	Expected Output
1	Add/Update item to cart (Working)	Authorization(Log ged in) { "productId": 15, "quantity": 2 }	{ "id": 84, "userID": "bedb86bd-0939-4e2f-9504-b65e91511d76", "status": "active", "totalPrice": 100, "discount": 0, "items": [{ "id": 136, "cartId": 84, "productId": 15, "quantity": 2, "subtotal": 100 }], "createdAt": "2025-06-28T16:49:15.988Z", "updatedAt": "2025-06-28T16:49:15.988Z" }
	Add/Update item to cart (Exceeds stock)	Authorization(Log ged in) { "productId": 15, "quantity": 9999 }	{ "message": "Requested quantity exceeds available stock", "error": "Bad Request", "statusCode": 400 }

			}
	Add/Update multiple items (Product not found)	Authorization(Log ged in) { "productId": 123, "quantity": 3 }	{ "message": "Product not found", "error": "Not Found", "statusCode": 404 }
2	Get cart for logged-in user (Working)	Valid Authorization header with token of existing user	{ "id": 84, "userID": "bedb86bd-0939-4e2f-9504-b65e91511d76", "status": "active", "totalPrice": 100, "discount": 0, "items": [{ "id": 136, "cartId": 84, "productId": 15, "quantity": 2, "subtotal": 100 }], "createdAt": "2025-06-28T16:49:15.988Z", "updatedAt": "2025-06-28T16:49:15.988Z" }
	Get cart for logged-in user (Unauthorized)	Missing or invalid Authorization header	{ "message": "Authorization header missing", // "Session Expired !",

			<pre>"error": "Unauthorized", "statusCode": 401 }</pre>
3	Remove item from cart (Working)	Authorization(Logged in) <pre>{ "itemId": 15 }</pre>	<pre>{ "id": 85, "userID": "f7f0ed8b-0942-48b1-894e-c919c1a4fe4d", "status": "active", "totalPrice": 0, "discount": 0, "items": [], "createdAt": "2025-06-28T16:56:36.415Z", "updatedAt": "2025-06-28T16:56:36.415Z" }</pre>
	Remove item from cart (Unauthorized)	Missing or invalid Authorization header	<pre>{ "message": "Authorization header missing",/"Session Expired !", "error": "Unauthorized", "statusCode": 401 }</pre>
4	Checkout cart (Working)	Authorization(Logged in) (userID)	<pre>{ "id": 85, "userID": "f7f0ed8b-0942-48b1-894e-c919c1a4fe4d", "status": "checked_out", "totalPrice": 150, "discount": 0, }</pre>

			<pre> "items": [{ "id": 137, "cartId": 85, "productId": 15, "quantity": 3, "subtotal": 150 }], "createdAt": "2025-06-28T16:56:36.415Z", "updatedAt": "2025-06-28T16:56:36.415Z" } </pre>
	Checkout cart (Empty Cart)	Authorization(Logged in) (userID)	<pre> { "message": "Cart is empty", "error": "Bad Request", "statusCode": 400 } </pre>
6	Pay for an order (Working)	Valid id of existing unpaid order	<pre> { "id": 46, "orderId": 85, "paymentMethod": "credit_card", "status": "completed", "amount": 150, "createdAt": "2025-06-28T17:49:22.678Z" } </pre>
	Pay for an order (Order not found or already paid)	Non-existing or already paid id	<pre> { "message": "Order cannot be paid at this stage.", "error": "Bad Request", "statusCode": 400 } </pre>

7	Get all payments (Working)	No input (for admin dashboard checks for roleID)	[{ "id": 1, "orderId": 1, "paymentMethod": "credit_card", "status": "completed", "amount": 24.99, "createdAt": "2025-04-20T21:26:41.040Z" }, { "id": 2, "orderId": 2, "paymentMethod": "credit_card", "status": "completed", "amount": 124.97, "createdAt": "2025-04-20T22:50:42.065Z" },]
8	Get single payment by ID (Working)	Valid id of existing payment	200 OK, returns PaymentDTO
	Get single payment by ID (Payment not found)	Non-existing id	200 OK, returns null
7	Create a shipment (Working)	Authorization(Logged in) { "orderId": 85, "address": "123 Main St, Springfield, IL, USA", "shippingMethod": "standard", "trackingNumber": 43, "cost": 50, }	{ "id": 43, "orderId": 85, "address": "123 Main St, Springfield, IL, USA", "shippingMethod": "standard", "trackingNumber": 43, "cost": 50, }

		<pre>"shippingMethod": "standard" }</pre>	<pre>"status": "shipping", "estimatedDelivery": "2025-07- 05T17:57:03.511Z", "actualDelivery": null }</pre>
	Create a shipment (non-existing ID)	<pre>{ "orderId": 999, "address": "123 Main St, Springfield, IL, USA", "shippingMethod": "standard" }</pre>	<pre>{ "message": "Order with id 999 not found", "error": "Not Found", "statusCode": 404 }</pre>
	Create a shipment (not paid order)	<pre>{ "orderId": 85, "address": "123 Main St, Springfield, IL, USA", "shippingMethod": "standard" }</pre>	<pre>{ "message": "Order must be paid before it can be shipped.", "error": "Not Found", "statusCode": 404 }</pre>
8	Create new Product(with invalid Category)	<pre>{ "product_code": "T001", "name": "mango", "description": "fruits", "imageUrl": "https://image.url", "stockQuantity": 10, "price": 99.99, "avgRating": 4.5, "categoryId": 1, "useAt": "Indoor", "material": "Metal", "sizeDimensions": "30x20 cm", "weight": 2.5, "bestUsedFor":</pre>	<pre>{ "message": "Category with ID 1 not found", "error": "Not Found", "statusCode": 404}</pre>

		"Gardening", "availability": "In Stock", "maintenanceTips": "Keep dry", "height": 100, "waterRequirement": "Moderate", "lightRequirement": "Full Sun", "soilType": "Loamy"}	
	Create new product(working)	<pre>{ "product_code": "T001", "name": "mango", "description": "fruits", "imageUrl": "https://image.url", "stockQuantity": 10, "price": 99.99, "avgRating": 4.5, "categoryId": 9, "useAt": "Indoor", "material": "Metal", "sizeDimensions": "30x20 cm", "weight": 2.5, "bestUsedFor": "Gardening", "availability": "In Stock", "maintenanceTips": "Keep dry",</pre>	<pre>{ "id": 42, "product_code": "T001", "name": "mango", "description": "fruits", "imageUrl": "https://image.url", "stockQuantity": 10, "price": 99.99, "avgRating": 4.5, "categoryName": "Hand Tool", "useAt": "Indoor", "material": "Metal", "sizeDimensions": "30x20 cm", "weight": 2.5, "bestUsedFor": "Gardening", "availability": "In Stock", "maintenanceTips": "Keep dry", "height": 100, "waterRequirement": "Moderate", "lightRequirement": "Full Sun", "soilType": "Loamy"}</pre>

		"height": 100, "waterRequirement": "Moderate", "lightRequirement": "Full Sun", "soilType": "Loamy" }	
9	Get top sold products	empty	<pre>[{ "id": 2, "name": "Pruning Shears", "price": 200, "imageurl": "https://i.postimg.cc/X7KsMT5T/Pruning-Shears.jpg", "totalquantitiesold": "22", "totalrevenue": 4400 }, { "id": 1, "name": "Hand Trowel", "price": 150, "imageurl": "https://i.postimg.cc/bJd5p3P1/Hand-Trowel.jpg", "totalquantitiesold": "14", "totalrevenue": 2100 }, { "id": 32, "name": "Tomato Plant", "price": 50, "imageurl": "https://i.postimg.cc/J48QQPFK/Tomato-Plant.jpg", "totalquantitiesold": "13", "totalrevenue": 650 }, { "id": 40, "name": "Rosemary Plant", "price": 50, "imageurl": "https://i.postimg.cc/xd85z8Z2/Rosemary-Plant.jpg", "totalquantitiesold": "10", "totalrevenue": 500 }, { "id": 5, "name": "Garden Gloves", "price": 100, "imageurl": "https://i.postimg.cc/hPmHXydr/Garden-Gloves.jpg", "totalquantitiesold": "9", "totalrevenue": 900 }, { "id": 7, "name": "Spade", "price": 350, "imageurl": "https://i.postimg.cc/Rh7LBHWc/Spade.jpg", "totalquantitiesold": "8", "totalrevenue": 2800 },</pre>

			<pre> { "id": 28, "name": "Organic Mulch", "price": 250, "imageUrl": "https://i.postimg.cc/mrpdXC9k/Organic- Mulch.jpg", "totalquantitiesold": "8", "totalrevenue": 2000 }, { "id": 8, "name": "Wheelbarrow", "price": 800, "imageUrl": "https://i.postimg.cc/bvZ0vMGY/Wheelbarrow.j pg", "totalquantitiesold": "8", "totalrevenue": 6400 }, { "id": 3, "name": "Garden Hoe", "price": 250, "imageUrl": "https://i.postimg.cc/zDymrXD4/Garden- Hoe.jpg", "totalquantitiesold": "6", "totalrevenue": 1500 }, { "id": 4, "name": "Watering Can", "price": 180, "imageUrl": "https://i.postimg.cc/ZKLxs6tm/Watering- Can.jpg", "totalquantitiesold": "5", "totalrevenue": 900 }] </pre>
10	Search product by keyword	“cherry”	<pre> [{ "id": 36, "product_code": "P005", "name": "Cherry Tree", "description": "A cherry tree that bears delicious fruits.", "imageUrl": "https://i.postimg.cc/Y9S3Q9y7/Cherry- Tree.jpg", "stockQuantity": 998, "price": 200, "avgRating": 4.5, "categoryName": "Fruit Tree", "useAt": "Outdoor", "material": null, "sizeDimensions": null, "weight": null, "bestUsedFor": "Fruit orchards", "availability": "Common", "maintenanceTips": "Requires cross-pollination for fruiting.", "height": 150, "waterRequirement": "Moderate", "lightRequirement": "Full Sun", "soilType": "Loamy" }] </pre>
11	Get product by ID(invalid ID)	Id : 78	<pre> { "message": "Product with ID 78 not found", "error": "Not Found", "statusCode": 404} </pre>
	Get product by ID(working)	Id : 41	<pre> { "id": 41, "product_code": "P010", "name": "Lavender Plant", "description": "A lavender plant known for its fragrance and essential oils.", "imageUrl": </pre>

			"https://i.postimg.cc/sxh0ZxvX/Lavender-Plant.jpg", "stockQuantity": 998, "price": 55, "avgRating": 4.5, "categoryName": "Herb", "useAt": "Both", "material": null, "sizeDimensions": null, "weight": null, "bestUsedFor": "Aromatic & medicinal use", "availability": "Common", "maintenanceTips": "Harvest flowers to encourage new growth.", "height": 50, "waterRequirement": "Low", "lightRequirement": "Full Sun", "soilType": "Well-draining"}
12	Get all products by category	categoryId : 37	[{ "id": 38, "product_code": "P007", "name": "Potato Plant", "description": "A potato plant that produces high-yield crops.", "imageUrl": "https://i.postimg.cc/HkB3nXBz/Potato-Plant.jpg", "stockQuantity": 1000, "price": 60, "avgRating": 4.5, "categoryName": "Vegetable", "useAt": "Outdoor", "material": null, "sizeDimensions": null, "weight": null, "bestUsedFor": "Vegetable farming", "availability": "Common", "maintenanceTips": "Hill soil around the base for better tubers.", "height": 50, "waterRequirement": "Moderate", "lightRequirement": "Full Sun", "soilType": "Sandy Loam" }, { "id": 39, "product_code": "P008", "name": "Pepper Plant", "description": "A pepper plant for home-grown spicy or sweet peppers.", "imageUrl": "https://i.postimg.cc/0QLVxvcq/Pepper-Plant.jpg", "stockQuantity": 1000, "price": 70, "avgRating": 4.5, "categoryName": "Vegetable", "useAt": "Both", "material": null, "sizeDimensions": null, "weight": null, "bestUsedFor": "Home gardening", "availability": "Common", "maintenanceTips": "Pinch flowers to boost fruit production.", "height": 60, "waterRequirement": "Moderate", "lightRequirement": "Full Sun", "soilType": "Loamy" }]

13	Create new category	{ "name": "fruits" }	{ "id": 38, "name": "fruits", "imageUrl": null }
	Create new category(invalid input)	{ "name": "" }	{ "statusCode": 400, "error": "Bad Request", "errors": [{ "field": "name", "message": "name should not be empty" }] }
14	Get category by id(invalid ID)	Id : 78	{ "message": "Category with ID 78 not found", "error": "Not Found", "statusCode": 404 }
	Get category by id(Working)	Id : 10	{ "id": 10, "name": "Cutting Tool", "imageUrl": null }
15	Get all categories	empty	[{ "id": 5, "name": "Plants", "imageUrl": "https://res.cloudinary.com/dkrdvrxttd/image/upload/v1745183188/categories/v3zsokwntokhb2ci0lla.webp" }, { "id": 6, "name": "Tools", "imageUrl": "https://res.cloudinary.com/dkrdvrxttd/image/upload/v1745183268/categories/bhqhzaqxk3ubgshza6ln.jpg" }, { "id": 7, "name": "Admin Category", "imageUrl": "https://res.cloudinary.com/dkrdvrxttd/image/upload/v1745666371/categories/qk9ra5fudqppyh3vg4tm.webp" }, { "id": 8, "name": "Cat 4", "imageUrl": "https://res.cloudinary.com/dkrdvrxttd/image/upload/v1745673465/categories/msvhzorbyxnm5qwcum0i.jpg" }, { "id": 9, "name": "Hand Tool", "imageUrl": null }, { "id": 10, "name": "Cutting Tool", "imageUrl": null },

16	Create Order(invalid cart status)	{ "cartID" : "83" }	{ "message": "Cart is not checked out", "error": "Bad Request", "statusCode": 400 }
	Create Order(working)	{ "cartID" : "83" }	{ "id": 87, "userId": "d6c283dd-126e-40b6-97bd-1192c08c5076", "orderItems": [{ "quantity": 2, "productName": "Plant Labels", "productPrice": 50 },], "totalPrice": 100, "status": "pending", "createdAt": "2025-06-28T17:25:03.715Z", "updatedAt": "2025-06-28T17:25:07.994Z"} }
17	Get total Statistics for orders	empty	{ "message": "Total Stats", "data": { "totalRevenue": 23586.809999999998, "totalOrders": 56 } }
18	Get last orders in 12 hours	empty	{ "message": "Last 12 Hours Orders", "data": [}

			<pre> { "hour": 3, "period": "PM", "count": 1, "hour24": 15 }, { "hour": 5, "period": "PM", "count": 1, "hour24": 17 }, { "hour": 7, "period": "PM", "count": 1, "hour24": 19 }] } </pre>
	Get orders of user	Authorization(Log ged In)	<pre> [{ "id": 35, "userId": "d6c283dd-126e-40b6-97bd-1192c08c5076", "orderItems": [], "totalPrice": 390.42, "status": "paid", "createdAt": "2025-05-04T11:57:19.660Z", "updatedAt": "2025-05-04T09:48:35.898Z" }] </pre>

			<pre> }, { "id": 36, "userId": "d6c283dd-126e-40b6-97bd-1192c08c5076", "orderItems": [], "totalPrice": 56.85, "status": "delivered", "createdAt": "2025-05-04T15:27:09.614Z", "updatedAt": "2025-05-04T09:02:17.425Z" }] </pre>
19	Cancel Order	Id : 87	<pre> { "message": "Order cancelled successfully" } </pre>
20	Delete order by id	Id : 87	<pre> { "message": "Order deleted successfully" } </pre>
21	Get order by ID(invalid ID)	Id : 87	<pre> { "message": "Order with id 87 not found", "error": "Not Found", "statusCode": 404 } </pre>
	Get order by ID(working)	Id : 81	<pre> { "id": 81, "userId": "9250b482-7106-4813-943f-feab65e5f981", "orderItems": [</pre>

			<pre> { "quantity": 1, "productName": "Wheelbarrow", "productPrice": 800 }, { "quantity": 1, "productName": "Iron Supplement", "productPrice": 350 }], "totalPrice": 1150, "status": "shipping", "createdAt": "2025-06-27T13:34:38.866Z", "updatedAt": "2025-06-27T13:34:57.213Z" } </pre>
--	--	--	---

5.2 Plant Management Package

#	Functionality	Input	Expected Output
1	Create a new plant (Working)	<pre> { "name": "Aloe Vera", "isIndoor": true, "description": "Succulent with thick leaves", "plantOverview": "Aloe vera is a succulent plant species of the genus Aloe...", "heightCm": 30, "waterNeeds": "Low", "lightNeeds": "Indirect sunlight", </pre>	<pre> { "id": 40, "name": "Aloe Vera", "isIndoor": true, "description": "Succulent with thick leaves", "plantOverview": "Aloe vera is a succulent plant species of the genus Aloe...", "heightCm": 30, "waterNeeds": "Low", </pre>

		"humidityPercent": 40, "fertilizer": "Monthly fertilizing during growing season", "temperatureCelsius": "Spring and Summer", "difficulty": "Easy", "howToGrow": "Well-drained soil, occasional watering", "soilType": "Sandy", "containerType": "Small pot with drainage", "growthTimeDays": 60, "image": "https://i.postimg.cc/B6VBYC16/Aloe-Vera.jpg", "bestSeason": "All year", "pruningTips": "Remove dead leaves occasionally", "scientificName": "Aloe barbadensis miller", "pruningGuide": "Prune lightly after flowering", "waterGuide": "Allow soil to dry between waterings", "sunlightGuide": "Needs 6 hours of sunlight daily" }	"lightNeeds": "Indirect sunlight", "humidityPercent": 40, "difficulty": "Easy", "howToGrow": "Well-drained soil, occasional watering", "soilType": "Sandy", "containerType": "Small pot with drainage", "image": "https://i.postimg.cc/B6VBYC16/Aloe-Vera.jpg", "bestSeason": "All year", "pruningTips": "Remove dead leaves occasionally", "scientificName": "Aloe barbadensis miller", "temperatureCelsius": "Spring and Summer", "fertilizer": "Monthly fertilizing during growing season", "growthTimeDays": 60, "pruningGuide": "Prune lightly after flowering", "waterGuide": "Allow soil to dry between waterings", "sunlightGuide": "Needs 6 hours of sunlight daily" }
	Create a new plant (Invalid payload)	{ "statusCode": 400, "error": "Bad Request", "errors": [{	{ "statusCode": 400, "error": "Bad Request", "errors": [{

		<pre> "field": "name", "message": "name must be a string" }, { "field": "heightCm", "message": "heightCm must be a number conforming to the specified constraints" }, { "field": "humidityPercent", "message": "humidityPercent must be a number conforming to the specified constraints" }, { "field": "image", "message": "image must be a URL address" }] } </pre>	<pre> "field": "name", "message": "name must be a string" }, { "field": "heightCm", "message": "heightCm must be a number conforming to the specified constraints" }, { "field": "humidityPercent", "message": "humidityPercent must be a number conforming to the specified constraints" }, { "field": "image", "message": "image must be a URL address" }] } </pre>
2	Get all plants (Working)	empty	<pre> [{ "id": 1, "name": "Aloe Vera", "isIndoor": true, "description": "Succulent with thick leaves", "plantOverview": "Aloe vera is a succulent plant species of the genus Aloe. It is a stemless or very short- stemmed plant growing to 60-100 cm </pre>

		(24-39 in) tall, with a spread of up to 1 m (3 ft 3 in). The leaves are thick and fleshy, green to grey-green, with some varieties showing white flecks on their upper and lower stem surfaces. The margin of the leaf is serrated and has small white teeth. The flowers are produced in summer on a spike up to 90 cm (35 in) tall, each flower being pendulous, with a yellow tubular corolla 2-3 cm (0.8-1.2 in) long.", "heightCm": null, "waterNeeds": "Low", "lightNeeds": "Indirect sunlight", "humidityPercent": null, "difficulty": "Easy", "howToGrow": "Well-drained soil, occasional watering", "soilType": "Sandy", "containerType": "Small pot with drainage", "image": https://i.postimg.cc/B6VBYC16/Aloe-Vera.jpg", "bestSeason": "All year", "pruningTips": "Remove dead leaves occasionally", "scientificName": "Aloe barbadensis miller", "temperatureCelsius": null, "fertilizer": "Monthly", "growthTimeDays": null, "pruningGuide": "Prune dead or damaged leaves at base with clean scissors. Avoid pruning healthy leaves.",
--	--	---

			"waterGuide": "Water every 2–3 weeks; let soil dry completely.", "sunlightGuide": "Bright, indirect sunlight; can tolerate some direct sun." }, { "id": 2, "name": "Basil", "isIndoor": false, "description": "Herb used in cooking", "plantOverview": "Basil (Ocimum basilicum) is an annual herb belonging to the mint family, Lamiaceae. It is native to India and other tropical regions of Asia and has been cultivated for over 5,000 years. Basil is a fragrant plant with oval-shaped leaves that are usually green or purple. The flowers are white and have a clustered shape. Its stem is square in cross-section and can grow up to two feet in height. Basil is used as an herb to flavor many dishes and can also be made into an essential oil.", "heightCm": null, "waterNeeds": "Medium", "lightNeeds": "Full sunlight", "humidityPercent": null, "difficulty": "Medium", "howToGrow": "Keep soil moist, prune regularly", "soilType": "Rich, well-drained", "containerType": "Medium-sized pot",
--	--	--	---

			<pre> "image": "https://i.postimg.cc/W1vCw2vy/Basil.jpg", "bestSeason": "Spring-Summer", "pruningTips": "Trim regularly to promote growth", "scientificName": "Ocimum basilicum", "temperatureCelsius": null, "fertilizer": "Weekly", "growthTimeDays": null, "pruningGuide": "Pinch off flower buds and top leaves weekly to promote bushiness.", "waterGuide": "Water deeply every 2–3 days; keep soil moist, not soggy.", "sunlightGuide": "6–8 hours of direct sunlight daily." },.] </pre>
3	Get plant by ID (Working)	<pre> { "id" : 1 } </pre>	<pre> { "id": 1, "name": "Aloe Vera", "isIndoor": true, "description": "Succulent with thick leaves", "plantOverview": "Aloe vera is a succulent plant species of the genus Aloe. It is a stemless or very short- stemmed plant growing to 60-100 cm (24-39 in) tall, with a spread of up to 1 m (3 ft 3 in). The leaves are thick and fleshy, green to grey-green, with some varieties showing white flecks on their upper and lower stem surfaces. The margin of the leaf is serrated and has small white teeth. The flowers are produced in summer on a spike up to 90 cm (35 in) tall, </pre>

		each flower being pendulous, with a yellow tubular corolla 2-3 cm (0.8-1.2 in) long.", "heightCm": null, "waterNeeds": "Low", "lightNeeds": "Indirect sunlight", "humidityPercent": null, "difficulty": "Easy", "howToGrow": "Well-drained soil, occasional watering", "soilType": "Sandy", "containerType": "Small pot with drainage", "image": https://i.postimg.cc/B6VBYC16/Aloe-Vera.jpg", "bestSeason": "All year", "pruningTips": "Remove dead leaves occasionally", "scientificName": "Aloe barbadensis miller", "temperatureCelsius": null, "fertilizer": "Monthly", "growthTimeDays": null, "pruningGuide": "Prune dead or damaged leaves at base with clean scissors. Avoid pruning healthy leaves.", "waterGuide": "Water every 2–3 weeks; let soil dry completely.", "sunlightGuide": "Bright, indirect sunlight; can tolerate some direct sun." }
--	--	---

	Get plant by ID (Non-existing ID)	{ "“id”" : 999 }	{ "message": "Plant with ID 999 not found", "error": "Not Found", "statusCode": 404 }
	Get plant by ID (Invalid ID)	{ "“id”" : kk }	{ "message": "Validation failed (numeric string is expected)", "error": "Bad Request", "statusCode": 400}
4	Update plant by ID (Working)	(plantId required) { "isIndoor": false }	{ "id": 1, "name": "Aloe Vera", "isIndoor": false, "description": "Succulent with thick leaves", "plantOverview": "Aloe vera is a succulent plant species of the genus Aloe. It is a stemless or very short-stemmed plant growing to 60-100 cm (24-39 in) tall, with a spread of up to 1 m (3 ft 3 in). The leaves are thick and fleshy, green to grey-green, with some varieties showing white flecks on their upper and lower stem surfaces. The margin of the leaf is serrated and has small white teeth. The flowers are produced in summer on a spike up to 90 cm (35 in) tall, each flower being pendulous, with a yellow tubular corolla 2-3 cm (0.8-1.2 in) long.", "heightCm": null, "waterNeeds": "Low", "lightNeeds": "Indirect sunlight", "humidityPercent": null,

			<pre> "difficulty": "Easy", "howToGrow": "Well-drained soil, occasional watering", "soilType": "Sandy", "containerType": "Small pot with drainage", "image": "https://i.postimg.cc/B6VBYC16/Aloe-Vera.jpg", "bestSeason": "All year", "pruningTips": "Remove dead leaves occasionally", "scientificName": "Aloe barbadensis miller", "temperatureCelsius": null, "fertilizer": "Monthly", "growthTimeDays": null, "pruningGuide": "Prune dead or damaged leaves at base with clean scissors. Avoid pruning healthy leaves.", "waterGuide": "Water every 2–3 weeks; let soil dry completely.", "sunlightGuide": "Bright, indirect sunlight; can tolerate some direct sun." } </pre>
	Update plant by ID (Invalid payload)	<pre> { "isTree": false } </pre>	<pre> { "statusCode": 400, "error": "Bad Request", "errors": [{ "field": "isTree", "message": "property isTree should not exist" }] } </pre>

			<pre> }] } </pre>
5	Delete plant by ID (Working)	<pre> { "id" : 1 } </pre>	<pre> { "success": true } </pre>
	Delete plant by ID (Non-existing ID)	<pre> { "id" : 999 } </pre>	<pre> { "message": "Plant with ID 999 not found", "error": "Not Found", "statusCode": 404 } </pre>
6	Create new garden (Working)	Authorization(logged in) <pre> { "name": "My New Garden", "latitude": 29.97, "longitude": 23.09 } </pre>	<pre> { "status": 201, "message": "Garden Created Successfully", "data": { "id": "a3fc5db7-0a37-4366-927a-be8c88ce359f", "name": "My New Garden", "latitude": 29.97, "longitude": 23.09, "userId": "afec87fc-9aa2-4f33-a383-91e037ff223b", "createdAt": "2025-06-29T19:57:57.911Z", "updatedAt": "2025-06-29T19:57:57.911Z", "deletedAt": null, "lastClimateUpdate": "2025-06-29T19:57:58.303Z" } } </pre>

			}
	Create new garden (Invalid payload)	Authorization(logged in) { "name": "My New Garden", "longitude": 23.09 }	{ "statusCode": 400, "error": "Bad Request", "errors": [{ "field": "latitude", "message": "latitude must not be more than 90" }, { "field": "latitude", "message": "latitude must not be less than -90" }, { "field": "latitude", "message": "latitude must be a number" }] }
7	Get all gardens (Working)	empty	[{ "id": "0242fa1f-a1b6-4f73-ba81-f134735174cb", "name": "My New Garden", "climate": { "climateID": 1, "temperature": 29.27, "humidity": 36,

			<pre> "weatherDescription": "clear sky", "wind": { "deg": 14, "gust": 11.71, "speed": 8.49 }, "sys": { "sunset": 1751218356, "country": "LY", "sunrise": 1751167762 }, "recordedAt": "2025-06-29T20:07:01.189Z" }, "latitude": 29.97, "longitude": 23.09, "userId": "afec87fc-9aa2-4f33-a383-91e037ff223b", "plants": [], "createdAt": "2025-06-29T20:07:01.144Z", "updatedAt": "2025-06-29T20:07:01.820Z", "deletedAt": null, "lastClimateUpdate": "2025-06-29T20:07:01.816Z" }, { "id": "d77d298c-ce2e-4b10-baad-28c9fc0e2654", "name": "My Garden", "climate": { </pre>
--	--	--	--

			<pre> "climateID": 2, "temperature": 29.27, "humidity": 36, "weatherDescription": "clear sky", "wind": { "deg": 14, "gust": 11.71, "speed": 8.49 }, "sys": { "sunset": 1751218356, "country": "LY", "sunrise": 1751167762 }, "recordedAt": "2025-06-29T20:07:05.654Z" }, "latitude": 29.97, "longitude": 23.09, "userId": "afec87fc-9aa2-4f33-a383-91e037ff223b", "plants": [], "createdAt": "2025-06-29T20:07:05.643Z", "updatedAt": "2025-06-29T20:07:05.772Z", "deletedAt": null, "lastClimateUpdate": "2025-06-29T20:07:05.769Z" }] </pre>
--	--	--	---

8	Get current user's gardens (Working)	Authorization(logged in) UserID	<pre>[{ "id": "0242fa1f-a1b6-4f73-ba81-f134735174cb", "name": "My New Garden", "climate": { "climateID": 1, "temperature": 29.27, "humidity": 36, "weatherDescription": "clear sky", "wind": { "deg": 14, "gust": 11.71, "speed": 8.49 }, "sys": { "sunset": 1751218356, "country": "LY", "sunrise": 1751167762 }, "recordedAt": "2025-06-29T20:07:01.189Z" }, "latitude": 29.97, "longitude": 23.09, "userId": "afec87fc-9aa2-4f33-a383-91e037ff223b", "plants": [], "createdAt": "2025-06-29T20:07:01.144Z",</pre>
---	--------------------------------------	--	---

			<pre> "updatedAt": "2025-06-29T20:07:01.820Z", "deletedAt": null, "lastClimateUpdate": "2025-06-29T20:07:01.816Z" }] </pre>
9	Get garden by ID (Working)	Authorization(logged in) <pre> { "id": "0242fa1f-a1b6-4f73-ba81-f134735174cb", } </pre>	<pre> { "id": "0242fa1f-a1b6-4f73-ba81-f134735174cb", "name": "My New Garden", "climate": { "climateID": 1, "temperature": 29.27, "humidity": 36, "weatherDescription": "clear sky", "wind": { "deg": 14, "gust": 11.71, "speed": 8.49 }, "sys": { "sunset": 1751218356, "country": "LY", "sunrise": 1751167762 }, "recordedAt": "2025-06-29T20:07:01.189Z" }, "latitude": 29.97, "longitude": 23.09, } </pre>

			<pre> "userId": "afec87fc-9aa2-4f33-a383-91e037ff223b", "plants": [], "createdAt": "2025-06-29T20:07:01.144Z", "updatedAt": "2025-06-29T20:07:01.820Z", "deletedAt": null, "lastClimateUpdate": "2025-06-29T20:07:01.816Z" } </pre>
10	Delete specific garden (Working)	Authorization(logged in) <pre> { "id": "0242fa1f-a1b6-4f73-ba81-f134735174cb", } </pre>	<pre> { "status": 200, "message": "Garden Deleted Successfully" } </pre>
11	Delete all gardens with their climates (Working)	empty	<pre> { "status": 200, "message": "All Gardens Deleted Successfully" } </pre>
12	Create user plant (Working)	Authorization(logged in) <pre> { "name": "Plant1", "potSizeL": "small", "plantId": 3, "gardenId": "9a29314c-6d4c-4dcd-b7ac-dd14c68ad324", "age": 1 } </pre>	<pre> { "id": "487fd1ff-e3fe-45d7-93b6-5896a9cd0e1e", "name": "Plant1", "potSizeL": "small", "userId": "afec87fc-9aa2-4f33-a383-91e037ff223b", "plantId": 3, "gardenId": "9a29314c-6d4c-4dcd-b7ac-dd14c68ad324", "lastDiagnosisId": null, "age": 1, } </pre>

			<pre> "createdAt": "2025-06-29T20:22:40.886Z", "updatedAt": "2025-06-29T20:22:40.886Z", "deletedAt": null } </pre>
	Create user plant (Invalid payload)	Authorization(logged in) <pre> { "name": "Plant1", "plantId": 3, "gardenId": "9a29314c-6d4c-4dcd-b7ac-dd14c68ad324" } </pre>	<pre> { "statusCode": 400, "error": "Bad Request", "errors": [{ "field": "potSizeL", "message": "potSizeL must be one of the following values: small, medium, large" }, { "field": "age", "message": "age must be a number conforming to the specified constraints" }] } </pre>
13	Get current user's plants (Working)	Authorization(logged in)	<pre> [{ "id": "487fd1ff-e3fe-45d7-93b6-5896a9cd0e1e", "name": "Plant1", "potSizeL": "small", "userId": "afec87fc-9aa2-4f33-a383-91e037ff223b", "plantId": 3, </pre>

			<pre> "gardenId": "9a29314c-6d4c-4dcd-b7ac-dd14c68ad324", "lastDiagnosisId": null, "age": 1, "createdAt": "2025-06-29T20:22:40.886Z", "updatedAt": "2025-06-29T20:22:40.886Z", "deletedAt": null }] </pre>
	Get current user's plants (Unauthorized)	Missing or invalid Authorization header	<pre> { "message": "Authorization header missing", "error": "Unauthorized", "statusCode": 401 } </pre>
14	Get user plant by ID (Working)	<pre> { "id": "487fd1ff-e3fe-45d7-93b6-5896a9cd0e1e", } </pre>	<pre> { "id": "487fd1ff-e3fe-45d7-93b6-5896a9cd0e1e", "name": "Plant1", "potSizeL": "small", "userId": "afec87fc-9aa2-4f33-a383-91e037ff223b", "plantId": 3, "gardenId": "9a29314c-6d4c-4dcd-b7ac-dd14c68ad324", "lastDiagnosisId": null, "age": 1, "createdAt": "2025-06-29T20:22:40.886Z", "updatedAt": "2025-06-29T20:22:40.886Z", } </pre>

			<pre>"deletedAt": null }</pre>
15	Update user plant (Working)	Authorization(logged in) <pre>{ "id": "487fd1ff-e3fe-45d7-93b6-5896a9cd0e1e", } { "age": 2 }</pre>	<pre>{ "id": "487fd1ff-e3fe-45d7-93b6-5896a9cd0e1e", "name": "Plant1", "potSizeL": "small", "userId": "afec87fc-9aa2-4f33-a383-91e037ff223b", "plantId": 3, "lastDiagnosisId": null, "age": 2, "createdAt": "2025-06-29T20:22:40.886Z", "updatedAt": "2025-06-29T20:31:28.294Z", "deletedAt": null }</pre>
16	Delete user plant (Working)	Authorization(logged in) <pre>{ "id": "487fd1ff-e3fe-45d7-93b6-5896a9cd0e1e", }</pre>	<pre>{ "status": 200, "message": "Plant Deleted Successfully" }</pre>

5.3 Care & Task Management Package

#	Functionality	Input	Expected Output
1	Create a new Schedule (No current one)	<pre>{ userPlantId: '9b4eb1c8-9035-4141-97ba-5b06f598f6a7', }</pre>	Notification sent to user informing him schedule is generated

		userId: "9b4eb1c8-9035-4141-97ba-5b06f598f6a7", plantId: 'tomato'potSizeL: 12 disease: 'healthy'stage: 'seedling', }	Care is created with tasks & task runs
2	Create a new Schedule (One is not expired yet)	{ userPlantId: '9b4eb1c8-9035-4141-97ba-5b06f598f6a7', userId: "9b4eb1c8-9035-4141-97ba-5b06f598f6a7", plantId: 'tomato'potSizeL: 12 disease: 'healthy'stage: 'seedling', }	null
3	Get all schedules with their generic tasks	Bearer Authentication Token	[{ "userId": "123", "userPlantId": "1234", "userPlant": {}, "tasks": [], "startDate": "2025-06-10T19:30:38.267Z", "endDate": "2025-06-17T19:30:38.267Z", "createdAt": "2025-06-10T19:30:38.268Z", "updatedAt": "2025-06-10T19:30:38.268Z", "deletedAt": null }]
4	Get schedules for specific user plant	- Bearer Authentication Token - Valid User plant id in path	[{ "userId": "123",

			<pre> "userPlantId": "1234", "userPlant": {}, "startDate": "2025-06-10T19:30:38.267Z", "endDate": "2025-06-17T19:30:38.267Z", "createdAt": "2025-06-10T19:30:38.268Z", "updatedAt": "2025-06-10T19:30:38.268Z", "deletedAt": null, "tasks": [] }] </pre>
5	Get Specific Schedule with its general tasks (Existing id)	• Bearer Authentication Token • Valid Schedule id in path	<pre> { "userId": "123", "userPlantId": "1234", "userPlant": {}, "tasks": [], "startDate": "2025-06-10T19:30:38.267Z", "endDate": "2025-06-17T19:30:38.267Z", "createdAt": "2025-06-10T19:30:38.268Z", "updatedAt": "2025-06-10T19:30:38.268Z", "deletedAt": null } </pre>
6	Get Specific Schedule with its general tasks (Not Existing id)	• Bearer Authentication Token	null

7	Get All Task Runs (Not specifiy Date)	Bearer Authentication Token	<pre>[{ "id": "382c24af-cda4-4f65-af37-72cb35a273ac", "type": "0", "taskId": "ec581999-3356-46af-af99-3f93827c88af", "userId": "7cf4633d-5727-49c6-b254-7bcfe070ec51", "scheduleId": "6e6f566d-cb69-45e3-8513-e13f52c37641", "value": "398", "isCompleted": false, "dueDate": "2025-06-10T03:20:00.000Z", "createdAt": "2025-06-10T19:30:38.595Z", "updatedAt": "2025-06-10T19:30:38.595Z", "deletedAt": null, "care": { "id": "6e6f566d-cb69-45e3-8513-e13f52c37641", "userId": "7cf4633d-5727-49c6-b254-7bcfe070ec51", "userPlantId": "f2e2bb41-40ce-4c4c-b692-fbfaad21f3eb", "startDate": "2025-06-10T19:30:38.267Z", "endDate": "2025-06-17T19:30:38.267Z", "createdAt": "2025-06-10T19:30:38.268Z", "updatedAt": "2025-06-10T19:30:38.268Z", } }]</pre>
---	--	---	--

			<pre> "deletedAt": null }, "userPlant": { "id": "f2e2bb41-40ce-4c4c-b692-fbfaad21f3eb", "userId": "7cf4633d-5727-49c6-b254-7bcfe070ec51", "plantId": 3, "gardenId": "5384ddd2-b6dd-47b4-b297-bd25b90f2d42", "lastDiagnosisId": 21, "age": 1, "createdAt": "2025-06-09T13:00:00.482Z", "updatedAt": "2025-06-09T14:32:29.961Z", "deletedAt": null } }] </pre>
8	Get All Task Runs (Specify Date)	• Bearer Authentication Token • Date Query	<pre> [{ "id": "382c24af-cda4-4f65-af37-72cb35a273ac", "type": "0", "taskId": "ec581999-3356-46af-af99-3f93827c88af", "userId": "7cf4633d-5727-49c6-b254-7bcfe070ec51", "scheduleId": "6e6f566d-cb69-45e3-8513-e13f52c37641", "value": "398", "isCompleted": false, </pre>

			"dueDate": "2025-06-10T03:20:00.000Z", "createdAt": "2025-06-10T19:30:38.595Z", "updatedAt": "2025-06-10T19:30:38.595Z", "deletedAt": null, "care": { "id": "6e6f566d-cb69-45e3-8513-e13f52c37641", "userId": "7cf4633d-5727-49c6-b254-7bcfe070ec51", "userPlantId": "f2e2bb41-40ce-4c4c-b692-fbfaad21f3eb", "startDate": "2025-06-10T19:30:38.267Z", "endDate": "2025-06-17T19:30:38.267Z", "createdAt": "2025-06-10T19:30:38.268Z", "updatedAt": "2025-06-10T19:30:38.268Z", "deletedAt": null }, "userPlant": { "id": "f2e2bb41-40ce-4c4c-b692-fbfaad21f3eb", "userId": "7cf4633d-5727-49c6-b254-7bcfe070ec51", "plantId": 3, "gardenId": "5384ddd2-b6dd-47b4-b297-bd25b90f2d42", "lastDiagnosisId": 21, "age": 1,
--	--	--	--

			<pre> "createdAt": "2025-06-09T13:00:00.482Z", "updatedAt": "2025-06-09T14:32:29.961Z", "deletedAt": null } }]</pre>
9	Update Task Run By Id (Valid Id)	<pre> { "isCompleted": true }</pre>	<pre> { "success": true }</pre>
	Update Task Run By Id (Not Valid Id)	<pre> { "isCompleted": true }</pre>	null
10	Token Missed in previous endpoints	Missed or wrong bearer token	401 response

5.4 Notifications Package

#	Functionality	Input	Expected Output
1	Create or Update User Device With FCM token (Valid Token)	Bearer Authentication Token	{success:true}
2	Create or Update User Device With FCM token (Not Valid Token)	Bearer Authentication Token	400 bad request

Chapter 6 Conclusions and Future Work

6.1 Supporting More Plants for Detection and Care Tasks.

- **Ai Model:** Extend the detection capabilities to support more than six plant species, Providing detailed care instructions and tasks for each.
- **Diverse Plants:** Focus on integrating a variety of plant species, including native, indoor, and outdoor plants, to enhance detection accuracy and usability.

6.2 Implementing Complete Purchasing Procedure.

- **Integration with Banks:** Although the purchasing feature is implemented, it currently lacks integration with banking systems. The system should validate the user's card and transaction status. If successful, the purchase proceeds; otherwise, an appropriate error message is displayed.
- **Bank Discounts:** Offers discounts and installation options in collaboration with partner banks, making purchases more accessible and affordable for users.

6.3 Crafting Complete Premium Experience.

- **Generous Detection Service:** provide premium users with more than three plant detections as part of the AI-based services included.
- **Generous Garden Limits:** allow premium users to manage more than three gardens. Each garden is associated with a unique location, enabling users to organize plants and care tasks per site.

6.4 Integration with LOT Devices for smart plant monitoring

- **Sensor Pairing Support:** Enable integration with moisture, light, and temperature Sensors to generate automated care insights.
- **Real-Time Alerts:** Provide real-time notification for actions like watering, light, adjustment, or reporting based on live sensor data and plant-specific needs.

6.5 Multilingual Support

- **Arabic Localization:** Translate the full UI and plant descriptions into Arabic to cater to a larger regional audience.
- **Dynamic Language Switching:** Let users toggle between supported languages without needing to reinstall or restart the app.

6.6 Community Forum & Plant Swap

- **In-App Forum:** Enable users to post questions, care tips, and showcase their plants, fostering peer learning.
- **Local Plant Swap System:** Create a geolocation-based exchange board for users to give away or trade plants, seeds, or tools.

6.7 AI-Powered Shatla Rule-Based System

- **Current System:** We currently rely on a rule-based engine that applies fixed thresholds to plant data, generating care tasks (e.g., watering, fertilizing) based on FAO guidelines. These tasks are translated into weekly doses through a postprocessor, considering factors such as disease risk.
- **Proposed Enhancement:** In future versions, we aim to introduce a machine learning model trained on real plant data to replace the current rule-based engine.

- **Data-Driven Predictions:** The model will learn from historical care data, sensor readings, and user feedback to generate personalized and dynamic care recommendations.
- **Adaptive and Self-Improving:** Unlike fixed rules, the AI model can adapt over time and improve accuracy based on new data and outcomes.
- **Personalized Care Plans:** The model will generate care tasks tailored to each user's environment, plant type, and habits, offering more precision than static thresholds.
- **IoT Integration:** Sensor data (e.g., moisture, temperature, light) will be used both for training and real-time decision-making to optimize plant health.

6.8 Pruning Care Task

- **Pruning Guidelines:** Develop specific care tasks to inform users on the proper timing, techniques, and tools for pruning different plant species.

6.8 Customer Support Tickets

- **Support Ticket System:** Implement a structured customer support feature where users can submit tickets when facing issues or problems within the app. These tickets are directed to the admin panel for review.
- **Admin Ticket Management:** Admins can view, track, and respond to submitted tickets through the dashboard, ensuring timely resolution of user-reported problems.
- **Status Tracking and Resolution:** Each ticket includes a status indicator (e.g., Open, In Progress, Resolved) to help both users and admins monitor the resolution progress and maintain clear communication.
- **Enhanced User Experience:** This feature fosters trust by providing users with a reliable method to report bugs, request help, or suggest improvements, ensuring a smoother overall app experience.